

VISION-BASED NAVIGATION PIPELINE FOR AMRs IN A DYNAMIC WAREHOUSE

MR. BOLTON ATHIT DAVIES

A PROJECT SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR
THE DEGREE OF BACHELOR OF ENGINEERING
(ROBOTICS AND AUTOMATION ENGINEERING)
INSTITUTE OF FIELD ROBOTICS
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY THONBURI
2026

Vision-Based Navigation Pipeline for AMRs in a Dynamic Warehouse

Mr. Bolton Athit Davies

A Project Submitted in Partial Fulfillment of the Requirements for
the Degree of Bachelor of Engineering (Robotics and Automation Engineering)
Institute of Field Robotics
King Mongkut's University of Technology Thonburi
2026

Project Committee

..... Chairman of Project Committee
and Project Advisor
(Dr. Rattanachai Ramaithitima)

..... Member
(.....)

..... Member
(.....)

Copyright reserved

Project Title	Vision-Based Navigation Pipeline for AMRs in a Dynamic Warehouse
Project Credits	6
Candidate	Mr. Bolton Athit Davies
Project Advisor	Dr. Rattanachai Ramaithitima
Program	Bachelor of Engineering
Field of Study	Robotics and Automation Engineering
Faculty	Institute of Field Robotics
Academic Year	2026

Abstract

Autonomous mobile robots in warehouses are today navigated with 2-D LiDAR and QR-code infrastructure, which is costly to install and inflexible when the warehouse changes. This project develops and evaluates a vision-based navigation pipeline in which a stereo-inertial visual SLAM front end is combined with a YOLO object detector that removes image features belonging to movable warehouse objects before they enter pose estimation. Two pipelines are compared: ORB-SLAM3 with detector-driven feature rejection, and VINS-Fusion with persistent-track culling feeding an RTAB-Map mapping back end. The work is organised as four experiments, each defined for both a Gazebo simulation of the warehouse and the real robot: a non-visual baseline and estimator comparison, an offline evaluation of the detector, a visual-inertial odometry evaluation with dynamic-environment filtering, and a dense-map reconstruction and map-quality evaluation.

At the reporting cutoff the four simulation experiments have been run and the four real-robot experiments remain planned. On five static recordings a five-state wheel-plus-gyroscope extended Kalman filter localises to 0.19–0.49 m aligned ATE, ahead of both visual pipelines as configured, but that ordering reverses on every recording once the filter is given the visual systems' own inertial noise model; the noise model, not the architecture, dominates. All three dead-reckoning estimators are overconfident by about two orders of magnitude. The detector reaches a pooled precision of 0.684 and a moving-object recall of 0.591 against projected simulator ground truth, with conditional class accuracy of 0.996. The dynamic-filtering hypothesis remains untested: the VINS-Fusion filter never activated, the ORB-SLAM3 treatment pairs were not input-matched, and the scripted dynamic worlds were found not to constitute a valid testbed. The RTAB-Map map of the small warehouse is geometrically faithful (occupied-cell precision 96.6%, observed-surface recall 89.5%) and drivable along an independent route; the large-warehouse map failed through tracking loss under repetitive structure. The report records these outcomes with their validity limits and sets out the remaining plan.

Keywords: Autonomous Mobile Robot / Dynamic Environment / Extended Kalman Filter / Object Detection / ORB-SLAM3 / RTAB-Map / Visual SLAM / VINS-Fusion / YOLO

หัวข้อโครงงาน	กระบวนการนำทางด้วยภาพสำหรับหุ่นยนต์เคลื่อนที่อัตโนมัติในคลังสินค้าที่เปลี่ยนแปลงตลอดเวลา
หน่วยกิต	6
ผู้เขียน	นายบอลตัน อาทิตย์ เดวิส
อาจารย์ที่ปรึกษา	ดร.รัตนชัย รมัยธิติมา
หลักสูตร	วิศวกรรมศาสตรบัณฑิต
สาขาวิชา	วิศวกรรมหุ่นยนต์และระบบอัตโนมัติ
คณะ	สถาบันวิทยาการหุ่นยนต์ภาคสนาม
ปีการศึกษา	2569

บทคัดย่อ

หุ่นยนต์เคลื่อนที่อัตโนมัติในคลังสินค้าปัจจุบันนำทางด้วยไลดาร์สองมิติร่วมกับคิวอาร์โค้ด ซึ่งมีต้นทุนติดตั้งสูงและปรับเปลี่ยนได้ยากเมื่อผังคลังสินค้าเปลี่ยนไป โครงงานนี้พัฒนาและประเมินกระบวนการนำทางด้วยภาพ โดยผสานระบบระบุตำแหน่งและสร้างแผนที่ด้วยภาพจากกล้องสเตอริโอและหน่วยวัดความเฉื่อยเข้ากับตัวตรวจจับวัตถุ YOLO ซึ่งคัดจุดเด่นบนวัตถุเคลื่อนย้ายได้ออกก่อนการประมาณตำแหน่ง และเปรียบเทียบไปป์ไลน์สองรูปแบบ ได้แก่ ORB-SLAM3 ที่ปฏิเสธจุดเด่นตามผลตรวจจับ และ VINS-Fusion ที่ตัดเส้นทางจุดเด่นคงค้างแล้วส่งต่อให้ RTAB-Map สร้างแผนที่ การประเมินจัดเป็นการทดลองสี่รายการ แต่ละรายการทำทั้งในสถานการณ์จำลองและบนหุ่นยนต์จริง ได้แก่ การเปรียบเทียบตัวประมาณค่าพื้นฐานที่ไม่ใช้ภาพ การประเมินตัวตรวจจับวัตถุแบบออฟไลน์ การประเมินการวัดระยะทางด้วยภาพและความเฉื่อยร่วมกับการกรองวัตถุพลวัต และการประเมินคุณภาพแผนที่

ณ วันตัดยอดรายงาน การทดลองในสถานการณ์จำลองทั้งสี่รายการเสร็จสิ้น ส่วนการทดลองบนหุ่นยนต์จริงยังอยู่ในแผน จากข้อมูลบันทึกฉากคงที่ห้าชุด ตัวกรองคาลมานขยายที่รวมล้อและไจโรสโคปให้ค่าความคลาดเคลื่อนเส้นทางสัมบูรณ์ 0.19 ถึง 0.49 เมตร ดีกว่าไปป์ไลน์ด้วยภาพทั้งสองตามค่าตั้งที่ใช้ แต่ลำดับกลับด้านในทุกชุดข้อมูลเมื่อให้ตัวกรองใช้แบบจำลองสัญญาณรบกวนชุดเดียวกับระบบภาพ แสดงว่าแบบจำลองสัญญาณรบกวนมีผลมากกว่าสถาปัตยกรรม ตัวตรวจจับวัตถุให้ความแม่นยำรวม 0.684 และความครบถ้วนของวัตถุเคลื่อนที่ 0.591 โดยจำแนกประเภทถูกต้อง 0.996 สมมติฐานการกรองวัตถุพลวัตยังไม่ได้รับการทดสอบที่ใช้ได้ เนื่องจากตัวกรองของ VINS-Fusion ไม่ทำงาน คู่ทดลองของ ORB-SLAM3 ได้รับข้อมูลนำเข้าไม่เท่ากัน และโลกจำลองพลวัตยังไม่เป็นสนามทดสอบที่ใช้ได้ แผนที่ RTAB-Map ของคลังสินค้าขนาดเล็กถูกต้องทางเรขาคณิต (ความแม่นยำร้อยละ 96.6 ความครบถ้วนของพื้นผิวที่สังเกตได้ร้อยละ 89.5) และใช้ขับผ่านเส้นทางอิสระได้ ส่วนแผนที่คลังสินค้าขนาดใหญ่ล้มเหลวจากการสูญเสียการติดตามในโครงสร้างที่ซ้ำกัน

คำสำคัญ : การตรวจจับวัตถุ / การระบุตำแหน่งและสร้างแผนที่ด้วยภาพ / คลังสินค้าพลวัต / ตัวกรองคาลมานขยาย / หุ่นยนต์เคลื่อนที่อัตโนมัติ

ACKNOWLEDGEMENTS

[To be written by the author. The template reserves this page for thanks to the project advisor, the examination committee, Gensurv Robotics, the Institute of Field Robotics, and any funding source. Nothing has been drafted here on the author's behalf.]

CONTENTS

PAGE

ABSTRACT ii

THAI ABSTRACT iii

ACKNOWLEDGEMENTS iv

CONTENTS v

LIST OF TABLES viii

LIST OF FIGURES x

LIST OF SYMBOLS xii

LIST OF TECHNICAL VOCABULARY AND ABBREVIATIONS xiii

CHAPTER

1. INTRODUCTION 1
- 1.1 Background and Significance . . . 1
- 1.2 Objectives . . . 1
- 1.3 Expected Benefits . . . 2
- 1.4 Scope . . . 2
- 1.5 Work Plan . . . 2

2. THEORY AND RELATED WORK 4
- 2.1 AMR Navigation Systems . . . 4
  - 2.1.1 Logistics management and the historical bottleneck . . . 4
  - 2.1.2 The era of automated guided vehicles (AGVs) . . . 5
  - 2.1.3 The transition to autonomous mobile robots (AMRs) . . . 5
- 2.2 Visual Simultaneous Localization and Mapping (Visual SLAM) . . . 5
  - 2.2.1 Components of a visual SLAM system . . . 6
  - 2.2.2 ORB-SLAM3 . . . 7
  - 2.2.3 VINS (Visual-Inertial System) . . . 8
  - 2.2.4 RTAB-Map back end and memory management . . . 10
- 2.3 Real-Time Object Detection . . . 11
  - 2.3.1 The role of object detection in semantic understanding . . . 11
  - 2.3.2 The YOLO (You Only Look Once) network architecture . . . 11
  - 2.3.3 Performance evaluation and edge-processor limits . . . 13
- 2.4 RY-SLAM: A Robust YOLO-Based Real-Time Visual SLAM Solution for Dynamic Environments . . . 13
  - 2.4.1 Concept and architecture of RY-SLAM . . . 13
  - 2.4.2 Dynamic feature masking mechanism . . . 13
  - 2.4.3 Relevance to this project . . . 15

3. METHODOLOGY 16
- 3.1 Problem, Requirements and Scope . . . 16

CONTENTS (Cont'd)

PAGE

3.2 System Architecture . . . 17
3.2.1 Control-loop view and data rates . . . 17
3.2.2 Semantic visual SLAM data flow . . . 17
3.2.3 The two navigation pipelines compared . . . 19
3.3 The Pipelines as Implemented . . . 20
3.3.1 Dynamic feature masking . . . 20
3.3.2 ORB-SLAM3 and semantic feature rejection . . . 21
3.3.3 VINS-Fusion front end . . . 22
3.3.4 RTAB-Map mapping and localization back end . . . 24
3.3.5 YOLO detector node, mask construction and synchronisation . . . 26
3.4 Simulation Platform . . . 26
3.4.1 Computing and software environment . . . 26
3.4.2 Simulated sensor rig and platform . . . 28
3.4.3 Simulation fidelity and its limits . . . 30
3.4.4 Coordinate-frame contract . . . 33
3.4.5 Simulation environments and controlled factors . . . 33
3.4.6 Realistic logistics layouts . . . 35
3.4.7 Dataset registry . . . 43
3.5 Real Platform . . . 46
3.5.1 AMR mobile robot testbed . . . 46
3.5.2 Sensors and computing unit . . . 46
3.5.3 Real sensor rig and calibration limits . . . 48
3.5.4 Test site and experimental variables . . . 50
3.6 The Four Experiments . . . 50
3.6.1 Experiment 1: Non-visual baseline and estimator comparison . . . 50
3.6.2 Experiment 2: Offline YOLO detector performance . . . 54
3.6.3 Experiment 3: Visual-inertial odometry with dynamic-environment filtering . . . 56
3.6.4 Experiment 4: Dense map reconstruction and map-quality evaluation 57
3.7 Evaluation Metrics and Analysis . . . 57
3.7.1 Trajectory evaluation . . . 57
3.7.2 Computational performance . . . 58
3.7.3 Collection protocol, validity and stored artifacts . . . 59
3.7.4 Reproducibility controls . . . 60
3.8 Planned Versus Implemented . . . 60

4. EXPERIMENTS AND RESULTS 63
4.1 Experiment 1: Non-Visual Baseline and Estimator Comparison . . . 63
4.1.1 Simulation . . . 63
4.1.2 Real robot . . . 74
4.2 Experiment 2: Offline YOLO Detector Performance . . . 74
4.2.1 Simulation . . . 74
4.2.2 Real robot . . . 78
4.3 Experiment 3: Visual-Inertial Odometry With Dynamic-Environment Filtering . . . 78
4.3.1 Simulation . . . 78
4.3.2 Real robot . . . 88
4.4 Experiment 4: Dense Map Reconstruction and Map-Quality Evaluation . . . 88
4.4.1 Simulation . . . 88

CONTENTS (Cont'd)

PAGE

4.4.2 Real robot 94
4.5 Cross-Experiment Discussion, Simulation-to-Real Comparison and Limitations 95
4.5.1 Simulation against the real robot 95
4.5.2 Where each pipeline is weak, block by block 95

5. CONCLUSION AND RECOMMENDATIONS **106**
5.1 Conclusions by Experiment 106
5.1.1 Experiment 1: is vision earning its place? 106
5.1.2 Experiment 2: how good is the detector? 106
5.1.3 Experiment 3: does filtering help? 106
5.1.4 Experiment 4: is the map usable? 107
5.2 Limitations and Threats to Validity 107
5.3 Recommendations and Next Steps 107

REFERENCES **109**

CURRICULUM VITAE **111**

LIST OF TABLES

TABLE **PAGE**

1.1 Work plan for the 2026 (B.E. 2569) academic year. 3

2.1 Comparison of object-detection model architectures. 12

3.1 The four experiments and the architecture block each one tests. 17
3.2 Implementation status at the reporting cutoff. 18
3.3 Design comparison of the two pipeline architectures, as proposed. . . . 21
3.4 ORB-SLAM3 configuration at the cutoff. 23
3.5 VINS-Fusion simulation configuration at the cutoff, from `vins_fusion_ros2/config/wil_sim/stereo_imu.yaml`. 25
3.6 Computing and software environment. 27
3.7 Simulated sensor configuration. 28
3.8 Inertial noise parameters supplied to the visual-inertial estimators, beside the values measured from the recorded IMU stream in section 3.6.1. . . 29
3.9 Platform geometry and drivetrain limits used by the wheel-odometry and EKF baselines. 29
3.10 Noise injected by the simulator, from the `<imu>` and `<camera>` sensor blocks of the robot model. 30
3.11 Gazebo physics configuration, identical in all 26 world files. 31
3.12 Measured wheel slip on the five multi-sensor recordings. 32
3.13 What the visual estimators are configured to expect, against what the simulator injects. 32
3.14 Ground models and the floor conditions they produce. 35
3.15 What each dataset contains. 37
3.16 Controlled experimental factors available in the simulation environment, with the levels realised at the reporting cutoff. 38
3.17 Zones of the two realistic worlds and the number of object instances each holds. 41
3.18 What `dataset_map_realistic` contains, read from the bag. 43
3.19 Complete dataset registry at the reporting cutoff, across both storage locations. 44
3.20 Specification of the AC-IMX390-H190 camera. 47
3.21 Summary of the hardware technical specification. 49
3.22 Real sensor rig, and the calibration status of each quantity. 49
3.23 Sensor noise and scale measured from each bag by `proprio_estimator.py calibrate`. 52
3.24 Implemented non-visual estimators. 52
3.25 Parameters of the detection evaluation. 55
3.26 Principal stored artifacts. 59
3.27 Planned-versus-implemented deviations at the reporting cutoff. 61
3.28 Original objectives (section 1.2) against the work reported here. 61

LIST OF TABLES (Cont'd)

PAGE

4.1 Status of the eight result slots at the reporting cutoff. 63
4.2 Conditions for the estimator comparison. 64
4.3 Loop-closing activity in the enabled ORB-SLAM3 configuration (`logging_20260916_01`). 65
4.4 Estimator comparison on the five `allsensor` bags, which span four worlds and both floor conditions; they are not repeats of one another except for `allsensor_000` and `allsensor_001`. 66
4.5 Wheel-odometry heading drift against the fitted yaw-rate scale. 67
4.6 The EKF's heading error follows the quality of the wheel yaw rate it uses to observe its gyroscope bias. 69
4.7 Covariance consistency of the non-visual estimators. 70
4.8 EKF accuracy with an in-sample and an out-of-sample noise model. . . 71
4.9 Which of the four IMU noise terms costs the EKF its accuracy. 73
4.10 Offline YOLO performance against projected Gazebo object ground truth. 75
4.11 Object-class assignment conditional on successful IoU association, pooled across the four dynamic datasets. 77
4.12 Relogged experiment progress and validity. 79
4.13 Anomaly catalogue for the relogged campaign. 80
4.14 Trajectory error against rosbag ground truth, median (IQR). 81
4.15 Relative pose error at $\Delta = 1$ s, median (IQR) over runs. 82
4.16 Jump-free translational RPE against interval, median over runs, in metres. 83
4.17 Truth-based trajectory error for every repeat of the relogged campaign. . 84
4.18 Dynamic-run capacity and resource measurements, mean $\pm$ sample SD, $n = 12$ per row. 86
4.19 Conditions for the dense-map evaluation. 89
4.20 Stored contents of `map/map_tiny.db`. 89
4.21 Occupied-cell geometry of `map/map_tiny.db` against `small_warehouse_static_tinymap.world` at 0.25 m tolerance. 90
4.22 Drivability of `map/map_tiny.db` under the true 0.83×0.50 m nav2 footprint, evaluated against two different routes. 91
4.23 Per-block evidence for **ORB-SLAM3** in the static condition, figure 4.13. 98
4.24 Per-block evidence for **VINS-Fusion** in the static condition, figure 4.13. 99
4.25 Per-block evidence for the **EKF wheel + IMU** baseline, figure 4.13. . . 100
4.26 Per-block evidence for **ORB-SLAM3** in the dynamic condition, figure 4.14. 102
4.27 Per-block evidence for **VINS-Fusion** in the dynamic condition, figure 4.14. 103
4.28 Limitations of the results reported in this chapter, and what each one prevents. 105

LIST OF FIGURES

FIGURE **PAGE**

1.1 Scope of the project within the AMR navigation loop. 2
1.2 Top view of the mock warehouse test field on the first floor of the Gensurv building. 3

2.1 The eight principal wastes of lean manufacturing. 4
2.2 Three warehouse robots with different navigation technologies: the orange robot uses QR-code navigation, the pallet truck in the centre uses SLAM navigation without floor infrastructure, and the white robot uses magnetic-tape guidance. 5
2.3 An autonomous driverless transport vehicle navigating precisely in the BMW Group press plant at Regensburg. 6
2.4 The indirect (feature-based) method common to SLAM systems: a front end producing pseudo-measurements and loop-closure constraints, and a back end performing geometric SLAM by factor-graph optimisation [2]. 6
2.5 Main components of ORB-SLAM3: the tracking, local mapping and loop and map-merging threads around the Atlas multi-map representation and the DBoW2 keyframe database [4]. 7
2.6 Main components of monocular VINS: measurement preprocessing, initialisation, sliding-window visual-inertial odometry with relocalization, and global pose-graph optimisation and reuse [5]. 8
2.7 Main components of RTAB-Map: sensor synchronisation, short-term and working memory, long-term memory, loop-closure and proximity detection, graph optimisation and global map assembly [7]. 10
2.8 Architecture of RY-SLAM: a YOLOv8-seg dynamic-object detection stage feeding a semantic mask into the ORB-SLAM3 tracking thread ahead of the Atlas multi-map system [8]. 14
2.9 Detail of the semantic-segmentation framework in RY-SLAM and its multi-threaded operation: ONNX Runtime inference producing the dynamic mask, the neighbourhood test that deletes feature points, and the viewer outputs [8]. 14

3.1 Implemented experimental architecture. 18
3.2 Pipeline 1, YOLO26 + ORB-SLAM3, as designed: stereo images and IMU are synchronised, tracked and passed to local mapping, loop closing and the Atlas map; the detector's boxes enter the tracking stage. 20
3.3 Pipeline 2, YOLO26 + VINS + RTAB-Map, as designed: VINS tracking, initialisation and sliding-window optimisation produce odometry, RTAB-Map builds the map from that odometry and the stereo images. 20
3.4 ORB-SLAM3 pipeline and the semantic-mask insertion point. 22
3.5 VINS-Fusion pipeline and its two semantic-mask insertion points. . . . 24
3.6 Simulated Ackermann platform, drawn to scale from the model definition. 28

LIST OF FIGURES (Cont'd)

PAGE

3.7 Coordinate-frame contract used by the composed RTAB-Map pipeline. . 33
3.8 Baked occupancy truth of the three large-warehouse object arrangements, each 42.2 m square at 0.05 m per cell with origin $(-21.1, -21.1)$ m. . . 34
3.9 The two recorded floor conditions, rendered across the full 42.2 m floor at 0.05 m per pixel. 34
3.10 Every simulation dataset as a scene: floor treatment, object layout and ground-truth route in one view. 36
3.11 Plan of the U-shape realistic world: zones, carrier jobs and dock doors, drawn by the generator from the tables that write the world file. 39
3.12 Plan of the I-shape realistic world, same conventions as the U-shape plan; dispatch docks are on the south wall. 40
3.13 Baked occupancy truth of the two realistic worlds, same frame and resolution as the earlier arrangements. 43
3.14 Scene of `dataset_map_realistic`: the U-shape realistic layout and the recorded ground-truth route. 45
3.15 Dimensions of the mock AMR used to test the navigation system: 57 cm long, 40.5 cm wide and 47.5 cm high to the top of the sensor mast. . . . 46
3.16 Hardware components of the mock AMR: steering servo and traction motor, BLDC drive and microcontroller on CAN, the NRU-51V computer, power supply, the AC-IMX390 stereo pair and IMU, and the Wi-Fi link to the smartphone joystick. 47
3.17 Dimensions of the AC-IMX390 camera family in millimetres; the H190 variant at the bottom is the one used. 47
3.18 The Neousys NRU-51V (NVIDIA Jetson Xavier NX), showing the four FAKRA camera ports. 48
3.19 Dimensions of the Neousys NRU-51V in millimetres. 48

4.1 Estimator comparison on `dataset_allsensor_001`, world frame. 68
4.2 Error against claimed uncertainty for `dataset_allsensor_001`. . 71
4.3 Per-dataset and pooled detector scores and box-area coverage. 76
4.4 Representative validation overlay used before accepting the offline metrics. 76
4.5 Object-class confusion conditional on class-agnostic IoU association. . 77
4.6 Run-level anomaly diagnostics. 81
4.7 Representative repeated-run trajectory diagnostic for `dataset_dynamic_nofloortexture_00_000/logging_20260916_01`, in the same visual format as the existing comparison figures. 85
4.8 Per-run robustness and keyframe observations for the dynamic datasets. 86
4.9 Raw dynamic-run capacity and process-resource values. 86
4.10 Module-latency distributions for dynamic datasets. 87
4.11 Dense-map evaluation of `map/map_tiny.db`. 92
4.12 The same database scored against the run that built it, `dataset/dataset_map_tiny`. 93
4.13 Per-block performance in the **static** condition, over the five `allsensor` datasets. 97
4.14 Per-block performance in the **dynamic** condition, over the twelve relogged dynamic runs of each estimator. 101

LIST OF SYMBOLS

SYMBOL		UNIT
a, a_x	longitudinal acceleration measured by the IMU	$\mathrm{m\,s^{-2}}$
B	wheel track (lateral wheel separation)	m
B_k	the k-th detected bounding box $\{x_k, y_k, w_k, h_k, c_k, P_k\}$	–
b_a, b_g, b_ω	accelerometer, gyroscope and yaw-rate biases	$\mathrm{m\,s^{-2}}$, $\mathrm{rad\,s^{-1}}$
Δ	relative-pose-error interval	s
$\mathbf{e}$	estimation error vector $[\Delta x, \Delta y, \Delta\theta]$	m, m, rad
E_i	relative pose error transform at pose i	–
$\mathbf{F}$, $\mathbf{G}$	state-transition and noise-input Jacobians	–
$\mathscr{F}_{\text{static}}$	set of image features surviving the dynamic mask	–
f_x, f_y, c_x, c_y	pinhole intrinsics	pixel
g	gravitational acceleration	$\mathrm{m\,s^{-2}}$
$\mathbf{H}$	measurement matrix	–
$\mathbf{K}$	Kalman gain	–
L	wheel base	m
n	number of experimental units (recordings)	–
$\mathbf{P}$	state covariance	–
P, Q	estimated and reference pose sequences (RPE definition)	–
p_x, p_y	planar position	m
$\mathbf{Q}$, $\mathbf{R}$	process- and measurement-noise covariances	–
R, T	rotation matrix and rigid transform	–
r	wheel radius	m
σ	standard deviation of the quantity it subscripts	as subscript
θ, θ_m	heading and midpoint heading	rad
τ	bias correlation time	s
v	longitudinal speed	$\mathrm{m\,s^{-1}}$
ω	yaw rate	$\mathrm{rad\,s^{-1}}$
$\mathbf{x}$	EKF state $[p_x, p_y, \theta, v, b_\omega]^\top$	–
χ	VINS sliding-window state vector	–

LIST OF TECHNICAL VOCABULARY AND ABBREVIATIONS

AGV	= Automated Guided Vehicle
AMR	= Autonomous Mobile Robot
ANEES	= Average Normalised Estimation Error Squared
AP, mAP	= Average Precision, mean Average Precision
ATE	= Absolute Trajectory Error
BA	= Bundle Adjustment
CPU, GPU	= Central / Graphics Processing Unit
DoF	= Degrees of Freedom
EKF	= Extended Kalman Filter
FOV, HFOV	= (Horizontal) Field of View
FN, FP, TP	= False Negative, False Positive, True Positive
GT	= Ground Truth
IMU	= Inertial Measurement Unit
IoU	= Intersection over Union
IQR	= Interquartile Range
KLT	= Kanade–Lucas–Tomasi optical-flow tracker
LC	= Loop Closure
LiDAR	= Light Detection and Ranging
LTM, STM, WM	= Long-Term, Short-Term and Working Memory (RTAB-Map)
MAP	= Maximum a Posteriori
NEES	= Normalised Estimation Error Squared
NMS	= Non-Maximum Suppression
QoS	= Quality of Service (ROS 2 middleware)
RMSE	= Root-Mean-Square Error
ROS	= Robot Operating System
RPE	= Relative Pose Error
RSS	= Resident Set Size
RTAB-Map	= Real-Time Appearance-Based Mapping
SfM	= Structure from Motion
SGBM	= Semi-Global Block Matching
SLAM	= Simultaneous Localization and Mapping
TF	= ROS transform tree
VIO	= Visual-Inertial Odometry
VINS	= Visual-Inertial System (VINS-Mono / VINS-Fusion)
YOLO	= You Only Look Once object detector

CHAPTER 1 INTRODUCTION

1.1 Background and Significance

Autonomous mobile robots (AMRs) play a central role in raising the efficiency of warehouse logistics. Gensurv Robotics currently relies on 2-D LiDAR sensing combined with QR-code navigation. That solution carries business limitations: LiDAR sensors are expensive, and the approach is inflexible because the company must spend money and time modifying the warehouse infrastructure to install artificial reference markers. Moving from LiDAR to vision-based navigation, which uses lower-cost camera sensors and yields richer information about the environment, is therefore an option that directly addresses the company's cost problem.

Bringing cameras and simultaneous localization and mapping from images (visual SLAM) into real warehouse use nevertheless raises a substantial academic challenge. Most standard visual SLAM algorithms were developed under the assumption that the environment is static. Saputra et al. [1] state clearly that when a SLAM system is deployed in a dynamic environment, such as a warehouse in which goods and obstacles are continually moved, its accuracy degrades severely, because the system draws features from moving objects into its pose computation. Cadena et al. [2] further stress that the greatest challenges facing SLAM today are robustness and lifelong mapping: the robot must be able to separate permanent structure from dynamic objects.

To address navigation in a dynamic environment, this project proposes the development of a vision-based navigation pipeline centred on integrating the YOLO26 deep-learning model to detect and filter dynamic objects out of the localization system. The project develops and compares the performance of two navigation-system architectures: (1) operation with the ORB-SLAM3 framework, and (2) operation with VINS combined with RTAB-Map, in order to find the most robust approach for a warehouse.

In addition, replacing an active sensor (LiDAR) with a passive one (a camera) may make the system sensitive to indoor lighting conditions. The project therefore designs an experiment that simulates the environment and the light intensity at different times of day, so that data can be collected and it can be assessed empirically whether warehouse lighting significantly affects the performance of the visual SLAM system. That assessment will be important input to the company's long-term development and improvement of its navigation system.

1.2 Objectives

1. Develop a visual-inertial odometry localization system with lifelong mapping, so that the warehouse map is updated in real time.
2. Develop image enhancement for low-light environments with the EnlightenGAN model.
3. Develop perception of obstacles and moving objects (dynamic objects) through deep-learning object detection.
4. Develop lane detection with PolyNET and lane tracking with a PID controller.
5. The integrated system must be able to build a 2-D cost map and localize the robot with an error of less than 10 cm under differing light intensities.

1.3 Expected Benefits

1. Help the company reduce the cost of modifying warehouse infrastructure for AMR navigation.
2. Obtain a navigation pipeline that is robust to a dynamic environment.
3. Allow the pipeline to be plugged into the control and motion-planning systems for real use on warehouse robots.

1.4 Scope

1. The main tests are carried out only on a small mobile robot computing on an NRU-51V processing unit and using AC-IMX390-H190 cameras.
2. Within the AMR navigation system, the author's work is framed only within the visual SLAM, object prediction and lane-tracking controller blocks (red frame) shown in figure 1.1.
3. The integrated system is tested only in a mock warehouse arena with changing mock goods, a walkway with a 5° slope, five stacks of goods of four boxes each, and a starting point at the grey square at the lower right of figure 1.2. The positions of the mock warehouse contents are changed between tests to simulate a dynamic environment.

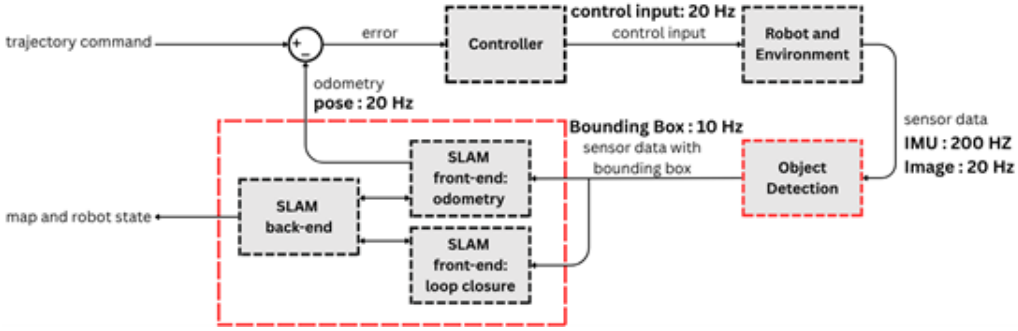


Figure 1.1 Scope of the project within the AMR navigation loop. The red dashed frame encloses the blocks this project develops: the SLAM front end (odometry and loop closure), the SLAM back end (map and robot state) and the object-detection block that supplies bounding boxes. The controller and the robot are existing components. Rates are those of the target system: control input 20 Hz, odometry pose 20 Hz, bounding boxes 10 Hz, IMU 200 Hz, images 20 Hz.

1.5 Work Plan

Table 1.1 reproduces the project work plan of the proposal. Task numbering follows the original plan, which has no item 5.

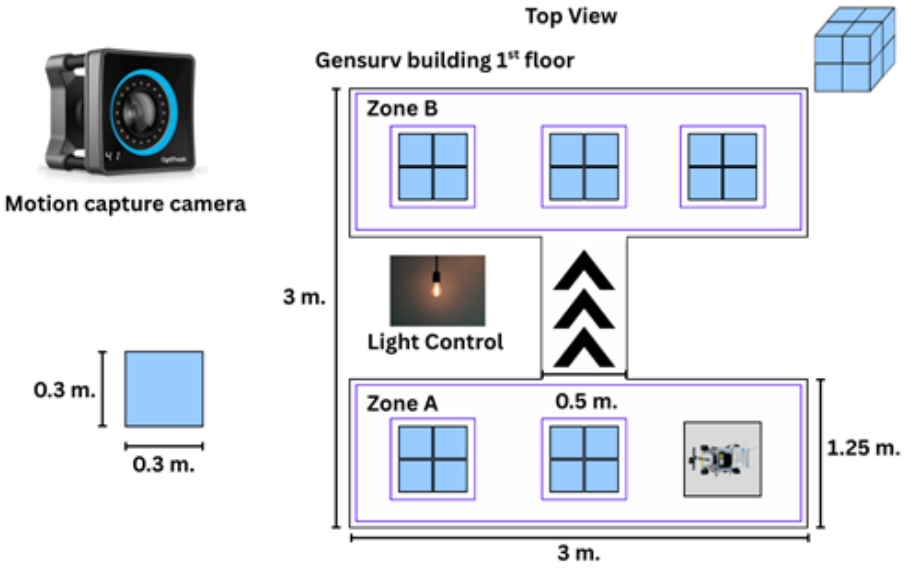


Figure 1.2 Top view of the mock warehouse test field on the first floor of the Gensurv building. Zone A and Zone B are each 3 m square and are joined by a walkway; the robot starts at the grey square in Zone A. Goods are 0.3 m cubes stacked in piles, a light-control fixture sets the illumination, and motion-capture cameras around the field provide the ground-truth reference.

Table 1.1 Work plan for the 2026 (B.E. 2569) academic year. Shaded cells mark the months allocated to each task.

Task	Jun	Jul	Aug	Sep	Oct	Nov	Dec
1. Study the concepts and theory of vision-based navigation	■						
2. Search and analyse information on visual SLAM algorithms, low-light image enhancement (EnlightenGAN), object detection (YOLO26) and lane tracking (PolyNET)	■						
3. Design the structure and software pipeline that joins the hardware and software systems		■					
4. Specify and connect the AC-IMX390-H190 camera sensors with the NRU-51V edge computing unit on the mobile robot		■					
6. Integrate all subsystems under the AMR navigation framework			■				
7. Prepare the mock warehouse area (Zone A and Zone B) with a 5° slope and arrange mock goods to create a dynamic environment			■				
8. Run the integrated system experiment (live test) with the robot moving at 1.5 m s^{-1} under scheduled changes in lighting				■			
9. Collect the test data and evaluate system performance on the defined metrics: trajectory error (ATE, RPE), system latency, and detection accuracy (IoU, mAP)					■		
10. Summarise the work, discuss the results and produce the complete engineering project report						■	■

CHAPTER 2 THEORY AND RELATED WORK

In carrying out the project "Vision-Based Navigation Pipeline for AMRs in Dynamic Warehouse", the author's aim is to develop and evaluate only the navigation pipeline, within the scope defined in section 1.4. The project does not set out to build a complete autonomous robot from scratch; it integrates navigation software into the existing architecture of an autonomous mobile robot (AMR). To make that development effective, the author studied and collected theory, documents and research on the following principal topics.

2.1 AMR Navigation Systems

Autonomous mobile robots are among the technologies transforming industry in the digital era, particularly in manufacturing, warehousing and materials handling. With their capacity to learn, their flexibility and their high precision, AMRs reduce cost and raise efficiency by a large step. This section reviews the properties, technology and use cases of AMRs to make the potential of the innovation clear.

2.1.1 Logistics management and the historical bottleneck

In a traditional warehouse, most wasted expenditure arises in the logistics of moving material from one point to another, work that exhausts staff who must walk while continuously carrying loads. These movements produce "motion waste", one of the eight principal wastes of lean manufacturing (figure 2.1). To stay competitive and cut logistics costs, smart warehouses have begun to bring autonomous robots in to work alongside people and reduce that time and waste.



Figure 2.1 The eight principal wastes of lean manufacturing. Source: https://teachme-biz.com/th/blog/reduce-waste/.

2.1.2 The era of automated guided vehicles (AGVs)

Research and development of autonomous mobile robots began in the 1950s with automated guided vehicles. AGV systems have an important limitation: the vehicle depends on infrastructure installed in the warehouse beforehand for navigation, such as magnetic tape, QR codes or artificial landmarks (figure 2.2). AGVs can also only follow fixed, pre-planned routes, so the system lacks spatial flexibility, and if the warehouse environment changes the AGV cannot adapt immediately.

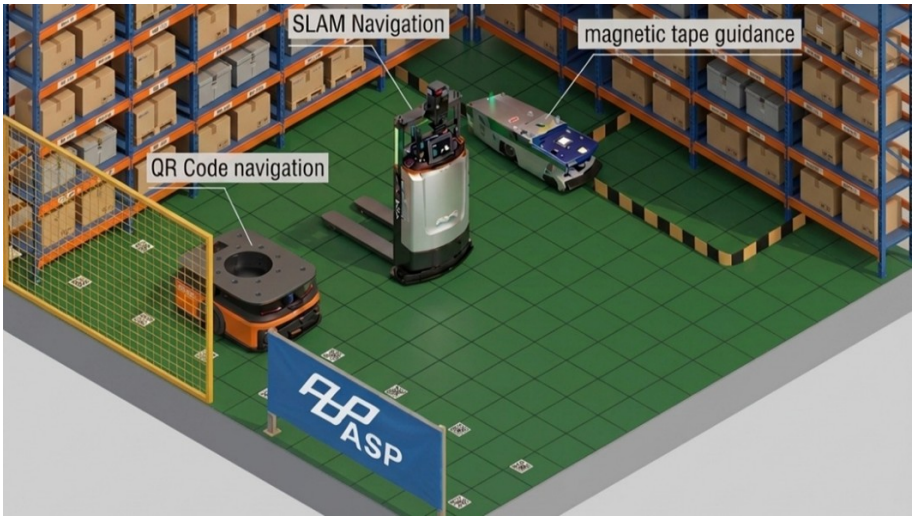


Figure 2.2 Three warehouse robots with different navigation technologies: the orange robot uses QR-code navigation, the pallet truck in the centre uses SLAM navigation without floor infrastructure, and the white robot uses magnetic-tape guidance. Source: https://www.linkedin.com/pulse/agv-amr-navigation-technologies-which-one-drives-your-abdul-rahman-6pjyc.

2.1.3 The transition to autonomous mobile robots (AMRs)

AMRs were developed to overcome the limitations of AGVs. The essential difference is that an AMR can make its own decisions and does not need pre-installed external infrastructure to navigate. Recent hardware advances, particularly in imaging sensors (2-D/3-D cameras and LiDAR) together with high-performance processors, let an AMR perceive and assess its surroundings accurately. The distinguishing capability that lets AMRs replace AGVs is operation in a dynamic environment: when an AMR detects an obstacle on its route it can compute alternative routes to avoid the obstacle or another robot automatically, without the user redesigning the route.

With this capacity for independent decision-making and navigation, large industrial users have moved to AMRs widely. One example is the deployment of autonomous driverless transport vehicles in the press plant of the BMW Group at Regensburg [3], whose vehicles navigate and manoeuvre precisely through busy, complex areas of a real factory (figure 2.3), showing that AMRs have fully overcome the limits of the older navigation systems.

2.2 Visual Simultaneous Localization and Mapping (Visual SLAM)

In the operation of an AMR, localization is the heart of what lets the robot move accurately. The widely accepted and used method is SLAM (simultaneous localization and mapping).



Figure 2.3 An autonomous driverless transport vehicle navigating precisely in the BMW Group press plant at Regensburg. Source: https://www.press.bmwgroup.com/global/video/detail/PF0009726/.

A SLAM navigation system extracts features present in the environment to build a map; the software gathers information from the robot's current position and the positions it has passed through to estimate a probability distribution and predict the robot's most likely position.

2.2.1 Components of a visual SLAM system

This project concentrates on visual SLAM, mapping and localization with image sensors, to reduce the cost of LiDAR sensors. Following the project's own data-flow diagram, a visual SLAM architecture has three main parts (figure 2.4):

1. **SLAM front end (visual odometry)**: receives camera images and extracts features to estimate the robot's relative motion and pose over short intervals.
2. **SLAM back end**: refines and corrects the accumulated drift of the front-end estimate using graph optimisation, to obtain the most accurate map and pose.
3. **Loop closure**: the process by which the system recognises a place the robot has visited before; when the robot returns, the current image is matched to a past one and the whole map is corrected to reduce accumulated error.

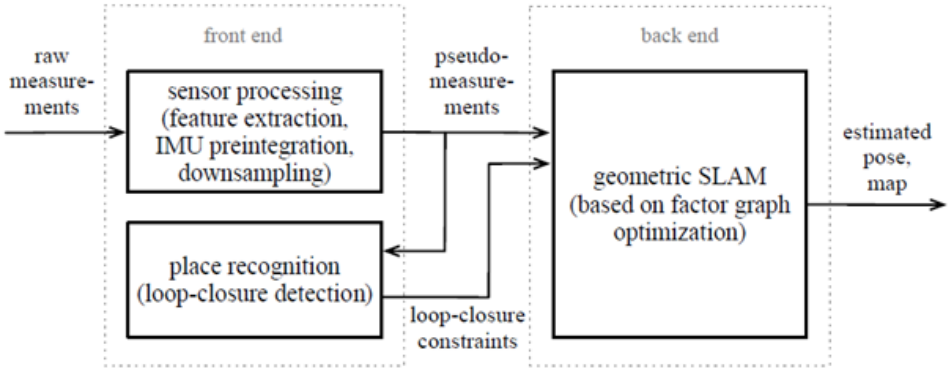


Figure 2.4 The indirect (feature-based) method common to SLAM systems: a front end producing pseudo-measurements and loop-closure constraints, and a back end performing geometric SLAM by factor-graph optimisation [2].

2.2.2 ORB-SLAM3

ORB-SLAM3 [4] is an open-source SLAM system that builds on the success of ORB-SLAM and ORB-SLAM2. It is the first framework to support visual, visual-inertial and multi-map SLAM with a range of camera types, monocular, stereo and RGB-D, and with both pinhole and fisheye lens models (figure 2.5). The parts of its architecture and mechanisms relevant to this project are as follows.

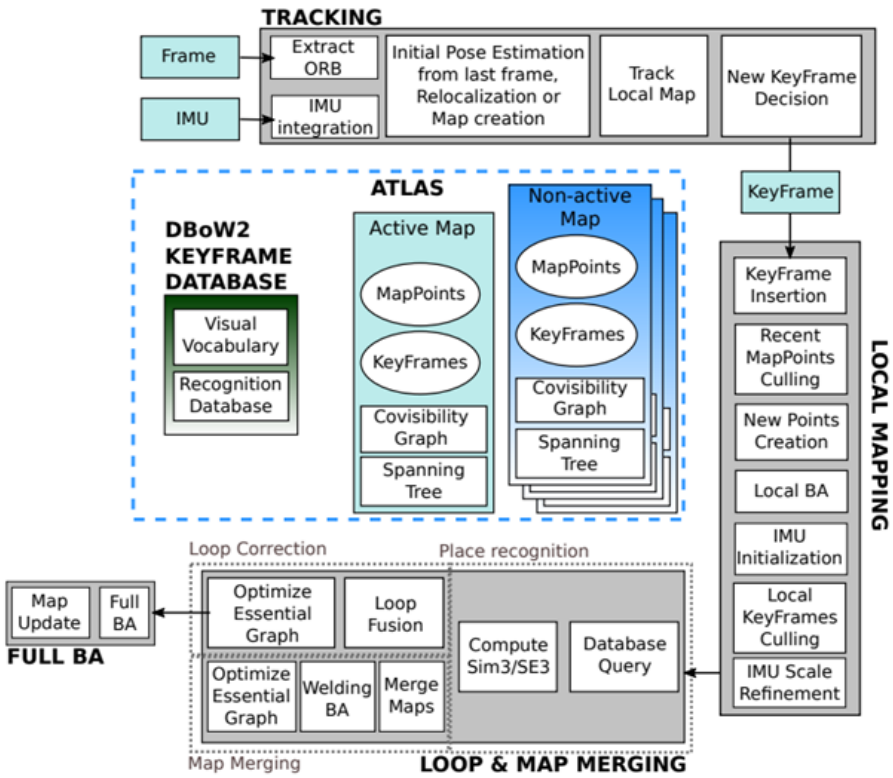


Figure 2.5 Main components of ORB-SLAM3: the tracking, local mapping and loop and map-merging threads around the Atlas multi-map representation and the DBoW2 keyframe database [4].

Tightly-coupled visual-inertial SLAM

One of the most important advances of ORB-SLAM3 is its visual-inertial SLAM (VI-SLAM), which fuses data tightly by maximum-a-posteriori (MAP) estimation both during initialisation and in normal operation, unlike earlier systems that separated the computations. Its mechanisms are:

- **Fast IMU initialisation.** The main problem of typical VI-SLAM systems is the delay in computing the IMU's initial parameters. ORB-SLAM3 proposes a MAP uncertainty computation that recovers the map scale, the gravity direction and the gyroscope and accelerometer biases within about two seconds of start-up, so the robot obtains a correct pose with a physically true distance scale immediately.
- **Joint local bundle adjustment.** While moving, the system solves for the robot pose together with the 3-D landmarks and the IMU states (velocity and biases) simultaneously within a local window, compensating the accumulated drift of the images and the inertial sensor effectively.

Atlas: multi-map system and place recognition

To support lifelong mapping in an environment that changes or has visual disturbances, such as a dynamic warehouse, ORB-SLAM3 introduces the Atlas map management architecture, holding maps in two states:

- **Active map**: the local sub-map the robot is currently using for tracking and mapping.
- **Non-active maps**: the set of past sub-maps the system has recorded and kept in memory.

When the robot passes through severe occlusion, or the lighting changes so far that the camera cannot follow the previous features (tracking lost), an ordinary SLAM system usually fails and stops. ORB-SLAM3 instead moves the map that lost tracking into the non-active set and immediately starts a new active map to continue localizing. When the robot later returns to a previously mapped area, the place-recognition algorithm, with its improved recall, detects the overlap and triggers map merging of the current active map with the earlier non-active map, correcting the error by loop closure. This mechanism permanently reduces the risk of the robot becoming lost in the warehouse.

Abstract camera representation for wide-angle lenses

Because this project selects the AC-IMX390-H190 camera with an ultra-wide 186° field of view (fisheye lens), ORB-SLAM3 is technically well suited: its architecture places an abstract representation layer between the feature extractor and the camera projection model, so the system can switch to the Kannala–Brandt fisheye model directly without affecting the back-end SLAM algorithms. The robot can therefore process wide-angle images correctly and without distortion in the localization result.

2.2.3 VINS (Visual-Inertial System)

VINS (VINS-Mono, VINS-Fusion) [5] is a high-performance open-source state estimator developed by the Aerial Robotics Group (HKUST). Its extended versions can fuse GPS, but for this project's indoor dynamic warehouse VINS is used as a local 6-DoF visual-inertial odometry (VIO) estimator (figure 2.6). Its main mechanisms are as follows.

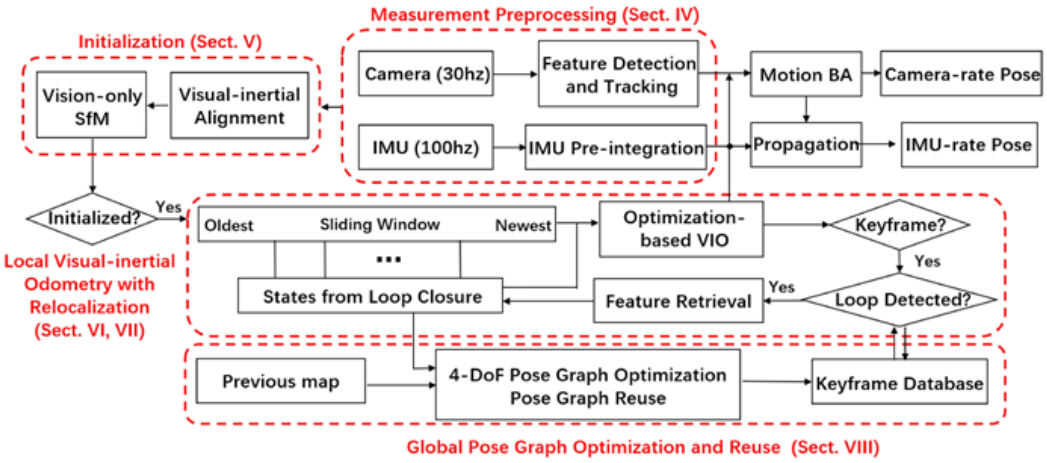


Figure 2.6 Main components of monocular VINS: measurement preprocessing, initialisation, sliding-window visual-inertial odometry with relocalization, and global pose-graph optimisation and reuse [5].

IMU pre-integration

In a fused sensor system the accelerometer and gyroscope deliver data at a very high rate (100–200 Hz) while the camera delivers images at a lower one (20–30 Hz). If a SLAM system re-integrated the IMU data every time a pose was adjusted, the computational load would be enormous. VINS solves this with IMU pre-integration: the accelerations and angular rates between two consecutive image frames are integrated in the robot's local body frame rather than the global frame, so the pre-integrated measurements do not depend on the initial state and need not be recomputed when the back end corrects the robot's pose. The system also propagates covariance and corrects gyroscope and accelerometer bias through a first-order Taylor expansion, giving a physically accurate pose estimate.

Robust estimator initialisation

A central challenge of visual-inertial SLAM is initialisation, because monocular or stereo images carry no true metric scale at the outset and the IMU carries accumulated bias. VINS establishes a tightly-coupled initialisation in two steps:

- **Local structure from motion (SfM).** Features are extracted and tracked over a window of time to build a 3-D structure and the relative camera poses first.
- **Visual-inertial alignment.** The SfM trajectory is set against the IMU pre-integration results to recover three initial quantities: the metric scale, the gravity direction relative to the robot, and the initial gyroscope bias. The AMR thereby perceives true metric distances in the warehouse immediately after it begins to move.

Tightly-coupled sliding-window optimisation

At the heart of the VINS pipeline is a sliding window that limits the number of image frames and robot states that must be solved for. VINS forms a nonlinear least-squares cost function, solved with Ceres Solver, in which three residuals are tightly coupled:

$$\min_{\chi}\Big\{\big\|r_p - H_p\chi\big\|^2 + \sum_{k\in\mathscr{B}}\big\|r_{\mathscr{B}}\big(\hat{z}^{b_k}_{b_{k+1}},\chi\big)\big\|^2_{P^{b_k}_{b_{k+1}}} + \sum_{(l,j)\in\mathscr{C}}\big\|r_{\mathscr{C}}\big(\hat{z}^{c_j}_{l},\chi\big)\big\|^2_{P^{c_j}_{l}}\Big\}, \tag{2.1}$$

where

- the **marginalisation residual** r_p is the prior obtained from past frames removed from the sliding window;
- the **IMU residual** $r_{\mathscr{B}}$ is the error between the robot's true state and the IMU pre-integration; and
- the **visual re-projection residual** $r_{\mathscr{C}}$ is the re-projection error of feature points on the stereo image plane.

Marginalisation for real-time load control

To keep the edge processor (NRU-51V) on the robot running in real time, VINS removes frames from the sliding window under two conditions:

- **Marginalise old (`MARGIN_OLD`).** If the newest frame is judged a keyframe, the oldest frame in the window is removed, and its visual and inertial information is converted into a prior factor by the Schur complement so that important past parameters are not lost.

- **Marginalise second newest (`MARGIN_SECOND_NEW`).** If the newest frame is judged a non-keyframe (insufficient parallax), the second-newest frame is discarded outright without forming a prior. This bounds the density of the matrices and saves a great deal of computation when the AMR is stationary or moving slowly in the warehouse.

Online temporal calibration

A distinctive feature of VINS [6] is its treatment of temporal misalignment between the camera and the IMU. In practice the camera trigger and the IMU signals suffer transmission and triggering delays, so the two sensors' timestamps disagree, an important cause of VIO failure when the robot moves fast or turns. VINS adds the time offset as a state variable estimated jointly in the back-end optimisation, computing the temporal shift of the image frames against the IMU angular rate. The pipeline can thereby compensate the latency of the AC-IMX390-H190 image stream on the processing board precisely, keeping tracking consistent even when the robot vibrates on the 5° warehouse slope.

2.2.4 RTAB-Map back end and memory management

Following Labbé and Michaud [7], RTAB-Map is an open-source SLAM library designed to avoid failure through computational and memory explosion when a robot operates over a large area or for a long time (lifelong mapping). RTAB-Map receives odometry and point-cloud data from VINS and performs graph SLAM in the back end through three main mechanisms (figure 2.7).

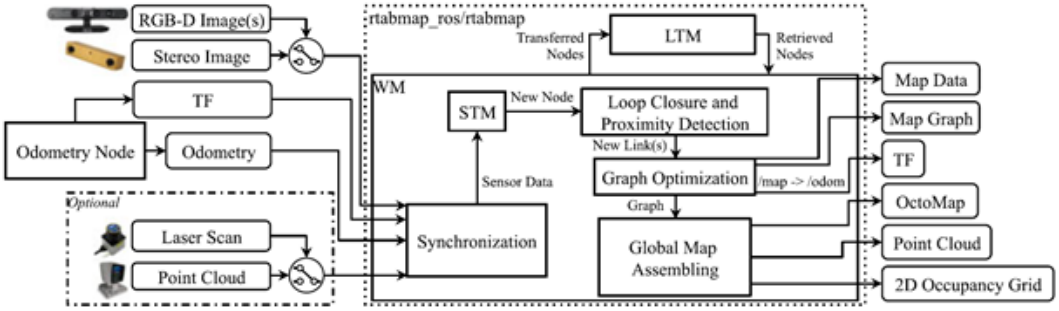


Figure 2.7 Main components of RTAB-Map: sensor synchronisation, short-term and working memory, long-term memory, loop-closure and proximity detection, graph optimisation and global map assembly [7].

Memory partitioning modelled on human memory

RTAB-Map does not keep and compute over every map node in RAM; it organises the data on three levels:

- **Short-term memory (STM)** receives the latest image and odometry from VINS, extracts features and creates a new node. Nodes in STM are not yet used for loop-closure computation.
- **Working memory (WM)** is the main RAM area in which the algorithm detects loop closures and performs graph optimisation. The WM size is held constant by a time threshold so that the computation time never exceeds the real-time limit (for example <30 ms).

- **Long-term memory (LTM)** is the database written to permanent storage (SSD or hard disk). Its contents do not enter the ordinary graph optimisation, conserving RAM and CPU.

Memory transfer and weighting heuristics

To lock the processing time at a constant ($O(1)$ time complexity) regardless of how large the warehouse map becomes, RTAB-Map moves data between WM and LTM as follows:

- **Node weighting.** When the robot traverses a previous route and a loop closure occurs, the nodes there gain weight; frequently traversed nodes (such as the main warehouse aisles) carry high weight.
- **WM-to-LTM transfer.** When WM processing time approaches the set limit, the algorithm forces the "oldest and least-weighted" nodes out of RAM into LTM temporarily.
- **LTM-to-WM retrieval.** If the robot returns to an old area and feature extraction finds a link to a past node, the algorithm automatically retrieves the neighbouring nodes from LTM back into WM to connect the map structure and correct accumulated drift.

Memory-constrained graph optimisation

Unlike ordinary SLAM systems, which must optimise the graph over the whole map and stall when run for a long time, RTAB-Map optimises the graph (through g2o or GTSAM) only over the nodes remaining in working memory. The NRU-51V edge processor can therefore run the navigation system and update the warehouse map smoothly over its whole service life without loss of speed.

2.3 Real-Time Object Detection

2.3.1 The role of object detection in semantic understanding

In traditional geometric visual SLAM the algorithm extracts geometric features without knowing what kind of object they belong to. For a warehouse that changes continually, geometry alone is not enough: the system needs semantic understanding to distinguish permanent structure from movable dynamic obstacles (goods, pallets, people). This project therefore integrates object detection into the front end to produce bounding boxes and pass the coordinates of dynamic objects to the SLAM system, which masks out the unstable features.

2.3.2 The YOLO (You Only Look Once) network architecture

For the system to respond in real time while the SLAM system is running, the project concentrates on the YOLO family, a one-stage detector distinguished by its processing speed. Unlike two-stage models (such as Faster R-CNN) that first generate region proposals and then classify them, YOLO predicts bounding-box regression and class probabilities for the whole image in a single forward pass, which makes YOLO (for example YOLO26) well suited to deployment on a resource-limited edge processor such as the NRU-51V board. Table 2.1 compares the main detector architectures.

Table 2.1 Comparison of object-detection model architectures.

Architecture	Category	Main strengths	Limitations	Suitable use
Faster R-CNN	Two-stage detector	Excels in classification and localization, achieving high mAP and IoU with a low overall error rate and high spatial accuracy	The multi-stage process slows inference	Work requiring high accuracy without real-time processing
YOLO series	Single-stage detector	Casts detection as a single-stage regression problem tuned for speed, balancing speed and accuracy well, with high precision and recall and a low false-positive rate	Although now well refined, early versions had trouble with small or densely clustered objects compared with two-stage models	Real-time work requiring high processing speed and low latency
SSD (Single Shot Detector)	Single-stage detector	Removes the region-proposal step and performs multi-scale detection within one network	Sometimes slightly slower than YOLO, but still very efficient	Applications needing a reliable balance of throughput and accuracy
MobileNet / EfficientNet	Lightweight model	Uses techniques such as depthwise-separable convolutions to cut parameters and computation substantially without reducing learning capacity	Overall accuracy may be below large models such as Faster R-CNN or the larger YOLO versions	Edge devices with limited memory and compute: smartphones, IoT devices, drones

2.3.3 Performance evaluation and edge-processor limits

In bringing a YOLO model into a visual SLAM pipeline, the essential point is the trade-off between accuracy and inference time (latency): too large a model creates a bottleneck and makes the SLAM system fall behind the camera frame rate. The evaluation of the YOLO model on a simulated warehouse dataset, through a platform such as Roboflow, therefore uses the following main metrics:

- **Intersection over union (IoU)**: the overlap ratio between the predicted box and the true object box, measuring localization accuracy.
- **Precision**: the fraction of dynamic-object predictions that are correct, out of all dynamic-object predictions (accuracy of the prediction).
- **Recall**: the fraction of dynamic objects correctly predicted, out of all dynamic objects actually present in the image (coverage of the search).
- **F1 score**: the harmonic mean of precision and recall, measuring the model's balance.
- **Average precision (AP) and mean average precision (mAP)**: the overall accuracy reflecting the model's combined performance across all object types under differing confidence thresholds.

Choosing the model with the highest mAP that still keeps latency within a safe budget (for example $<$15–20 ms) is the key to a complete integration of YOLO and SLAM.

2.4 RY-SLAM: A Robust YOLO-Based Real-Time Visual SLAM Solution for Dynamic Environments

2.4.1 Concept and architecture of RY-SLAM

Chen et al. [8] present RY-SLAM (Robust YOLO SLAM), an advanced visual SLAM system designed to solve navigation failure in dynamic environments. RY-SLAM integrates the ORB-SLAM3 framework with a YOLO-family semantic detection and segmentation network (for example YOLOv8s-seg). Its architecture relies on multi-threaded parallel processing with GPU acceleration: the semantic-segmentation thread is separated from the SLAM tracking thread, so the system can analyse dynamic-object coordinates and estimate the camera pose at the same time without losing real-time throughput (figure 2.8).

2.4.2 Dynamic feature masking mechanism

The heart of RY-SLAM's localization accuracy is its dynamic feature masking, in three technical steps:

- **Semantic mask generation.** The YOLO model analyses the input frame to detect dynamic objects (such as people or vehicles) and generates a mask around each object's area.
- **Outlier elimination.** In the ORB-SLAM3 feature-extraction step the system checks the position of each ORB feature; any feature falling inside a dynamic-object mask is deleted immediately.
- **Static pose estimation.** Only features extracted from the permanent structure of the environment (static background) are fed to bundle adjustment, reducing accumulated drift and tracking loss significantly compared with standard ORB-SLAM3.

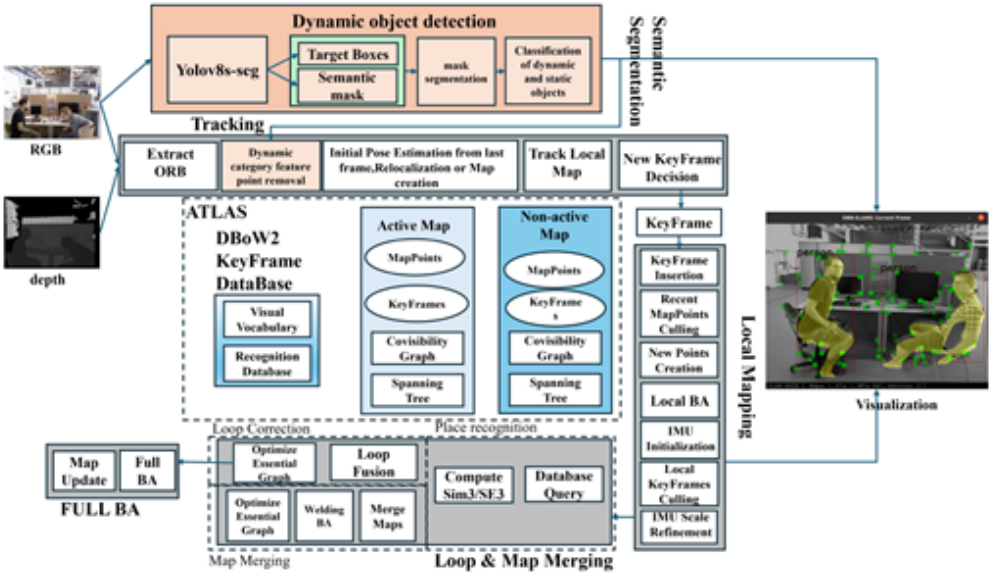


Figure 2.8 Architecture of RY-SLAM: a YOLOv8-seg dynamic-object detection stage feeding a semantic mask into the ORB-SLAM3 tracking thread ahead of the Atlas multi-map system [8].

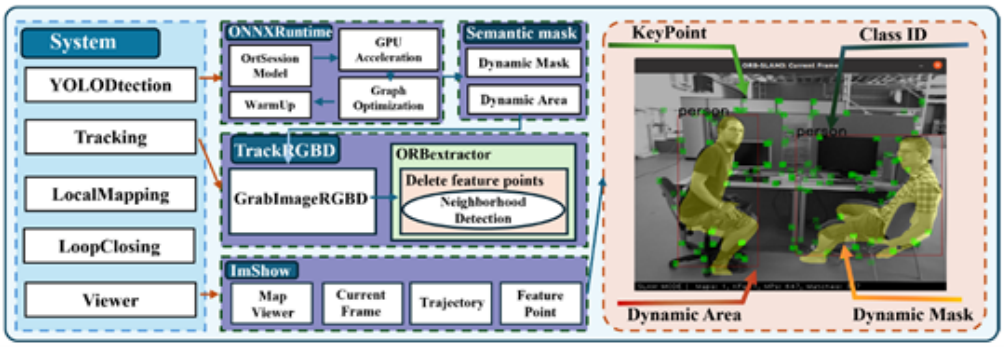


Figure 2.9 Detail of the semantic-segmentation framework in RY-SLAM and its multi-threaded operation: ONNX Runtime inference producing the dynamic mask, the neighbourhood test that deletes feature points, and the viewer outputs [8].

2.4.3 Relevance to this project

RY-SLAM is important academic evidence for the effectiveness and soundness of combining a YOLO-family model with ORB-SLAM3. For this project the author adopts the basic concept and architecture of RY-SLAM and develops it further in pipeline 1 (YOLO26 + ORB-SLAM3), raising system performance by choosing the newer YOLO26 model to filter obstacles and mock goods in the warehouse in real time, together with the Atlas multi-map architecture of ORB-SLAM3 on the NRU-51V edge processor, so as to build a navigation system that is robust both to close-proximity dynamic objects and in long-term (lifelong) mapping, as the objectives require.

CHAPTER 3 METHODOLOGY

To let the system solve the problem set out in chapter 1 and be deployed on an edge AI computer, this project integrates a neural object detector (YOLO26) into the core of a visual SLAM framework, a combination referred to as semantic visual SLAM. This chapter describes the problem and its scope, the system architecture, the two navigation pipelines as they were actually built, the simulation and real platforms, the four experiments that structure the evaluation, the metrics, and the points at which the implemented system departs from the method originally proposed.

Reporting cutoff. Every implementation claim in this chapter describes the repository at 25 September 2026. Components are labelled **implemented**, **partial**, **blocked** or **planned**; nothing planned is described in the present tense as though it existed.

3.1 Problem, Requirements and Scope

The engineering problem is to localize a warehouse AMR with cameras and an IMU, without floor infrastructure, in an environment whose contents move. The company's present LiDAR-and-QR-code navigation is accurate but expensive to install and inflexible (section 1.1); a camera-based replacement is attractive only if it stays accurate when goods, pallets and people move through the field of view, which is exactly the case standard visual SLAM assumes away.

Performance targets. The project states two trajectory targets for the integrated system at the robot's working speed of $1.5\,\mathrm{m\,s^{-1}}$: an absolute trajectory error of at most 0.05 m, and a relative pose error over 1 s of at most 1.5 m. The original objective list (section 1.2) states the localization requirement as an error below 10 cm under differing illumination. Both figures are carried into chapter 4 and every trajectory result is read against them.

Site. The real tests take place on the first floor of the Gensurv building, in a mock warehouse of roughly 10 m by 7 m divided into two operating zones, Zone A and Zone B, each about 3 m square and joined by a walkway (figure 1.2). The walkway carries a 5° slope. Goods are 0.3 m cubes in stacks whose positions are changed between runs, and a light-control fixture sets the illumination to a schedule. A motion-capture system provides the ground-truth reference (section 3.5.4).

Two domains. Every experiment is defined once and run twice, in a Gazebo simulation of the warehouse and on the real robot. The simulation supplies exact ground truth, exact calibration and repeatable dynamic scenes; the real robot supplies the sensors, the lighting and the physics the project is ultimately about. The two domains are never pooled.

What is excluded. Path planning and motion control are outside the project: the pipeline delivers a pose and a map to an existing planner and controller (figure 1.1). Of the five original objectives, this report addresses objectives 1, 3 and 5 (visual-inertial localization with mapping, dynamic-object detection, and the integrated localization requirement). Objective 2 (EnlightenGAN low-light enhancement) and objective 4 (PolyNET lane detection with PID lane tracking) were not pursued within the reporting period and are recorded as deviations in section 3.8.

3.2 System Architecture

3.2.1 Control-loop view and data rates

Figure 1.1 places the project inside the AMR's control loop. The controller issues control input at 20 Hz; the SLAM front end returns an odometry pose at 20 Hz; the object detector returns bounding boxes at 10 Hz on the real robot; the IMU streams at 200 Hz and the cameras at 20 Hz. In simulation the cameras run at 30 Hz and detections are produced per frame, and the wheel odometry of the simulated platform is published at 50 Hz (table 3.9). The four experiments of section 3.6 are each located on one block of this loop:

Table 3.1 The four experiments and the architecture block each one tests.

#	Experiment	Block under test
1	Non-visual baseline and estimator comparison	the robot's own odometry against the SLAM output: is vision earning its place?
2	Offline YOLO detector performance	the object-detection block, in isolation
3	Visual-inertial odometry with dynamic-environment filtering	the SLAM front end, with and without the detector's mask
4	Dense map reconstruction and map-quality evaluation	the SLAM back end and the map it delivers

3.2.2 Semantic visual SLAM data flow

The architecture is organised in two parts, front-end sensor processing and back-end SLAM and mapping.

- **Front-end processing pipeline.** *Sensor data ingestion* receives raw images from the wide-angle stereo cameras (Sony IMX390, 186° field of view) together with acceleration and angular rate from the IMU, through ROS 2 nodes. The *dynamic object detection module* passes each image to the YOLO26 model to detect, in real time, movable warehouse objects (goods, pallets, people) and emits their bounding boxes. The *dynamic feature masking protocol* compares those boxes with the geometric keypoints extracted by the SLAM front end (ORB or FAST corners); any keypoint that falls inside a dynamic object's box is masked and discarded, so that only features of the permanent warehouse structure remain.
- **Back-end SLAM and mapping.** The filtered features enter odometry and graph optimisation to estimate the 6-DoF robot state, update the 3-D point map of the permanent structure, and build the 2-D occupancy grid the navigation system consumes.

Figure 3.1 shows the data flow of the system as it exists.

As shown in figure 3.1, the current work contains two related but separate evaluation paths. The first measures front-end trajectory behaviour with and without semantic feature filtering. The second studies map reconstruction and external global correction through RTAB-Map. RTAB-Map's `map` → `odom` transform does not modify the internal feature map, state history, or previously recorded trajectory of ORB-SLAM3 or VINS-Fusion. A unified architecture for relocalization, global optimization, and multi-session map merging is **planned**, not implemented. Path planning is outside the present research scope.

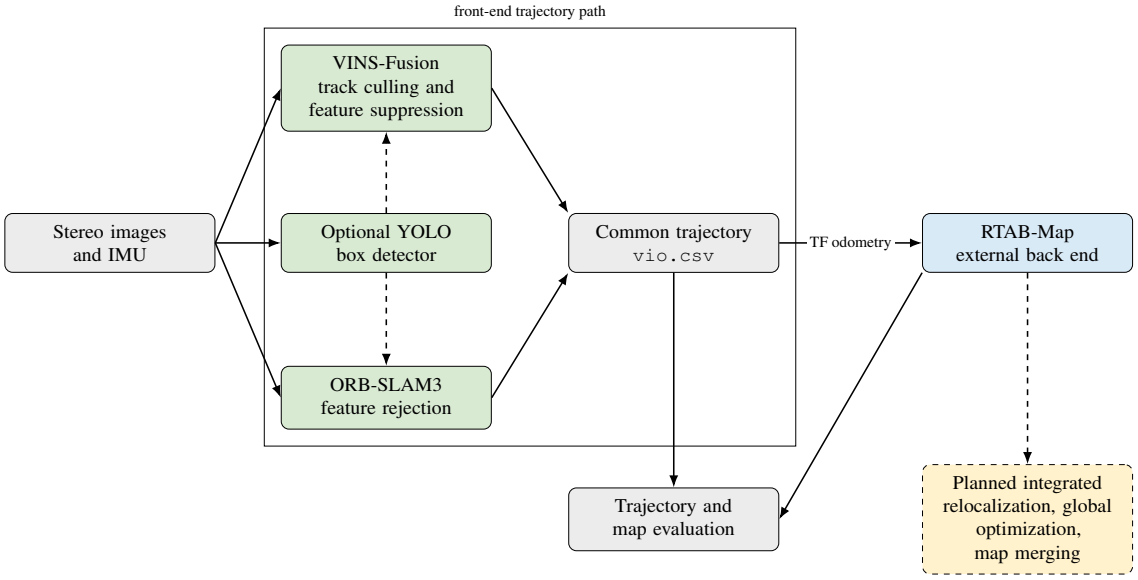


Figure 3.1 Implemented experimental architecture. Green blocks are implemented project components, blue denotes the external RTAB-Map back end, and the dashed yellow block is future work. The diagram deliberately does not show a feedback connection from RTAB-Map into either front end because that integration has not been implemented.

Table 3.2 Implementation status at the reporting cutoff.

Component	Status	Scope of evidence
Warehouse simulation and controlled worlds	**Implemented; verified**	Static, dynamic, textured-floor, and featureless-floor world and launch files are present.
ORB-SLAM3 ROS 2 stereo-inertial wrapper	**Implemented; verified in simulation**	Live topics, worker-thread tracking, TF, and common trajectory output are implemented.
ORB-SLAM3 semantic filtering	**Implemented; mechanism verified**	Detector messages reach the tracker and masked ORB candidates are rejected before octree distribution.
VINS-Fusion stereo-inertial estimator	**Implemented**	Simulation and real-rig configurations, odometry publication, TF, and trajectory output are present.
VINS-Fusion semantic filtering	**Implemented; mechanism verified**	Existing KLT tracks are culled and new features are suppressed in detected regions.
Offline YOLO evaluation	**Implemented; partially evaluated**	Ground-truth projection, IoU matching, per-frame CSV files, and validation overlays are present.

Component	Status	Scope of evidence
RTAB-Map construction and saved-map localization	**Implemented; partially verified**	Construction and basic read-only localization passed; one tiny-map database has been quantitatively evaluated for geometry and drivability (section 4.4.1). Arbitrary-start aliasing rejection and the large-warehouse evaluation have not been done.
RTAB-Map extend and multi-session merging	**Implemented at launch level; experimentally unverified**	The mode exists, but the planned Level 4 experiment has not been completed.
Live VINS-Fusion and RTAB-Map co-execution	**Blocked**	VINS diverges under the combined workload on the current computer; the mechanism is unresolved.
RPE and complete compute evaluation	**Partially implemented**	Front-end, back-end, local-mapping, loop-closing, delivery/state, cumulative process CPU, and resident-memory records are now logged. RPE was added after the cutoff and applied to the campaign (section 4.3.1). External-process monitoring and the repeated hardware matrix remain planned.

The repository state used for this draft is superproject commit `4425a415f624d6a45da1da4f3b2404cbf7914197`. The checked-out submodules are VINS-Fusion `4a2223c9e54dfc5d2e94eb2ec1dfc8fd3b8045bb`, RTAB-Map ROS `56064e4388f6f3ad6b994dbff98bcc0a41488102`, and the warehouse environment `3df54c6e6ec267fad204e208db61c4ab491be410`. The ORB-SLAM3 core is pinned to `5c2b00f732775f78836751dba13db25463a0aaa1`. The VINS-Fusion worktree contains an uncommitted output-directory creation fix at the cutoff; this must be committed or otherwise recorded before a reproducibility release.

3.2.3 The two navigation pipelines compared

The project compares two pipeline architectures built on that front end.

Pipeline 1: YOLO26 + ORB-SLAM3 (Atlas multi-map system). A tightly-coupled visual-inertial SLAM that performs both odometry and map construction through one graph. Lifelong mapping relies on ORB-SLAM3's Atlas framework: when tracking is lost because the environment changed, the system opens a new active map and keeps the old one as non-active; when the robot returns to a known area the maps are merged and joined by loop closure across maps. Its strength is high accuracy from tight image–IMU fusion, suited to areas with clear geometric features (figure 3.2).

Pipeline 2: YOLO26 + VINS + RTAB-Map (decoupled VIO and memory man-

agement). A decoupled architecture that separates high-rate front-end odometry from long-term back-end memory management. The front end runs VINS sliding-window visual-inertial odometry on the static features from YOLO26, giving low-latency odometry that remains stable when the robot vibrates or drives the 5° slope. Lifelong mapping is handled by RTAB-Map, which holds its working memory at a constant size to bound computation time and moves old nodes to long-term memory on disk. Its strength is stable use of hardware resources: it prevents RAM exhaustion on a small processing board during long continuous operation (figure 3.3).

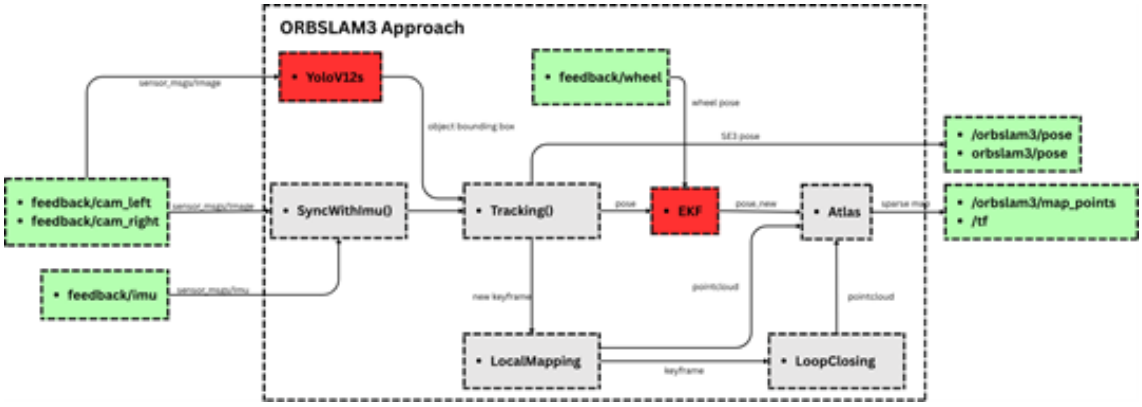


Figure 3.2 Pipeline 1, YOLO26 + ORB-SLAM3, as designed: stereo images and IMU are synchronised, tracked and passed to local mapping, loop closing and the Atlas map; the detector's boxes enter the tracking stage. The EKF block that fuses the SLAM pose with wheel feedback is a planned integration point and is not exercised in this report.

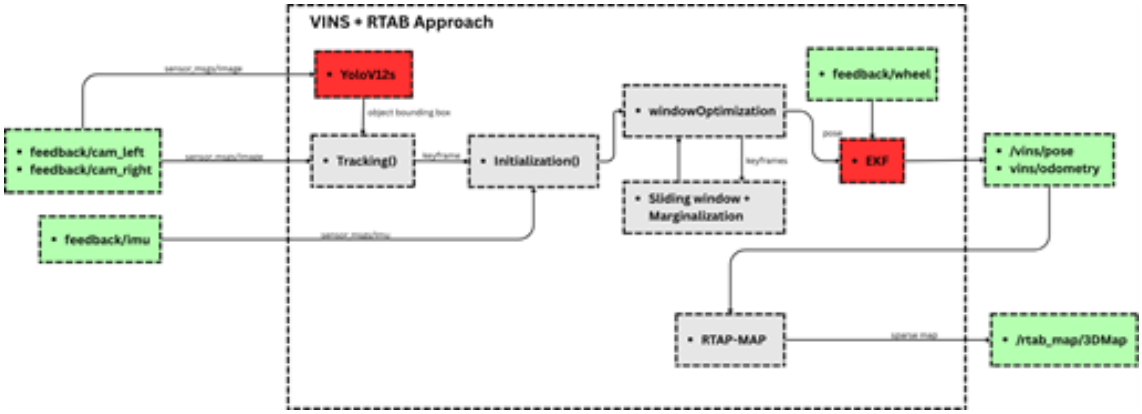


Figure 3.3 Pipeline 2, YOLO26 + VINS + RTAB-Map, as designed: VINS tracking, initialisation and sliding-window optimisation produce odometry, RTAB-Map builds the map from that odometry and the stereo images. As in figure 3.2, the EKF fusion with wheel feedback is planned.

3.3 The Pipelines as Implemented

3.3.1 Dynamic feature masking

In a dynamic warehouse a conventional SLAM algorithm accumulates drift if it uses features on moving objects to compute distance. The project therefore designs a shared front end in which the detector's output filters the geometric features before they reach the back end. When the stereo camera delivers frame I_t at time t, the image is passed to the

Table 3.3 Design comparison of the two pipeline architectures, as proposed.

Aspect	Pipeline 1: YOLO26 + ORB-SLAM3	Pipeline 2: YOLO26 + VINS + RTAB-Map
System structure	tightly coupled (centralised)	decoupled (odometry separate from back end)
Odometry	ORB-SLAM3 tracking thread; tightly-coupled MAP estimation over visual and IMU terms	VINS-Fusion local VIO; sliding-window optimisation
Lifelong-mapping mechanism	Atlas multi-map system and map merging (Sim(3))	RTAB-Map memory management (WM / LTM node transfer)
Computation-time bound	grows with the number of keyframes and sub-maps in Atlas	held at a constant $O(1)$ ceiling in working memory
RAM / CPU use	grows with accumulated map size	bounded through the processing-time threshold $T_{\max}$
Robustness to vibration	moderate to high; relies on ORB features and IMU pre-integration	very high; online temporal and spatial calibration in VINS

YOLO26 network, which returns a set of bounding boxes

$$\mathscr{B} = \{B_1, B_2, \ldots, B_K\}, \qquad B_k = \{x_k, y_k, w_k, h_k, c_k, P_k\}, \tag{3.1}$$

where (x_k, y_k) is the box centre, (w_k, h_k) its width and height, c_k the class label and P_k the confidence. Boxes with $P_k < \tau_{\text{conf}}$ are rejected to limit false positives. Meanwhile the SLAM tracking thread extracts the frame's feature set $\mathscr{F} = \{f_1, \ldots, f_N\}$ with $f_i = (u_i, v_i)$. A feature is labelled dynamic when

$$\exists B_k \in \mathscr{B} : \left(x_k - \tfrac{w_k}{2} \le u_i \le x_k + \tfrac{w_k}{2}\right) \wedge \left(y_k - \tfrac{h_k}{2} \le v_i \le y_k + \tfrac{h_k}{2}\right), \tag{3.2}$$

and is removed immediately, leaving the static feature set

$$\mathscr{F}_{\text{static}} = \{\, f_i \in \mathscr{F} \mid f_i \notin B_k \; \forall k \,\}, \tag{3.3}$$

which is passed to feature matching and re-projection-error minimisation. The following subsections state where in each estimator that test is actually applied, because the two front ends re-use features differently and a single insertion point is not sufficient for both.

3.3.2 ORB-SLAM3 and semantic feature rejection

The project provides a purpose-built ROS 2 Humble wrapper around the pinned ORB-SLAM3 core. Approximate-time synchronization pairs left and right images with a 0.02 s slop. Raw and compressed transports are supported. Image callbacks place stereo pairs in a one-frame queue; a worker thread calls `TrackStereo()` so blocking visual tracking does not starve the 200 Hz IMU subscription. If tracking falls behind, the older pending image is discarded and the drop count is reported.

With `filter:=false`, no detection subscription is created and an empty mask passes

through the patched interfaces. With filtering enabled, a bounded buffer is searched for the detection closest in source-image time, accepted only within 0.15 s. Boxes are expanded by eight pixels, clipped to the image, and rasterized to an 8-bit mask in which 255 retains a feature and zero rejects it. A mask covering more than 0.8 of the frame is discarded as a safety measure.

The mask passes through `System`, `Tracking`, and `Frame` to the ORB extractor. Candidate keypoints are tested before octree distribution, so the per-level feature quota can be redistributed across surviving static regions. At each pyramid level, the implemented boundary rule rejects a candidate when at least five samples in a 3×3 neighbourhood are dynamic. Only left-camera features are masked because ORB-SLAM3 establishes stereo matches from left keypoints to right candidates.

The current simulation YAML requests 1500 ORB features, eight pyramid levels, a 1.2 scale factor, initial FAST threshold 20, and fallback threshold 7; table 3.4 collects these. An earlier draft of this section reported 1000 and attributed 1500 to a superseded engineering note. That was backwards: `ORBextractor.nFeatures` is 1500 in the committed configuration, and no uncommitted change touches it. No controlled feature-count or FAST-threshold sweep is complete.

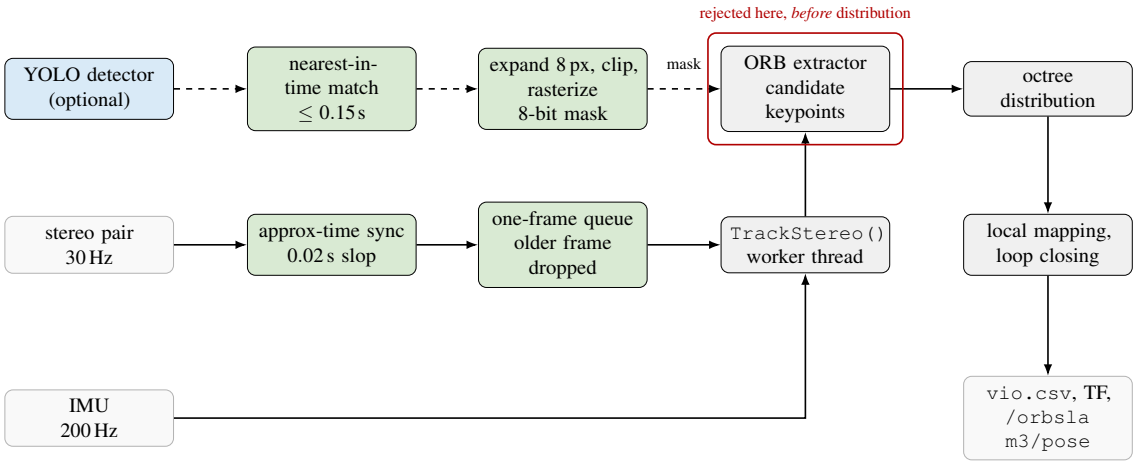


Figure 3.4 ORB-SLAM3 pipeline and the semantic-mask insertion point. Green is built by this project, grey is the pinned upstream core, blue the optional detector path. The mask is applied to candidate keypoints *before* octree distribution, which is what allows the per-level feature quota to redistribute onto surviving static regions rather than simply shrink. A candidate is rejected when at least five samples of its 3×3 neighbourhood are dynamic, and only the left camera is masked, because ORB-SLAM3 forms stereo matches outwards from left keypoints.

The wrapper publishes `/orbslam3/odometry`, `/orbslam3/pose`, and TF after valid tracking. It writes the common 11-column trajectory format directly and flushes each row. Repository evidence is located in `orbslam3_ros2/src/orbslam3_node.cpp`, `orbslam3_ros2/src/slam_wrapper.cpp`, `orbslam3_ros2/src/trajectory_writer.cpp`, and `orbslam3_ros2/config/wil_sim/stereo_imu.yaml`.

3.3.3 VINS-Fusion front end

VINS-Fusion uses stereo features and inertial measurements in a sliding-window nonlinear estimator. The ROS wrapper synchronizes the raw image topics and uses separate callback

Table 3.4 ORB-SLAM3 configuration at the cutoff. Wrapper values are from `orbslam3_ros2/src/`; extractor and IMU values from `config/wil_sim/stereo_imu.yaml`. Noise densities are given in table 3.8 and are not repeated here.

Parameter	Value	Role
Wrapper (built here)		
Stereo sync slop	0.02 s	approximate-time pairing
Image queue depth	1 frame	older frame dropped, drop count logged
Detector match window	0.15 s	nearest in source-image time
Box expansion	8 px	before rasterizing the mask
Mask reject threshold	0.8 of frame	safety cut-out on runaway coverage
Boundary rule	≥ 5 of 3×3	candidate rejected when dynamic
Masked camera	left only	stereo matches form from left keypoints
Extractor and core		
`ORBextractor.nFeatures`	1500	per-frame feature budget
`ORBextractor.nLevels`	8	pyramid levels
`ORBextractor.scaleFactor`	1.2	inter-level scale
`ORBextractor.iniThFAST`	20	initial corner threshold
`ORBextractor.minThFAST`	7	fallback threshold
`Stereo.ThDepth`	90.0	close/far stereo split
`IMU.Frequency`	200 Hz	preintegration rate
`loopClosing`	1	comment in the same file says “set to 0 deliberately”

groups for images, IMU, feature messages, and detections. The simulation configuration processes every image (`image_skip: 1`), uses two cameras and IMU, fixes simulated extrinsics, targets 25 Hz feature publication, and uses 10-pixel keyframe parallax. Image queues were increased to 100 and the IMU queue to 2,000 to reduce losses during initialization and backlog.

VINS cannot use only the ORB strategy because it propagates existing features with pyramidal KLT optical flow and detects new features only for replenishment. A feature attached to an object before detection could otherwise persist after masking. The implementation initializes `FeatureTracker::setMask()` from the YOLO mask. Tracks in zero-valued regions fail the retention test, while the same mask is passed to `goodFeaturesToTrack()` to suppress new points. The count of persistent tracks removed in dynamic regions is accumulated as a mechanism diagnostic.

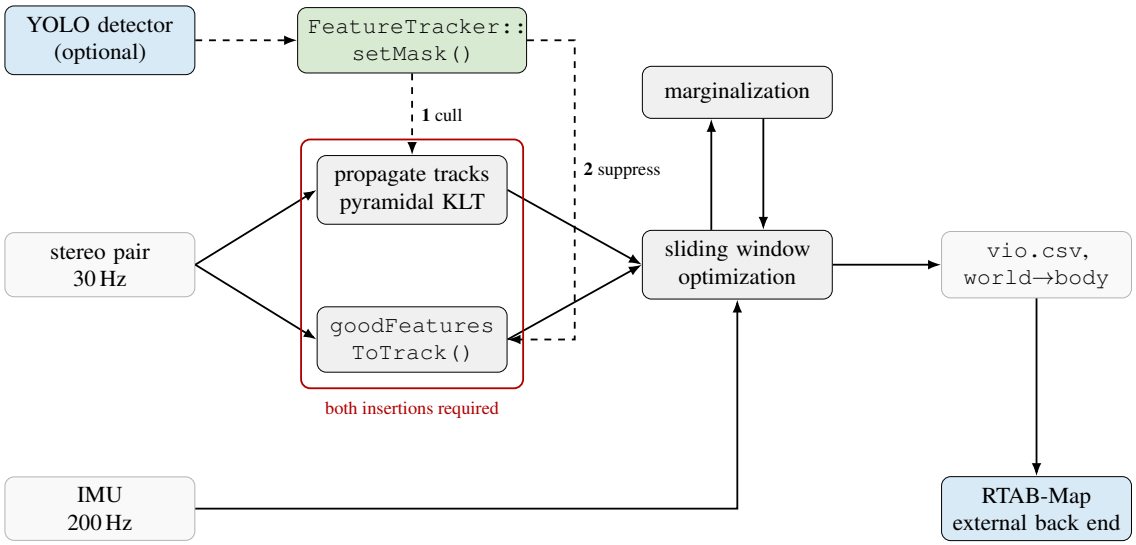


Figure 3.5 VINS-Fusion pipeline and its two semantic-mask insertion points. Because the front end propagates existing KLT tracks and only detects new features to replenish them, suppressing new detections alone is insufficient: a track that attached to an object before it was detected would survive masking. Tracks in masked regions must therefore also be culled (1) while the same mask suppresses replenishment (2). Contrast figure 3.4, where re-extraction every frame makes a single insertion sufficient. The count of persistent tracks removed is logged as a mechanism diagnostic — the counter that read zero in all twelve filtered runs.

VINS writes the optimized state after each estimator update to `vio.csv` and broadcasts `world → body` using configurable frame names. The ROS `output_path` parameter overrides the YAML path, recursively creates missing directories, and reports an explicit error if `vio.csv` cannot be opened. This behaviour is an uncommitted worktree change at the cutoff. Evidence is in `vins_fusion_ros2/src/vins_estimator.cpp`, `vins_fusion_ros2/vins/src/featureTracker/feature_tracker.cpp`, and `vins_fusion_ros2/vins/src/estimator/estimator.cpp`.

3.3.4 RTAB-Map mapping and localization back end

RTAB-Map receives front-end odometry through TF and stereo imagery through `rtab_camera_shim.py`. The shim reconstructs `CameraInfo`, decompresses the pair, and publishes validated static camera transforms from the common simulation calibration. TF was selected over an odometry topic because adding high-rate odometry to multi-topic

Table 3.5 VINS-Fusion simulation configuration at the cutoff, from `vins_fusion_ros2/config/wil_sim/stereo_imu.yaml`. Inertial noise densities are in table 3.8.

Parameter	Value	Role
`num_of_cam` / `imu`	2 / 1	stereo-inertial
`image_skip`	1	every image processed
`max_cnt`	150	tracked-feature budget
`min_dist`	30 px	minimum feature separation
`freq`	25 Hz	target feature publication
`keyframe_parallax`	10 px	keyframe decision
`F_threshold`	1.0	RANSAC fundamental-matrix gate
`max_solver_time`	0.08 s	Ceres budget per update
`max_num_iterations`	100	Ceres iteration cap
`estimate_extrinsic`	0	simulated extrinsics held fixed
`estimate_td`	0	camera–IMU delay held at 0 s
`g_norm`	$9.8\,\mathrm{m\,s^{-2}}$	matches the world's gravity
Image queue	100	raised to survive initialization backlog
IMU queue	2000	raised for the same reason

approximate synchronization starved the lower-rate images. The cost is that odometry covariance is not supplied to graph edge weighting.

The back end uses stereo SGBM (`Stereo/DenseStrategy: 1`), height-based ground separation, ray tracing, spatial noise filtering, and odometry-derived gravity links. Its measured default depth range is 10 m. Because the featureless floor provides essentially no stereo ground points, ray tracing supplies free-space evidence between the sensor and observed obstacles. These settings affect occupancy assembly; visual-word loop closure is separate.

Three modes are provided by one launch file:

construct starts a new incremental database;
extend opens an existing database, places prior nodes in working memory, and permits new nodes; and
localize disables incremental memory, loads existing nodes, serves the saved map, and can enable `Mem/LocalizationReadOnly`.

Localization still extracts features from current stereo images and matches stored RTAB-Map signatures. Read-only mode prevents database modification; it does not disable perception. The outputs are an external `map` → `odom` correction and `/localization_pose`. Neither RTAB-Map features nor corrected poses are injected into ORB-SLAM3's Atlas or VINS-Fusion's sliding window.

The composed localization launch raises `RGBD/OptimizeMaxError` from 3.0 to 10.0 because the default rejected all loop closures proposed with the observed VINS drift. This is effective operationally but is not yet proven safe against perceptual aliasing in repeated warehouse bays. Occupancy creation is disabled in localization mode because the stored map is reused. Live VINS-Fusion and RTAB-Map co-execution nevertheless remains blocked on the current machine. Recorded `vio.csv` can be replayed as

odom → vio_body for a deterministic back-end test, but this is not proof of real-time co-execution.

Evidence is in rtabmap_ros/rtab_sim.launch.py, rtabmap_ros/rtab_camera_shim.py, aws-robomaker-small-warehouse-world/launch/localization_rtabmap.launch.py, and script/gt_tf_publisher.py.

3.3.5 YOLO detector node, mask construction and synchronisation

The semantic front end is a standalone Python ROS 2 node using Ultralytics. The project checkpoint is weight/best.pt, a custom detector for bin, box, and bucket, fine-tuned from yolo26m.pt using 640-pixel inputs. It produces bounding boxes, not segmentation masks. The stock yolo26n.pt and yolo26m.pt files are not project-trained models and are not evidence of custom training.

At startup, the node resolves requested classes against checkpoint metadata, warms the model with three dummy images, and selects CUDA when available. Current defaults are confidence 0.35, NMS IoU 0.5, and image size 640. Only the left image is processed. Each Detection2DArray preserves the source image header, so nearest-timestamp selection measures sensor-time correspondence rather than inference-completion time. Empty arrays distinguish a valid frame with no detections from detector failure.

The detector intentionally remains outside stereo synchronization. A three-input image/image/detection synchronizer would discard pairs whenever inference lagged. The implemented design continues without filtering if no recent detection exists. Filtered runs write yolo_mask.csv with timestamp, box count, coverage, match state, detection age, and rejection status.

Offline evaluation in script/yolo_eval.py transforms Gazebo model poses and collision-mesh vertices into the left-camera frame to form projected truth boxes. Object speed over $0.05\,\mathrm{m\,s^{-1}}$ defines the moving subset. Predictions are greedily matched by class at IoU 0.5. Heavily occluded truth boxes and predictions covered by known but unlabeled box-like scenery are ignored at stated thresholds. Outputs are per-frame yolo_eval.csv, per-detection yolo_detections.csv, and overlays. Overlays are inspected before aggregate metrics because invalid pose stamps, centimetre mesh units, or malformed camera transforms can produce plausible but invalid numbers.

The current semantic rule assumes all selected-class detections are dynamic. No optical-flow, stereo-depth, or world-motion gate is implemented, and axis-aligned boxes also remove background pixels. These are explicit method limitations.

3.4 Simulation Platform

3.4.1 Computing and software environment

One machine produced every simulation result in this report. Table 3.6 records it. Each entry was re-read from the running system on 20 September 2026 rather than copied forward from the engineering log, and all agreed.

Two consequences follow from the table rather than from any experiment. The proposed Jetson-versus-laptop study has not been instrumented or executed, so every timing, through-

Table 3.6 Computing and software environment. A single laptop produced every result reported here; there is no second platform, so no entry below supports a cross-hardware comparison.

Component	Version	Role
Platform		
Operating system	Ubuntu 22.04.5 LTS	
Middleware	ROS 2 Humble	
CPU	16 logical cores	
Memory	15 GB, no swap	no swap, so memory pressure fails outright rather than degrading
GPU	NVIDIA RTX 3070 Laptop	detector inference only
Build toolchain		
GCC	11.4.0	
CMake	3.22.1	
OpenCV (system)	4.5.4	links both estimators
Pangolin	0.9.1	ORB-SLAM3 built out of tree
Detector environment		
PyTorch	2.5.1+cu121	
torchvision	0.20.1+cu121	
Ultralytics	8.4.138	YOLO training and inference
NumPy	1.26.4	
OpenCV (Python)	4.11.0	differs from the 4.5.4 the estimators link
Mapping back end		
`ros-humble-rtabmap`	0.23.7	apt core library
RTAB-Map ROS wrappers	project checkout	only packages through `rtabmap_slam` are built

put and CPU figure in chapter 4 describes this machine alone and none supports a hardware comparison. And the detector runs a different OpenCV build from the one the estimators link against, which is unremarkable for two separate processes but would matter if image handling were ever moved across that boundary.

3.4.2 Simulated sensor rig and platform

The simulated Ackermann robot carries two ideal pinhole cameras and an IMU on a forward mast, and is driven through a front-steering, rear-driven four-wheel chassis. Figure 3.6 gives its dimensioned plan and elevation, and tables 3.7 and 3.9 list the sensor and platform constants. Every value in the figure and both tables is read from `aws-robomaker-small-warehouse-world/models/ackermann_robot/model.sdf`, so the drawing is to scale rather than illustrative.

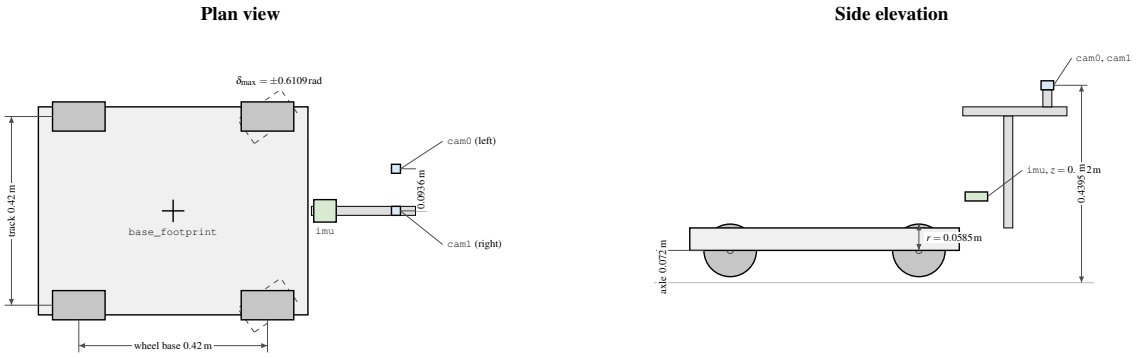


Figure 3.6 Simulated Ackermann platform, drawn to scale from the model definition. Distances are in metres relative to `base_footprint`. The stereo pair is mounted asymmetrically: `cam1` sits on the vehicle centreline and `cam0` is offset 0.0936 m to its left, so the optical centre of the pair is not the chassis centreline. The wheel base and track are equal at 0.42 m, which is what makes the rear-wheel differential a poor yaw-rate estimator (section 3.6.1).

Table 3.7 Simulated sensor configuration. Intrinsics are analytic rather than calibrated: the simulator renders an exact pinhole camera, so $f_x = f_y$ follow from the image width and the horizontal field of view and the distortion coefficients are identically zero. The stereo baseline was independently checked from positive disparity in a recorded bag rather than taken from the model alone.

Quantity	`cam0` (left)	`cam1` (right)	`imu`
Resolution	1280×720	1280×720	–
Rate	30 Hz	30 Hz	200 Hz
Horizontal FOV	$\pi/2$ rad (90°)	$\pi/2$ rad (90°)	–
$f_x = f_y$	640	640	–
(c_x, c_y)	$(640, 360)$	$(640, 360)$	–
Distortion	zero	zero	–
Position from `base_footprint` (m)	$(0.496, 0.0936, 0.4395)$	$(0.496, 0, 0.4395)$	$(0.338, 0, 0.192)$
Position from IMU body (m)	$(0.158, 0.0936, 0.2475)$	$(0.158, 0, 0.2475)$	$(0, 0, 0)$
Stereo baseline	0.0936 m		–

The fixed camera rotation converts the ROS body convention (forward, left, up) to the optical convention (right, down, forward). The VINS simulation configuration fixes the extrinsics (`estimate_extrinsic: 0`) and disables temporal-offset estimation (`estimate_td: 0`); the assumed inertial noise model is given in table 3.8, and the same values are mapped to the ORB-SLAM3 configuration.

Table 3.8 Inertial noise parameters supplied to the visual-inertial estimators, beside the values measured from the recorded IMU stream in section 3.6.1. The configured values are inherited estimator defaults, not measurements of this simulated IMU; the discrepancy is carried forward as a queried item rather than silently reconciled.

Parameter	Configured (VINS, ORB)	Measured from bag	Units
`acc_n`	0.02	0.0139–0.0294	$\mathrm{m\,s^{-2}}/\sqrt{\mathrm{Hz}}$
`gyr_n`	0.002	0.000300–0.000364	$\mathrm{rad\,s^{-1}}/\sqrt{\mathrm{Hz}}$
`acc_w`	0.001	not measurable	$\mathrm{m\,s^{-3}}/\sqrt{\mathrm{Hz}}$
`gyr_w`	0.0001	not measurable	$\mathrm{rad\,s^{-2}}/\sqrt{\mathrm{Hz}}$

Table 3.9 Platform geometry and drivetrain limits used by the wheel-odometry and EKF baselines. `AckermannSteering` exposes no noise parameter, so the simulated encoders and odometry are noise free at source.

Quantity	Symbol	Value	Source field
Wheel base (front to rear axle)	L	0.42 m	`wheel_base`
Track (wheel separation)	B	0.42 m	`wheel_separation`
Kingpin width	–	0.42 m	`kingpin_width`
Wheel radius	r	0.0585 m	`wheel_radius`
Wheel width	–	0.065 m	wheel `<length>`
Steering limit	$\delta_{\max}$	± 0.6109 rad	`steering_limit`
Chassis mass	–	10.1 kg	`base_footprint` `<mass>`
Maximum speed	–	$10\,\mathrm{m\,s^{-1}}$	`max_velocity`
Maximum acceleration	–	$3\,\mathrm{m\,s^{-2}}$	`max_acceleration`
Odometry publish rate	–	50 Hz	`odom_publish_frequency`
Joint-state publish rate	–	1 kHz	measured from bag

3.4.3 Simulation fidelity and its limits

Every simulation result in this report rests on how faithfully the simulator reproduces the sensors and the contact physics. This subsection states what is modelled, what is idealised, and in which direction each idealisation biases the comparison between the visual and proprioceptive estimators.

Inertial measurement. Table 3.8 reports the noise measured from the recorded stream and queries it, because no `dataset_allsensor` bag contains a stationary interval long enough for a sound estimate. That query can be closed without a new recording: the noise is *declared*, not emergent, so the simulator's own configuration is the ground truth the measurement was trying to recover.

Table 3.10 collects what the simulator actually injects, together with the measured values and the EuRoC reference. The measured gyroscope range brackets the declared value, which corroborates both. The measured accelerometer range does not: it is three to seven times the declared figure, and that discrepancy is the estimator's rather than the sensor's, because the residual on the driving axis carries real longitudinal acceleration that a 1.54 s window cannot separate from noise. Prefer the declared values wherever a number is needed.

Table 3.10 Noise injected by the simulator, from the `<imu>` and `<camera>` sensor blocks of the robot model. Densities are per-sample σ divided by $\sqrt{f_s}$ at 200 Hz, in the channel's own units per $\sqrt{\text{Hz}}$. The EuRoC column is the ADIS16448 of that benchmark, on which both visual estimators were published.

Channel (units of σ)	Declared σ	Density	Measured	EuRoC
Gyroscope, per axis ($\text{rad}\,\text{s}^{-1}$)	0.0048	3.394×10^{-4}	3.00×10^{-4}–3.64×10^{-4}	1.70×10^{-4}
Accelerometer, per axis ($\text{m}\,\text{s}^{-2}$)	0.0566	4.002×10^{-3}	1.39×10^{-2}–2.94×10^{-2}	2.00×10^{-3}
Camera, per pixel (normalised intensity)	0.005	—	not measured	—
Bias, first-order Gauss–Markov: σ_b and correlation time τ				
Gyroscope bias ($\text{rad}\,\text{s}^{-1}$)	3.8785×10^{-5}	$\tau = 1000\,\text{s}$	not measurable	—
Accelerometer bias ($\text{m}\,\text{s}^{-2}$)	0.006	$\tau = 300\,\text{s}$	not measurable	—

Two readings follow. The simulated IMU is *twice as noisy* as the EuRoC reference on both channels, so it is not an optimistic sensor. The camera, at about 1.3 grey levels out of 255 and with no distortion terms, is very much cleaner than any real one. Both bias correlation times exceed the 85–130 s run lengths, so within a run the bias behaves as a slowly varying constant and its instability is never exercised.

The IMU is therefore not optimistic in its noise. Three things are nonetheless absent: chassis vibration, which on a real platform is frequently the dominant error; scale-factor, cross-axis and g-sensitivity errors; and temperature drift. The correlation times also exceed the 85–130 s run lengths, so within a run the bias behaves as a slowly varying constant and its instability is never exercised. Finally, the message's `orientation` field carries no noise block at all and is therefore ground truth; no estimator in this project consumes it,

and none should.

Wheel encoders. `JointStatePublisher` reports the true solver angle. Slip consequently appears as a *discrepancy* rather than an error — the joint angle stays true, so encoder-integrated distance over-reads whenever traction breaks, which is exactly a real encoder's behaviour. What is idealised is resolution: a physical 1024 CPR encoder quantises to 6.1×10^{-3} rad, setting a floor on low-speed velocity estimation that the simulation does not have. `script/encoder_sim.py` exists to impose that floor downstream and has not been applied to any result in this report.

Physics engine and solver. Table 3.11 records the simulation's physics configuration, which is identical across all 26 world files. Two entries deserve comment because they are easy to misread.

The `<physics>` element declares `type="ignored"`. That is not a misconfiguration: in Gazebo Fortress the engine is selected by the physics system plugin, not by that attribute, and the attribute is disregarded. No world passes an `<engine>` parameter to the plugin, so the engine is Fortress's default, `dartsim`. This matters for the friction discussion below, and it is the reason `slip1` and `slip2` have no effect.

No solver block is declared anywhere — no iteration count, no SOR parameter, no constraint-force mixing or error-reduction terms. Every one of those therefore takes the engine default. A 1 ms step is a fine integration step for a $1.4\,\mathrm{m\,s^{-1}}$ ground vehicle, but the untuned contact solver is the part of this setup least able to support a quantitative claim about contact behaviour.

Table 3.11 Gazebo physics configuration, identical in all 26 world files. Entries marked "engine default" are not declared anywhere in the project and are therefore whatever `dartsim` chooses.

Setting	Value	Note
Engine	`dartsim`	Fortress default; no world sets `<engine>`
`<physics type>`	`"ignored"`	disregarded in Fortress; the plugin selects the engine
`max_step_size`	1 ms	1 kHz integration
`real_time_factor`	1	no deliberate slow-down
`real_time_update_rate`	engine default	not declared
Solver type and iterations	engine default	no `<solver>` block anywhere
CFM / ERP	engine default	no `<constraints>` block anywhere
Gravity	$-9.8\,\mathrm{m\,s^{-2}}$	against ORB-SLAM3's hard-coded 9.81, a 0.1 % difference
`slip1`, `slip2`	0 on all grounds	an ODE feature; `dartsim` does not implement it, so inert at any value

Achieved real-time factor was not logged during recording, so the table states the requested factor rather than the realised one. Replay rate, which is logged, is a separate control and is given per experiment in chapter 4.

Contact friction, and why the encoder is flattered. The wheels declare $\mu = 1$, $\mu_2 = 1$ and every ground model declares $\mu = 100$, $\mu_2 = 50$. Both sets sit inside `<frict`

ion><ode> elements, which is where SDF places them irrespective of engine, and dartsim consumes them (table 3.11). How it combines two differing coefficients is not established here, so the effective value is not asserted; what is measured is the outcome in table 3.12.

slip1 and slip2 are pinned at zero on all three ground models, but that is not why force-dependent slip is absent. Those parameters are an ODE feature, and dartsim does not implement them, so they would be inert at any value. Contact is rigid until Coulomb traction is exceeded regardless.

Table 3.12 gives the consequence, measured rather than assumed, by integrating rear-wheel angle against true path length.

Table 3.12 Measured wheel slip on the five multi-sensor recordings. The ratio is encoder-integrated distance over true path length; above one is slip, below one is a rolling-radius error and cannot be slip. Measured by script/check_slip.py.

Recording	Encoder (m)	Truth (m)	Ratio
allsensor_000	117.12	117.05	1.0006
allsensor_001	112.91	113.11	0.9982
allsensor_002	115.37	115.20	1.0015
allsensor_003	141.68	140.61	1.0076
allsensor_004	142.36	142.25	1.0008

Slip is present but under 1 % everywhere, and on allsensor_001 the encoder under-reads, which cannot be slip. Against a wheel-odometry drift of 2.15–6.57 %, slip is therefore not the dominant error; the yaw-rate geometry is, as the falsification of H1 establishes.

Noise models supplied to the estimators. The two are not told the truth about the IMU, as table 3.13 shows.

Table 3.13 What the visual estimators are configured to expect, against what the simulator injects. Both estimators use identical values. Inflating noise above datasheet figures is standard practice in visual-inertial odometry and the configuration records that intent; the ratios are given so the size of the margin is visible.

Parameter	Configured	Injected	Ratio	Quantity
gyr_n	0.002	3.394×10^{-4}	5.9×	gyroscope noise density
acc_n	0.02	4.002×10^{-3}	5.0×	accelerometer noise density
gyr_w	0.0001	1.735×10^{-6}	58×	gyroscope bias random walk
acc_w	0.001	4.899×10^{-4}	2.0×	accelerometer bias random walk

The injected bias random walks are the Gauss–Markov driving densities $\sigma_b\sqrt{2/\tau}$ implied by table 3.10. Inflating noise above datasheet values is standard practice in visual-inertial odometry, and the configuration records that intent. The EKF baseline, by contrast, derives its noise model by measuring it against ground truth from the same recording.

That asymmetry was tested directly rather than left as a caveat. Configurations carrying the true injected densities — stereo_imu_truenoise.yaml for both estimators — were run on all five recordings on 18 September 2026, after the reporting cutoff. Accuracy improved wherever a run completed: VINS-Fusion on four of five, by 43 % and 32 %

on two of them. But three of ten runs diverged outright, reaching 111–3899 m of error with 516–1545 tracking jumps, where the inflated configuration diverged in none. The behaviour is bimodal rather than graded: surviving runs carry zero to three jumps, failed runs hundreds. This confirms the configuration comment's own claim that too-large noise degrades gracefully while too-small does not, and the inflated setting is retained for every result in this report. These runs are recorded as `logging_20260916_03` and are not part of the reported campaign.

3.4.4 Coordinate-frame contract

Standalone VINS-Fusion publishes `world` → `body`. Standalone ORB-SLAM3 publishes an estimator-specific world frame. For RTAB-Map integration, both publish `odom` → `vio_body`; a measured static transform connects `vio_body` → `base_footprint` by $(-0.338, 0, -0.192)$ m. RTAB-Map then publishes the external correction `map` → `odom` (figure 3.7).

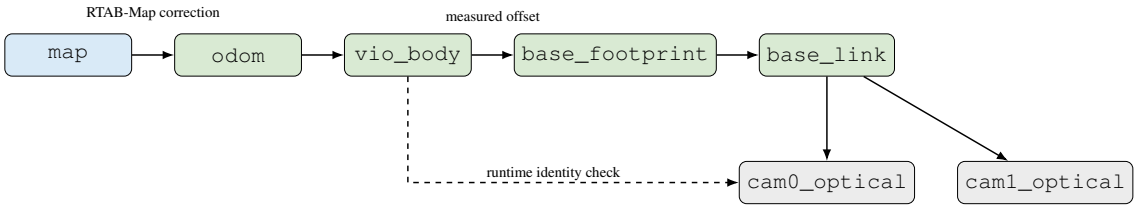


Figure 3.7 Coordinate-frame contract used by the composed RTAB-Map pipeline. The dashed connection is a calibration consistency check, not a second pose publisher.

The two independent paths to `cam0_optical` provide a runtime calibration check: `tf2_echo cam0_undis cam0_optical` must return zero translation and the identity quaternion. This check has been measured successfully. It is needed because an earlier OpenCV YAML parsing error yielded a plausible translation but a non-rigid rotation and invalidated a map-quality campaign.

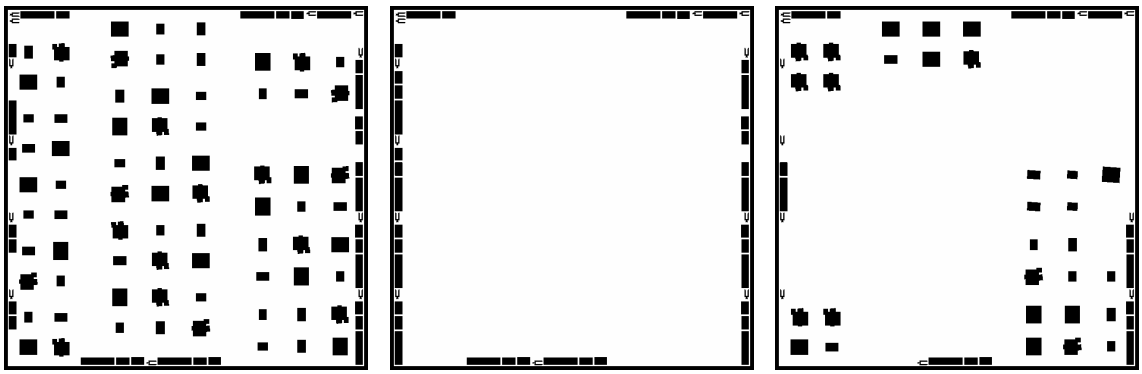
The standalone simulation ORB launch still labels the estimated IMU-body pose as `base_footprint`. The composing RTAB-Map launch corrects this by using `vio_body`; standalone consumers must not assume that the default label is the chassis origin. Constant offsets are mostly absorbed by rigid trajectory alignment but remain material for map placement and robot-footprint tests.

3.4.5 Simulation environments and controlled factors

The principal large warehouse is a 41.2833 m square environment produced by rescaling the AWS warehouse to 1704 m^2, while retaining a 9.020 m interior height. Fourteen marked storage bays and five reduced-width walkways are represented. Static and dynamic families are stored under `worlds/small_warehouse_static/` and `worlds/small_warehouse_dynamic/`. The original tiny warehouse under `worlds/small_warehouse/` is a distinct environment; tiny- and large-layout results are not repeated trials of the same condition.

The occupancy truth of the object arrangements is shown in figure 3.8. These grids are not SLAM products: they are rasterised directly from the world's collision meshes by `script/bake_map.py`, sliced at robot height, so they contain no unknown space and regenerate whenever the world changes. They are the reference against which any

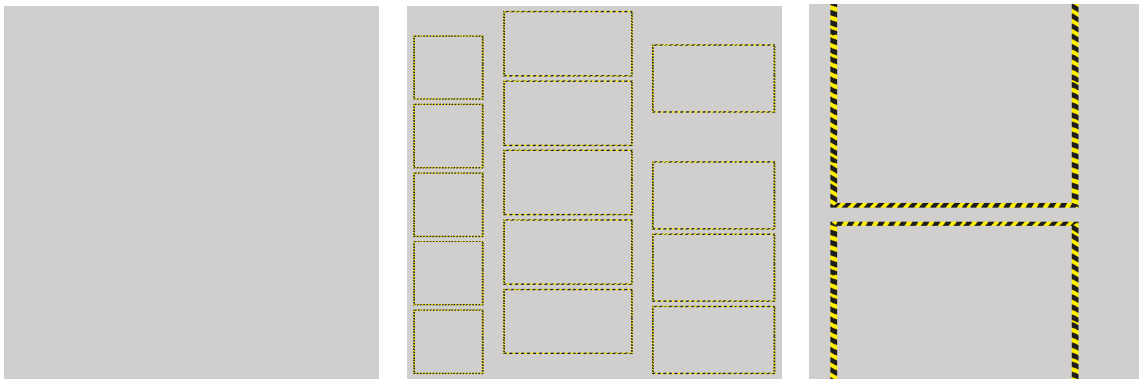
reconstructed map is scored.



(a) static `_01`, distributed (b) `objonwallonly` (c) dynamic

Figure 3.8 Baked occupancy truth of the three large-warehouse object arrangements, each 42.2 m square at 0.05 m per cell with origin $(-21.1, -21.1)$ m. Black is occupied, white is free; there is no unknown space because the grids are rasterised from collision geometry rather than observed. Occupied cells are 17.79 % of (a), 8.74 % of (b) and 10.98 % of (c). Panel (b) clears the central floor and keeps obstacles against the walls, which is the arrangement used by `dataset_allsensor_002` and `dataset_allsensor_003`. *No occupancy grid is shown per floor treatment, and none exists:* the `bayline` and plain floors differ only in surface material, which carries no collision geometry, so their baked grids are identical to the panels above. See figure 3.9.

The floor treatment is a separate factor and is invisible to occupancy. It is set by which ground model a world includes; those models share one mesh and differ only in the visual materials attached to it (table 3.14). Figure 3.9 renders both recorded conditions at true scale using `script/render_floor.py`, which drives the 2-D viewer's own floor rasteriser so that the figure shows what the overlay and the simulated cameras see rather than a texture file out of context.



(a) plain, `allsensor_000`–`_002` (b) `bayline`, `allsensor_003`, `_004` (c) 12 m detail of (b)

Figure 3.9 The two recorded floor conditions, rendered across the full 42.2 m floor at 0.05 m per pixel. Both share the same flat diffuse base, RGB $(0.814, 0.811, 0.804)$; the only difference between them is the 112 painted marking faces that `bayline` adds, forming the hazard-striped outlines of fourteen storage bays. Panel (c) is a 12 m square detail showing the strips at 0.145 m width. Everything not outlined is untextured in both conditions.

Each recorded dataset is the combination of three things that vary independently and are easy to confuse when read off three separate artifacts: the floor treatment, the object

Table 3.14 Ground models and the floor conditions they produce. The mesh and its collision geometry are identical in all three, so the floor treatment changes what a camera sees and nothing a wheel, an occupancy grid or a collision check responds to. The concrete-grain texture belongs to the stock model only, and no recorded dataset uses it.

Ground model	Base	Concrete	Markings	Used by
`..._GroundB_01` (stock)	texture	yes	112 faces	no recorded dataset
`..._GroundB_01_bayline`	flat diffuse	no	112 faces	`allsensor_003`, `allsensor_004`
`..._GroundB_01_plain`	flat diffuse	no	none	`allsensor_000–_002`, all `nofloortexture` datasets

arrangement, and the route the robot actually drove. Figure 3.10 draws all three together for every dataset, and table 3.15 gives the numbers behind the pictures. Both are produced by `script/render_dataset_scene.py`, which resolves each dataset's world from its own run manifest rather than from its name—necessary because `dataset_static_nofloortexture_001` sits in the `_01` arrangement while `_000` and `_002` sit in the canonical one, and all three carry `nofloortexture` in the name.

The available controlled factors are summarised in table 3.16.

The plain floor removes the painted bay outlines; neither recorded condition carries the stock concrete texture. Collision geometry and occupancy truth are preserved in both (table 3.14). Dynamic props follow deterministic routes through a kinematic Gazebo plugin. This preserves repeatable motion and acceptable real-time factor, but it does not reproduce physical collision response if the robot overlaps a moving prop.

World identity is recorded by exact `.world` path. ROS launches require `colcon build --symlink-install`; a plain build can leave an obsolete copied world in `install/`. When object motion is required by the 2-D viewer, model poses must also be bridged from the simulator.

3.4.6 Realistic logistics layouts

Status: implemented after the reporting cutoff (6 October 2026); no result slot in chapter 4 uses these worlds yet. The object arrangements of section 3.4.5 were built as controlled stimuli: props distributed over the floor, props confined to the wall region, and props shuttling back and forth along fixed segments. None of them is laid out as a warehouse is laid out, and none of their dynamic props does anything a warehouse vehicle does. Two further worlds were therefore generated from the warehouse-design literature collected under `docs/references/warehouse/`, whose common prescription is a chain of zones—receiving, storage, picking, packing, dispatch—served by cross-aisles, with the fast-moving goods nearest the docks, bulk storage at the back or side, and offices and equipment charging out of the flow [9, 10]. The two canonical flow shapes of that literature are both provided:

- **U-shape flow**, `worlds/small_warehouse_dynamic/small_warehouse_dynamic_realistic_nofloortexture.world`: receiving and

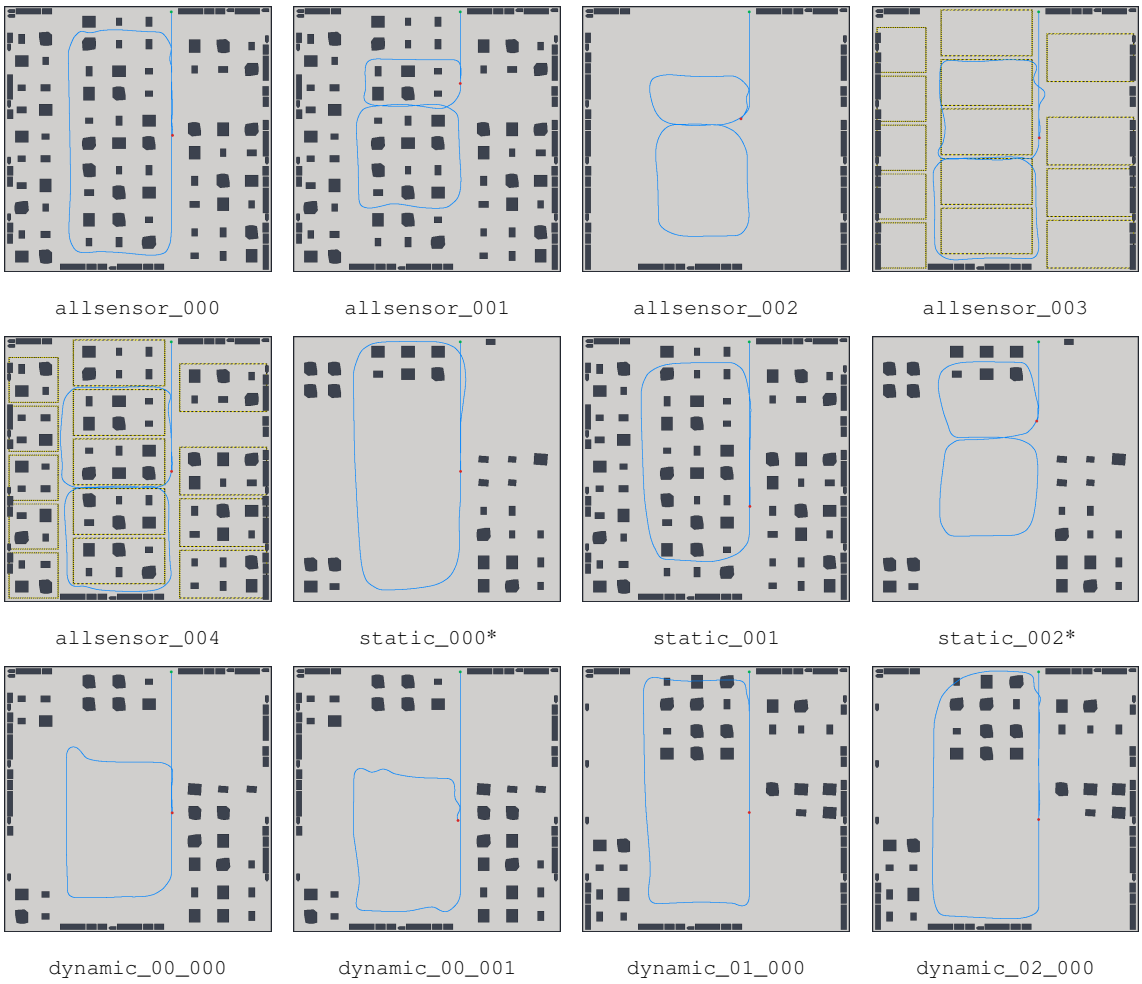


Figure 3.10 Every simulation dataset as a scene: floor treatment, object layout and ground-truth route in one view. Grey is open floor; dark polygons are obstacle footprints, taken as convex hulls of each collision mesh sliced at robot height, the same geometry the occupancy bake uses. Yellow hazard striping is the painted bay outline of the `bayline` floor and is absent on plain floors. The blue trace is the recorded `/ground_truth/odometry` route, green marking its start and red its end. Moving props are drawn at their authored pose, which is where the world file places them at $t = 0$; this is a layout figure and does not show where a prop was at any instant during a dynamic recording. * marks a dataset whose recorded world no longer exists and whose objects were reconstructed—see the note below table 3.15.

Table 3.15 What each dataset contains. "Objects" counts obstacle instances whose collision geometry intersects the robot-height band, excluding the wall ring. "Route" is the ground-truth path length and "sim span" its duration in simulated time. The world column drops the floor suffix, which is given separately, so two rows sharing a world arrangement but differing in floor are visible as such.

Dataset	World arrangement	Floor	Objects	Route [m]	Sim span [s]	Truth samples
dataset_allsensor_000	static_01	plain	117	117.0	87.1	4354
dataset_allsensor_001	static_01	plain	117	113.1	81.8	4091
dataset_allsensor_002	static_objonwallonly	plain	49	115.2	86.7	4338
dataset_allsensor_003	static_objonwallonly	bayline	49	140.6	102.4	5119
dataset_allsensor_004	static_01	bayline	117	142.3	100.5	5026
dataset_dynamic_nofloortexture_00_000	dynamic_00	plain	69	97.1	77.4	3873
dataset_dynamic_nofloortexture_00_001	dynamic_00	plain	69	97.3	74.2	3710
dataset_dynamic_nofloortexture_01_000	dynamic_01	plain	68	122.8	84.6	4230
dataset_dynamic_nofloortexture_02_000	dynamic_02	plain	68	126.6	89.5	4475
dataset_static_nofloortexture_000*	static	plain	31	122.9	86.7	4334
dataset_static_nofloortexture_001	static_01	plain	117	113.1	82.0	4100
dataset_static_nofloortexture_002*	static	plain	31	114.7	89.5	4475

Table 3.16 Controlled experimental factors available in the simulation environment, with the levels realised at the reporting cutoff. A factor is listed as exercised only where a recorded dataset exists for more than one of its levels.

Factor	Levels	Status at cutoff
Scene dynamics	static props; scripted dynamic props	exercised
Floor markings	bay outlines present (`bayline`); absent (plain)	exercised; no dataset carries surface texture
Prop or route arrangement	canonical; `_00`; `_01`; `_02`	exercised
Stocking level	partial; fully stocked static	fully stocked recorded
Object placement	distributed; restricted to the wall region	exercised, three bags to one
Domain	simulation; real robot	exercised, no real ground truth
World scale	tiny `small_warehouse`; large rescaled families	distinct environments, never pooled

dispatch docks on the same (north) wall with the management office between them, the fast pick faces nearest the docks, rack storage in north–south aisles, picking and packing east, block storage west, and the charging area, empty-pallet store and general office along the south wall.

- **I-shape (through) flow**, `..._realistic_nofloortexture_01.world`: receiving docks on the north wall and dispatch docks on the south wall, rack storage in the middle, the pick faces between storage and dispatch, packing beside dispatch, block storage west and both offices on the east wall, so that goods cross the building in one direction.

Both are written by `script/make_realistic_world.py` (`-layout u` or `i`) from parametric tables and are never edited by hand. Everything that is not the object arrangement is held fixed: the same 41.28 m shell, the same plain (`nofloortexture`) floor, lights, physics configuration (table 3.11) and robot spawn. The only change to the shell is a regenerated wall model carrying dock-door openings, since the stock wall's geometry is frozen in its mesh. The controlled factors of table 3.16 are therefore unchanged; the new worlds add two levels to the *prop or route arrangement* factor and, as explained below, make *scene dynamics* a within-arrangement factor for the first time.

Zones and contents. Table 3.17 lists what each zone holds. Aisles are 3.0 m between zones and 2.8 m between rack rows; pallet pitches are 3.2–4.0 m, widened from the 2.4 m of a stock bay so that a loaded pallet jack has the 2.6 m of swing room it needs to turn. The dock doors are 3.6 m by 4.5 m openings in the wall; a closed door is a roll-up panel in the wall thickness, an open door shows a docked 13.6 m semi-trailer and a yard slab, so that a camera looking out through it sees a trailer interior and tarmac rather than the sky colour. The U-shape world has five doors on the north wall (two open), the I-shape five on the north and three on the south (three open in all).

Traffic as jobs, not shuttles. The dynamic props of the earlier family move back and forth along a segment. In the realistic worlds the moving objects are four pallet jacks on logistics jobs, driven by a second world-level Gazebo system, `gz_kinematic_trajectory/LogisticsScheduler.cc`: a carrier drives to a load, takes it onto its forks (the load's

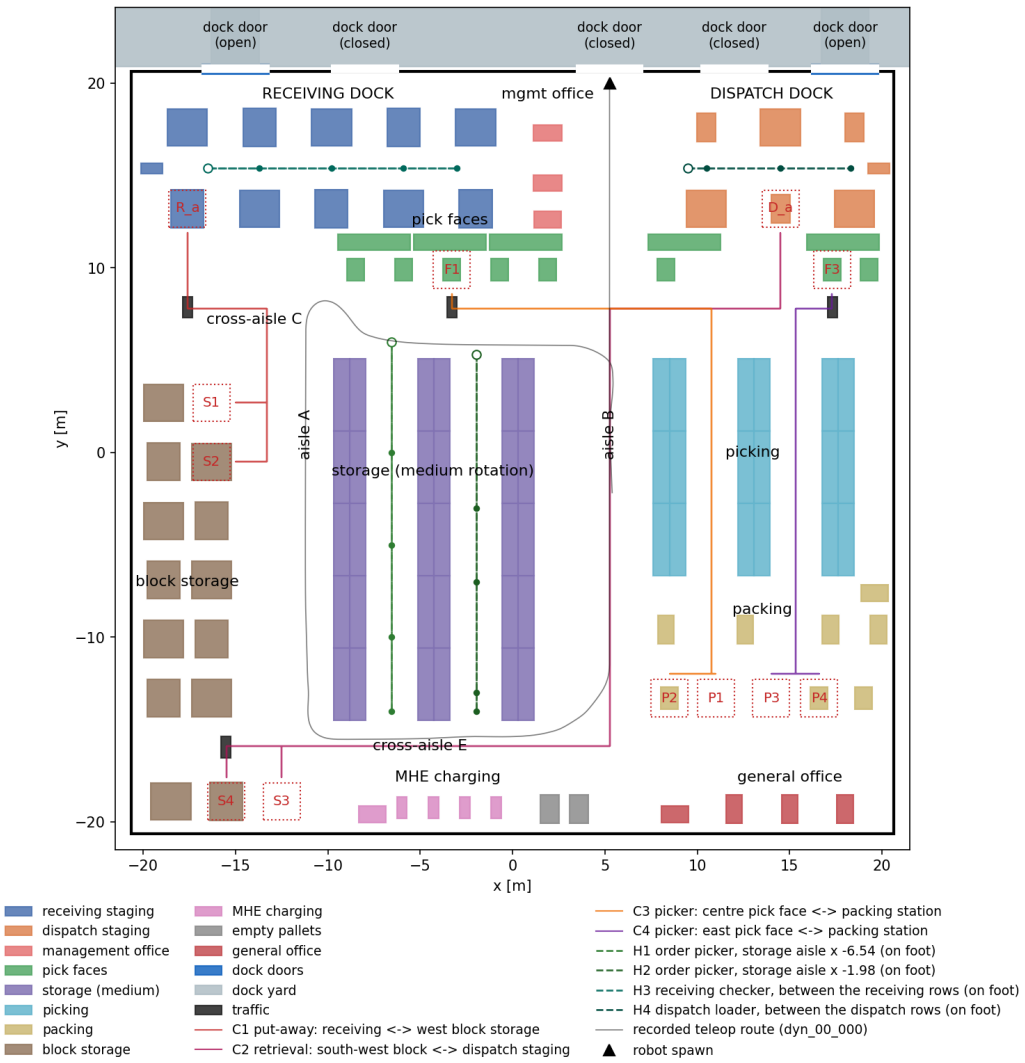


Figure 3.11 Plan of the U-shape realistic world, drawn by the generator from the same tables that write the world file. Filled rectangles are object footprints coloured by zone; dotted red boxes are the exchange slots the carrier jobs pick from and drop on (labels `R_a`, `S1`...`P4`); the coloured polylines are the four carrier routes; the dashed green lines are the routes of the four people on foot, with a dot at each place they stop and an open circle where they start; the grey trace is the ground-truth route of `dataset_dynamic_nofloortexture_00_000`, which the main aisles A and B and cross-aisles C and E were placed to keep drivable. The band above the north wall is the dock yard with the two docked trailers.

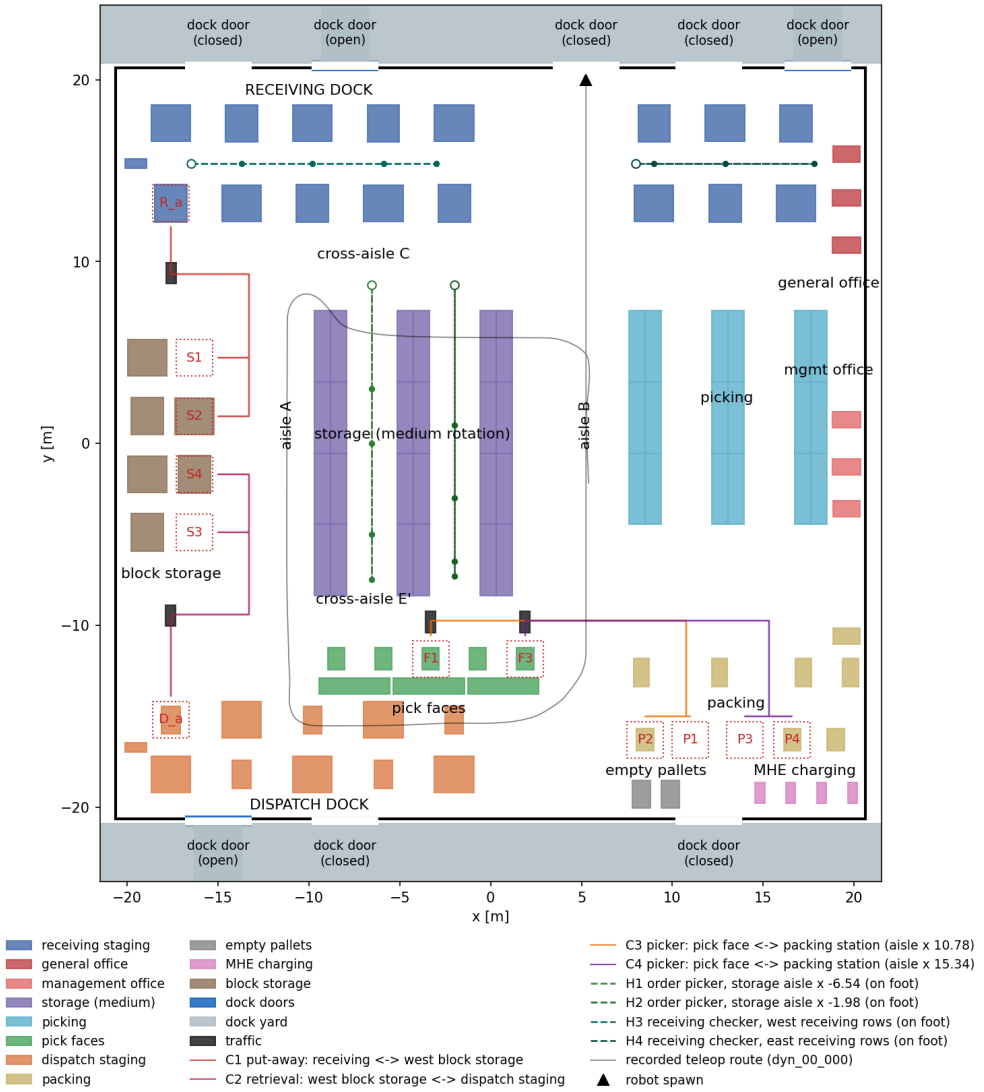


Figure 3.12 Plan of the I-shape realistic world, with the conventions of figure 3.11. Receiving docks are on the north wall and dispatch docks on the south wall; the pick faces sit between the storage racks and dispatch, and both offices are on the east wall because with docks on opposite walls the office cannot lie between reception and dispatch. The recorded teleop route (grey) crosses the storage racks here and is *not* drivable in this layout.

Table 3.17 Zones of the two realistic worlds and the number of object instances each holds, counted from the generator's tables. "Inside" excludes the dock doors, trailers and yard, which lie in or beyond the wall line. The four carriers are the pallet jacks the traffic plugin drives.

Zone	Contents	U-shape	I-shape
Receiving staging	pallet loads in two rows, one parked jack	11	17
Dispatch staging	pallet loads in two rows, one parked jack	7	11
Pick faces	east–west shelves with totes in front	13	8
Storage, medium rotation	three double rack rows	30	24
Picking	three double rack rows	18	18
Packing	four desks, totes, bin	8	8
Block storage	floor-stacked pallets, one empty cell per job	13	6
Management office	two desks, bin	3	3
General office	desks, bin	4	3
MHE charging	parked pallet jacks, bin	5	4
Empty pallets	stacks	2	2
Carriers (traffic)	pallet jacks on jobs	4	4
People on foot (actors)	order pickers and dock checkers; not objects, not counted below	4	4
Inside the walls		118	108
Dock doors, trailers, yard		8	13
Total object instances		126	121
Baked occupancy (% of cells)		20.7	19.1

pose relative to the carrier is captured at pick-up and held rigidly through every turn), carries it through the aisles, sets it down on a target slot and goes for the next. Two jacks move pallets between receiving, block storage and dispatch; two move totes between the pick faces and the packing desks. Each job is a six-move rotation over two loads and three slots that returns to its starting state, so it loops indefinitely and no drop ever lands on an occupied slot. Carriers move at 1.2–1.3 $\mathrm{m\,s^{-1}}$, turn in place at 1.5 $\mathrm{rad\,s^{-1}}$, spend 4–5 s on each pick or drop, and carry the load 1.3 m ahead of the body. Slots are entered head-on and left in reverse, as a real jack does; the first design spun the loaded jack on the spot and swung the pallet into the neighbouring loads. The mechanism is the same kinematic pose command used by the earlier dynamic props, so the fidelity caveat of section 3.4.3 is unchanged: the vehicles cost no physics and offer no collision response if the robot drives into them.

Before a world is written the generator proves it: every static pair is at least 0.25 m apart (rack runs excepted), every job's slot bookkeeping is simulated, and for every leg and every turn the swept box of the carrier plus its load is checked against every object not in that carrier's own job. Both worlds pass with every clearance positive, the tightest being 0.10 m at a packing-desk turn in each. Carriers can still cross one another briefly at an aisle intersection; that is a visual overlap, not a path conflict, and it is accepted.

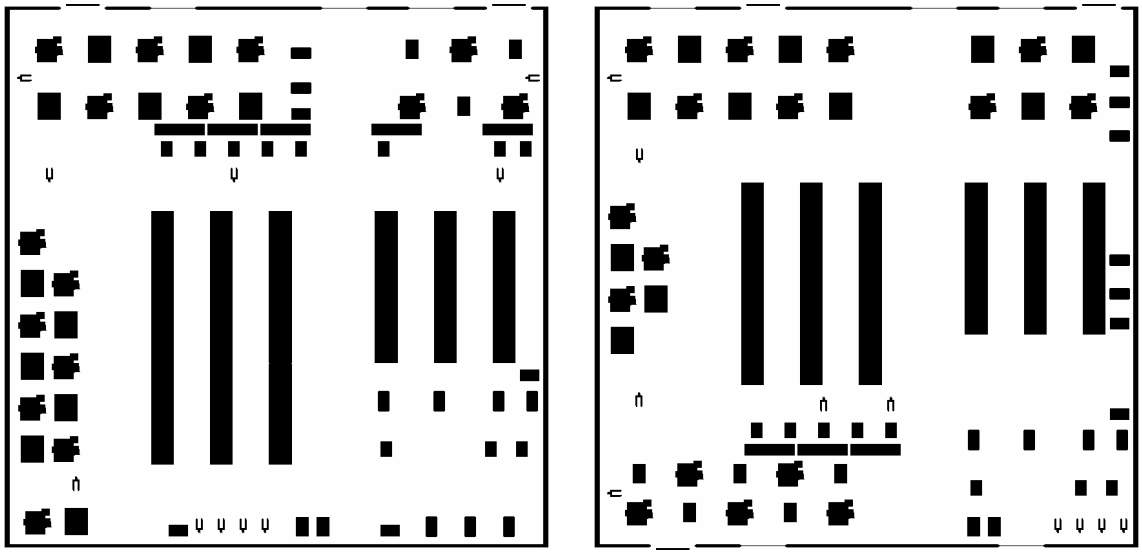
People on foot. A warehouse floor also has people on it, and they are the dynamic objects a visual pipeline finds hardest: articulated, self-occluding and close to the camera. Each world therefore carries four walking people (added 9 October 2026), written as Gazebo *actors*: the stock Fuel walking-person skin, packaged as `models/aws_robomaker_`

warehouse_Person_01, animated with a walk clip between stops and a stand clip at them, on a scripted closed route with one spline per leg so that legs stay straight in the aisles. The roles follow the same references as the layout and the Eaton case study [11]: two order pickers, one per storage aisle, who walk the aisle, stop at bays for six seconds and turn at the ends; and two dock checkers who walk the lane between the staging rows and stop at loads—in the U-shape one at receiving and one at dispatch, in the I-shape one on each receiving group. Walking speed is 1.1–1.2 $\mathrm{m\,s^{-1}}$, starts are staggered, and the number was kept small deliberately: four people on 1704 $\mathrm{m^2}$ with four vehicles is a working floor, not a crowd. People keep out of the vehicle lanes by construction. The generator sweeps a 0.7 m person box along every leg and checks it against the statics (at least 0.25 m), the carrier corridors and turn sweeps (no overlap) and the other people; the tightest clearances are 0.59 m to a static and 0.34 m to a carrier sweep. The pickers turn at the aisle mouths, which places them in view of a robot driving the cross-aisle. Actors have no collision geometry and no physics: nothing can bump into them, the occupancy bake does not see them, and the static twins omit them. One limitation matters for scoring: a scripted actor's animated pose is computed on the rendering side and is not written back to the entity state, so /world/default/pose/info reports a person at its initial pose throughout. The script is deterministic, so the pose is instead evaluated from the world file and the simulation clock by script/people_script.py, which the 2-D viewer uses to draw the people (linear between the 2 m waypoints, within 0.2 m of the rendered spline). Projecting a detector ground truth from it is not yet implemented, so Experiment 2 cannot yet score the people and Experiment 3 sees them only as unlabelled motion.

Static twins. For each layout a static world, worlds/small_warehouse_static/small_warehouse_static_realistic_nofloortexture.world and its ..._01.world counterpart, is derived by script/make_static_world.py: every object is marked static and the traffic plugin removed, with nothing else changed. This is the methodological point of the family. In the earlier worlds the static arrangement (_01) and the dynamic arrangements (_00 to _02) differ in geometry as well as in motion, so a static-versus-dynamic comparison on them is confounded by the arrangement. A realistic world and its twin share every footprint and differ only in whether four vehicles move.

Occupancy truth and routes. The baked grids (figure 3.13) are produced exactly as those of figure 3.8, with the moving carriers and their loads rasterised at their $t = 0$ pose, which is where the static twin holds them permanently. The closed dock doors appear as a one-cell line in the wall; the open ones are gaps onto the yard. The U-shape world keeps the main aisles A ($x \approx -11.2$ m) and B ($x = 5.25$ m, the dock lane the robot spawns in) and the cross-aisles C and E where the recorded teleop route of dataset_dynamic_nofloortexture_00_000 drove, which that route clears by at least 0.69 m; the other three recorded routes wander into what is now receiving staging. The I-shape world fits none of the recorded routes and needs new recordings.

First recording. One dataset has been recorded in the family, dataset/dataset_map_realistic (8 October 2026), shown in figure 3.14 and summarised in table 3.18. It is the first recording outside the dataset_allsensor set to carry the proprioceptive topics, so the baseline of section 3.6.1 can be computed on it, and its median speed of 1.38 $\mathrm{m\,s^{-1}}$ is the closest any simulation recording comes to the 1.5 $\mathrm{m\,s^{-1}}$ of section 3.1. The route is a single anticlockwise loop: down the dock lane, west along cross-aisle C, south through aisle A, east along cross-aisle E and north between the picking racks and the



(a) U-shape, 20.7 % occupied

(b) I-shape, 19.1 % occupied

Figure 3.13 Baked occupancy truth of the two realistic worlds, in the frame of figure 3.8: 42.2 m square at 0.05 m per cell, origin $(-21.1, -21.1)$ m. Carriers and loads are baked at their $t = 0$ pose. Closed dock doors are the one-cell segments in the north (and, in (b), south) wall; open doors are the gaps.

east wall, ending in dispatch staging.

Table 3.18 What `dataset_map_realistic` contains, read from the bag's SQLite store rather than its `metadata.yaml`, following section 3.4.7. The simulator ran below real time, so the simulated span is shorter than the recording's wall-clock duration.

Quantity	Value
Recorded	8 October 2026, 166.5 s wall clock, 126.9 s simulated
World	U-shape realistic layout, props stationary (see query)
Floor	plain (`nofloortexture`)
Ground-truth route	151.1 m, 6 347 samples, start $(5.25, 20.0)$ m
Speed	median $1.38\,\mathrm{m\,s^{-1}}$, 95th percentile $1.51\,\mathrm{m\,s^{-1}}$
Images	3 847 compressed stereo pairs
IMU	25 387 messages
Proprioception	126 938 joint states, 6 347 wheel-odometry messages
`/clock`	present (126 942 messages)
Run manifest	none

3.4.7 Dataset registry

Datasets are held in two places and the registry must cover both: the workspace tree `dataset/` and a removable drive mounted at `/media/ambushee/32E4AAB1E4AA772F/`, whose simulation bags sit under `dataset/` and whose real-robot recordings sit at its root. Table 3.19 lists all eighteen recordings across both locations. Six exist in one place only on the drive, seven in the workspace only, and three are held in both.

The seven featureless-floor simulation bags of the relogged campaign (section 4.3.1) are the static and dynamic families listed above; five were read from the drive and two from the workspace, for the reason recorded next.

Table 3.19 Complete dataset registry at the reporting cutoff, across both storage locations. The `bayline` families carry a textured floor; every other simulation entry is featureless. "WS" is the workspace tree `dataset/` and "FD" the removable drive; a size is given where that location holds a copy. "Prop." marks the proprioceptive group (`/model/ackermann_robot_001/joint_state` and `/model/ackermann_robot_001/odometry`); "IMU" counts inertial messages, on `/imu` for simulation and `/hwt101ct_yaw_publisher` for the real rig. Durations are rosbag wall-clock recording durations: the simulator ran below real time, so a bag's sim-time span is shorter than the figure shown. Integrity was checked by opening each SQLite store and counting rows against the count its `metadata.yaml` declares, not by trusting the metadata.

Dataset	Family	WS (MB)	FD (MB)	Dur. (s)	Messages	`/clock`	Truth	Prop.	Integrity
`dataset_allsensor_000`	static `_01`	689	–	129.3	211 811	✓	✓	✓	ok
`dataset_allsensor_001`	static `_01`	674	–	126.2	198 873	✓	✓	✓	ok
`dataset_allsensor_002`	static, wall only	750	–	105.5	215 528	✓	✓	✓	ok
`dataset_allsensor_003`	bayline, wall only	1056	–	141.2	255 699	✓	✓	✓	ok
`dataset_allsensor_004`	bayline, distributed	1311	–	168.6	253 942	✓	✓	✓	ok
`dataset_static_nofloortexture_000`	static	–	592	101.5	118 597	✓	✓	–	ok
`dataset_static_nofloortexture_001`	static	617	1	92.0	113 332	✓	✓	–	FD stub, WS ok
`dataset_static_nofloortexture_002`	static	–	688	103.3	122 386	✓	✓	–	ok
`dataset_static_nofloortexture_objonwallonly`	static, wall only	648	–	110.5	114 523	✓	✓	–	ok
`dataset_dynamic_nofloortexture_00_000`	dynamic `_00`	–	653	100.3	111 988	✓	✓	–	ok
`dataset_dynamic_nofloortexture_00_001`	dynamic `_00`	–	778	167.7	114 169	✓	✓	–	ok
`dataset_dynamic_nofloortexture_01_000`	dynamic `_01`	713	710	165.4	126 790	✓	✓	–	FD malformed, WS ok
`dataset_dynamic_nofloortexture_02_000`	dynamic `_02`	–	758	172.1	134 995	✓	✓	–	ok
`dataset_map_01`	mapping, large	818	–	193.5	152 291	✓	✓	–	ok
`dataset_map_tiny`	mapping, tiny world	726	–	63.3	82 555	✓	✓	–	ok
`dataset_real_000`	real rig	2964	2964	116.6	36 166	–	–	–	ok, 23 257 IMU
`dataset_real_001`	real rig	–	2099	85.5	9 629	–	–	–	0 IMU messages
`dataset_real_002`	real rig	–	2869	113.5	12 682	–	–	–	0 IMU messages

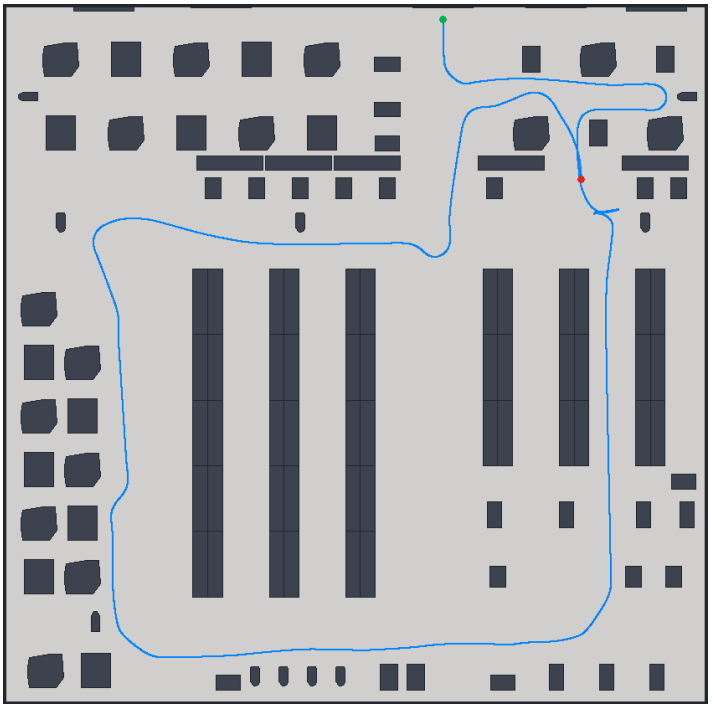


Figure 3.14 Scene of `dataset_map_realistic`, drawn with the conventions of figure 3.10: obstacle footprints at robot height over the plain floor, the `/ground_truth/odometry` route in blue from the green start to the red end. The objects are drawn from the U-shape static twin, the world the bag's own model inventory identifies (see the query below).

Historical output folders do not prove that their raw bags remain available or valid. Conversely, a dataset is not marked broken merely because its only copy lives on the removable drive rather than in the repository; six of the fifteen recordings are drive-only and are sound.

Before a new experiment, the bag's absolute path, topics, duration, message counts, and storage health are inspected. Each paired comparison uses the same recording, and estimators run separately to avoid resource-contention effects. Environment, trajectory, calibration, replay rate, image transport, initialization treatment, and revision are controlled. A run is excluded if input overlap, initialization, processed sample count, or sensor delivery differs materially from its paired condition.

Recorded simulation bags used by RTAB-Map contain `/clock`. They are played without rosbag's additional `-clock` option because competing clock sources cause TF time jumps. This choice is made after inspecting each bag, not applied as a universal command rule.

Recordings added after the cutoff. Table 3.19 is the registry at 25 September 2026. Three recordings have been added since and are listed here so the registry stays complete; none is used by a result slot in chapter 4. `dataset_map_realistic` (8 October 2026) is the first recording in the realistic family and is described in section 3.4.6. `dataset_realv2_000` (35 s) and `dataset_realv2_001` (96 s) were recorded on the real robot at the Gensurv site on 1 October 2026 and are the first real recordings with an independent ground truth: a planar optical tracker at about 7 Hz, stored under `dataset/gt/`. The tracker and the robot keep separate, unsynchronised clocks, so the time offset must be estimated per bag before scoring (`script/eval_real_gt.py`); that scoring is provisional and is not reported in this document.

3.5 Real Platform

3.5.1 AMR mobile robot testbed

The system is installed and tested on a Gensurv Robotics mobile-robot base designed to reproduce the behaviour of an industrial warehouse AMR (figures 3.15 and 3.16). Its maximum speed is set to $1.5\,\mathrm{m\,s^{-1}}$, the limit of the SLAM stability tests. The drivetrain is a rear brushless motor (Flipsky 6384) through a BLDC drive with a Dynamixel RX-64 steering servo, a GigaDevice GD32407V microcontroller on a CAN link to the NRU-51V, an LM2596 regulator from a lithium-polymer pack, and a Wi-Fi router for tele-operation from a smartphone. The faculty template describes the base as a differential-drive robot; the hardware diagram (a single traction motor with a steering servo) and the simulated model of figure 3.6 are Ackermann. The description here follows the hardware and should be confirmed by the author.

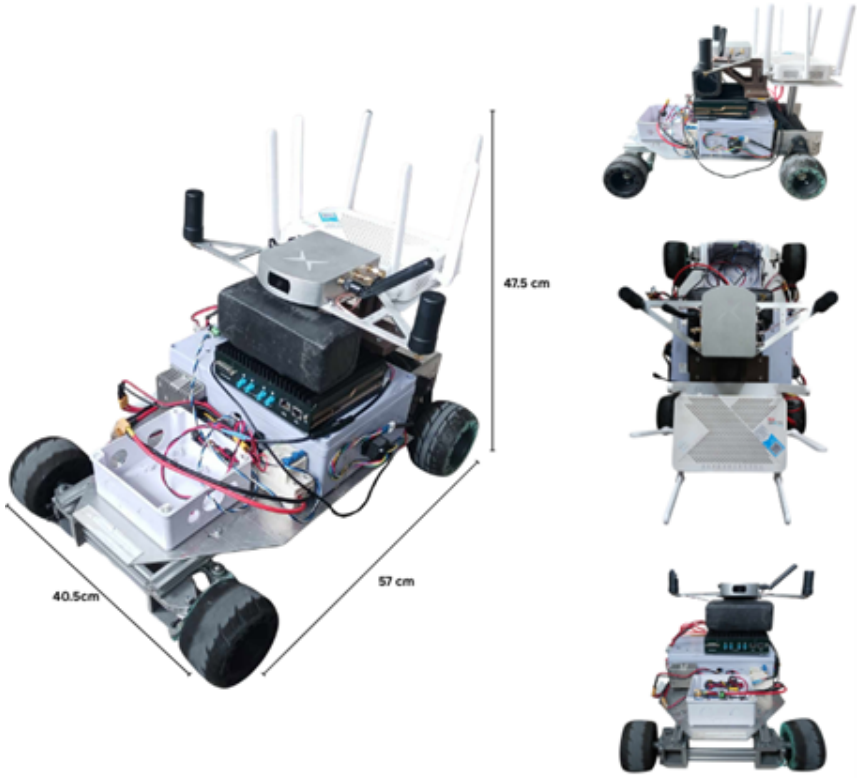


Figure 3.15 Dimensions of the mock AMR used to test the navigation system: 57 cm long, 40.5 cm wide and 47.5 cm high to the top of the sensor mast.

3.5.2 Sensors and computing unit

Wide-angle stereo cameras (AC-IMX390-H190). Two AC-IMX390-H190 modules form the stereo pair (figure 3.17); table 3.20 lists the specification.

Inertial measurement unit. A six-axis IMU measures linear acceleration and angular rate at 100–200 Hz. Its data enter IMU pre-integration and are fused with the camera images to support the odometry estimate while the camera moves fast or the image is momentarily blurred. On the recorded real datasets the inertial stream is the `/hwt101ct_yaw_publisher` topic (table 3.22).

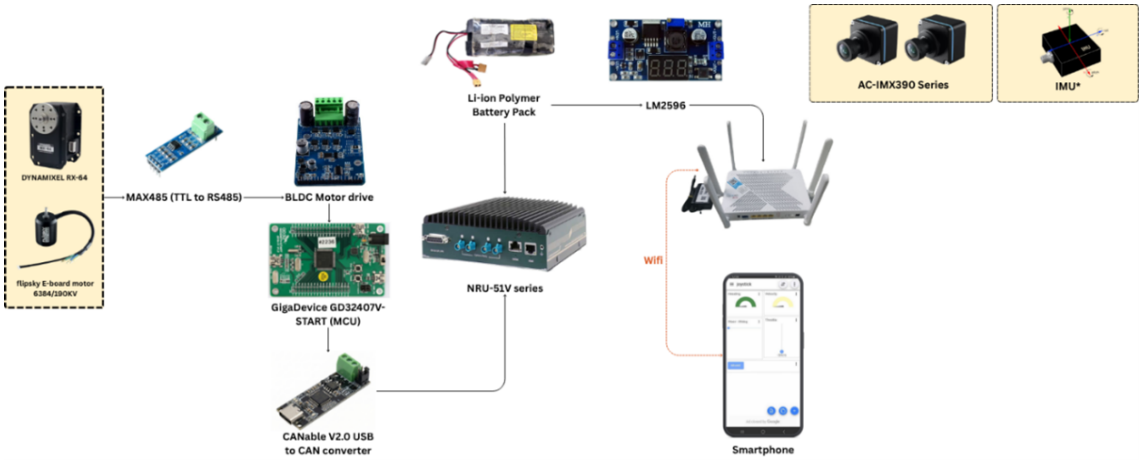


Figure 3.16 Hardware components of the mock AMR: steering servo and traction motor, BLDC drive and microcontroller on CAN, the NRU-51V computer, power supply, the AC-IMX390 stereo pair and IMU, and the Wi-Fi link to the smartphone joystick.

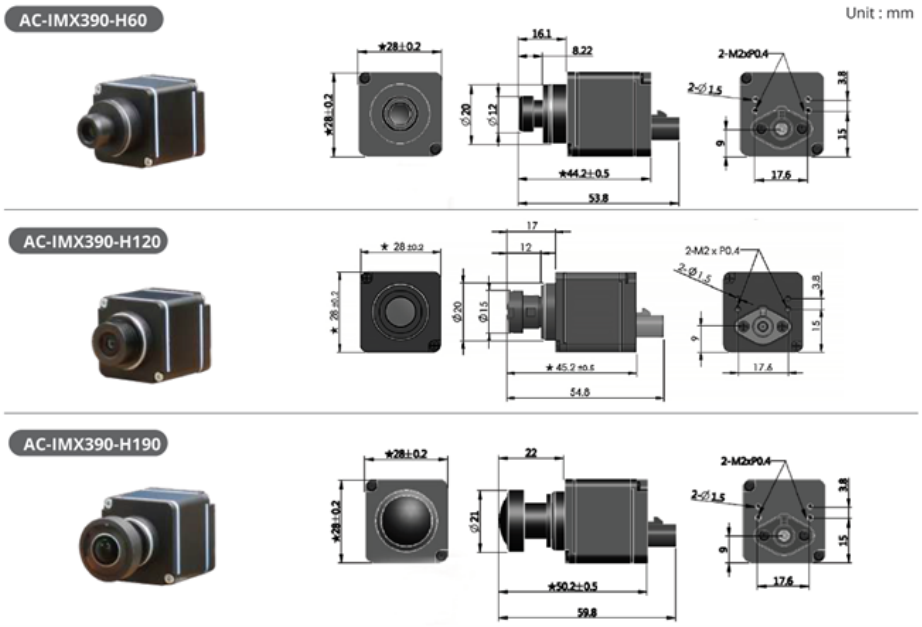


Figure 3.17 Dimensions of the AC-IMX390 camera family in millimetres; the H190 variant at the bottom is the one used.

Table 3.20 Specification of the AC-IMX390-H190 camera.

Item	Detail
Sensor and ISP	Sony IMX390 CMOS sensor with GEO Semiconductor GW5200 ISP
Resolution and frame rate	1920 × 1080 (2 MP) at 30 fps
Horizontal field of view	186°
Video output	uncompressed YUV422 (UYVY)
Dynamic range	120 dB high dynamic range (HDR)
Special features	LED flicker mitigation (LFM), 5-axis active alignment
Environmental rating	IP67 and IP69K (lens side)
Operating temperature	−40–85 °C
Connector	male FAKRA

Edge AI computing unit: Neousys NRU-51V (NVIDIA Jetson Xavier NX). To run the deep-learning model (YOLO26) and the visual-inertial SLAM on the robot without a central computer, the project uses the Neousys NRU-51V, a compact industrial edge computer designed for vision and autonomous-vehicle systems (figures 3.18 and 3.19). Table 3.21 summarises the hardware.



Figure 3.18 The Neousys NRU-51V (NVIDIA Jetson Xavier NX), showing the four FAKRA camera ports.

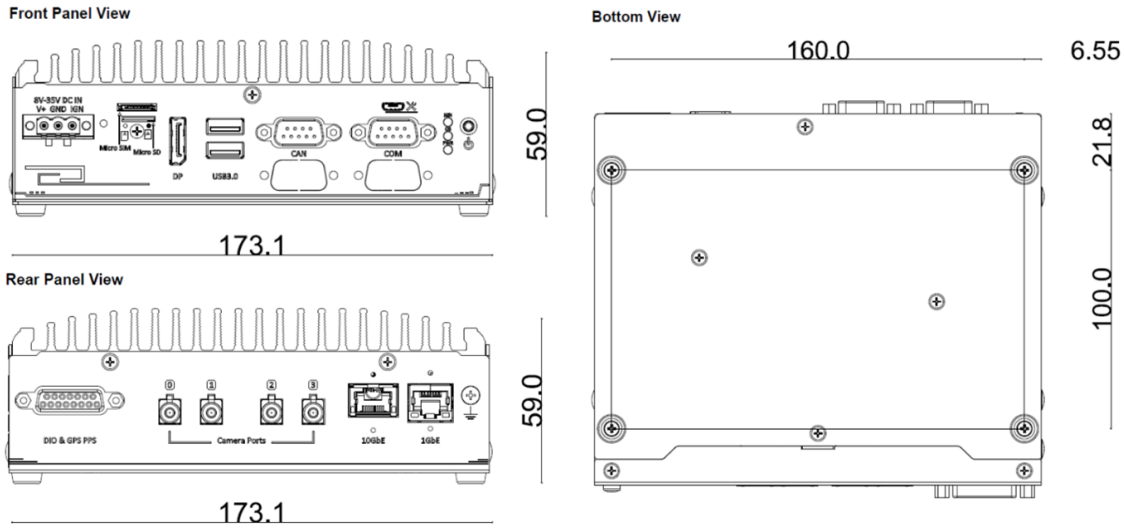


Figure 3.19 Dimensions of the Neousys NRU-51V in millimetres.

3.5.3 Real sensor rig and calibration limits

The real configuration is summarised in table 3.22. Its calibration is materially weaker than the simulated rig's, and the weaknesses are load bearing rather than cosmetic.

The real IMU-to-camera rotation has observational support, but its translation is a zero placeholder. The stationary accelerometer magnitude is recorded as $9.2642\,\mathrm{m\,s^{-2}}$, rather than $9.81\,\mathrm{m\,s^{-2}}$. VINS-Fusion uses the observed magnitude as `g_norm`; the ORB-SLAM3 real inertial launch rescales acceleration by $9.81/9.2642$. These treatments make the configurations internally consistent but do not replace an independent camera–IMU calibration. Real trajectories can be inspected for plausibility and tracking continuity, but metric accuracy cannot be claimed without independent ground truth and improved extrinsic calibration.

Table 3.21 Summary of the hardware technical specification.

Device / sensor	Model / specification	Role in the system
Edge processing unit	Neousys NRU-51V (NVIDIA Jetson Xavier NX, 21 TOPS)	runs YOLO26, the SLAM system and the ROS 2 navigation stack
Stereo camera module	AC-IMX390-H190 (Sony IMX390 sensor)	records 186° stereo imagery for object detection and feature extraction
Inertial sensor (IMU)	6-DoF industrial IMU (100–200 Hz)	measures acceleration and angular rate to support pose estimation (VIO)
Mobile testbed	AMR platform (speed 1.5 $\mathrm{m\,s^{-1}}$, slope 5°)	robot base for online navigation tests in the mock environment

Table 3.22 Real sensor rig, and the calibration status of each quantity. Only the first three rows are measured; the remainder are approximations or placeholders that bound what can be claimed from real-robot data.

Quantity	Value	Status
Stereo resolution	1920×1080 fisheye	measured
Stereo baseline	0.093 604 m	measured
IMU stream	`/hwt101ct_yaw_publisher`	separate RealSense source
Camera model (ORB-SLAM3)	`KannalaBrandt8`	configured
Camera model (VINS)	project fisheye files	configured
Second-camera intrinsics	reuse of the first camera	approximation
IMU-to-camera rotation	observational support	partial
IMU-to-camera translation	zero	placeholder
Stationary $\|\|a\|\|$	9.2642 $\mathrm{m\,s^{-2}}$	measured, deviates from 9.81
Extrinsic estimation (VINS)	`estimate_extrinsic: 1`	online
Time-offset estimation (VINS)	`estimate_td: 1`	online

3.5.4 Test site and experimental variables

Spatial layout. The test area is divided into two operating zones, Zone A and Zone B, separated by and connected through a walkway (each zone about 3 m square, walkway 1.25 m wide; figure 1.2). The division reproduces the traffic pattern of a warehouse AMR that shuttles continuously between a pick-up and a drop-off point. The robot's linear speed is limited to $v_{\max} = 1.5\,\mathrm{m\,s^{-1}}$ throughout.

Ground truth by motion capture. Evaluating SLAM error (ATE and RPE) needs a reference accurate to millimetres. A motion-capture camera system is installed around the test area and tracks reflective markers placed at the robot's centre of mass at a rate above 100 Hz.

Coordinate-frame alignment. The SLAM pose P_{slam} is expressed in the local odometry frame fixed at start-up, whereas the motion-capture pose P_{gt} is expressed in the room's global frame. Before errors are computed the two trajectories are aligned by a rigid-body transformation; the alignment actually used for every result in this report, and its treatment of scale, is defined in section 3.7.1.

Dynamic-environment variable. Each time the robot completes a zone transfer (for example arrives in Zone B from Zone A), the operator randomly re-positions and re-counts the stacks of goods in the opposite zone before the robot returns. This forces the SLAM system to face a scene that no longer matches its map (map obsolescence), to test whether ORB-SLAM3's Atlas or RTAB-Map's memory management can resolve the conflicting spatial evidence without tracking loss.

Illumination variable. Data are collected along the same route at three times of day representing different light levels and directions: morning (08:00), oblique natural light casting shadows across the test area; midday (12:00), maximum brightness, with possible high contrast or over-exposure in places; and evening (18:00), low light relying mainly on the building's artificial lighting, with raised image noise. The variable tests both the camera hardware (Sony IMX390 HDR) and the robustness of the two SLAM frameworks in keeping enough inlier features for odometry under variable light.

Real-dataset status. Three real recordings exist (table 3.19). Only `dataset_real_000` carries inertial data; `dataset_real_001` and `_002` contain zero IMU messages and support image-only work at most. No real recording carries motion-capture ground truth or wheel odometry at the cutoff, which is why every real-robot result slot in chapter 4 is planned rather than filled.

3.6 The Four Experiments

Each experiment below is defined by its objective, its process, its evaluation metrics, its constraints and the validity rule that decides whether a run counts. Each is run in both domains; chapter 4 reports each domain in its own slot.

3.6.1 Experiment 1: Non-visual baseline and estimator comparison

Objective. Establish a dead-reckoning baseline to determine whether the visual pipelines (ORB-SLAM3 and VINS-Fusion) genuinely outperform the robot's own wheel odometry, its IMU, and a wheel-plus-IMU extended Kalman filter, and whether the uncertainty each estimator reports describes the error it makes.

Process. Wheel odometry, IMU dead reckoning and the EKF are run offline on the same recordings the visual estimators consumed, from proprioceptive topics only. A calibration pass measures each sensor's noise and scale against ground truth first; the estimators then run and are scored on the same interval as the visual runs.

Evaluation metrics. Aligned and anchored ATE, final drift as a fraction of distance, final heading error, RPE at 1 s, and the covariance consistency measures NEES and $k\sigma$ coverage.

Validity rule. A run counts only if the planar assumption holds (maximum tilt below 1°), the IMU orientation field is not consumed, and the number of wheel corrections equals the number of wheel-odometry messages.

The visual estimators were, until the cutoff, scored only against ground truth. That establishes how large their error is but not whether it is good, because no alternative was measured on the same data. Three non-visual estimators were therefore implemented in `script/proprio_estimator.py` to supply a dead-reckoning floor: wheel odometry, IMU dead reckoning, and an extended Kalman filter fusing the two. They consume only proprioceptive topics and no images.

Implementation form and why it is offline

All three read the recorded bag directly and are deterministic functions of it. Run as live ROS nodes they would instead depend on replay rate, QoS delivery and CPU contention, none of which is a property of the estimator being compared. The cost is a stated asymmetry: the visual figures they are compared against were produced by live runs, so any timing-induced error is present on one side of the comparison only. This is revisited in section 4.1.1.

Sensor model derived from data

Every covariance field in the source bags is identically zero—0 of 17 416 IMU messages, 0 of 4 355 wheel-odometry messages, and 0 of 4 354 ground-truth messages in `dataset_allsensor_000`. Gazebo Fortress does not populate them, and `AckermannSteering` exposes no noise parameter. The noise model is consequently measured rather than read, by a `calibrate` step that writes `sensor_noise.yaml`; table 3.23 gives the result for both bags.

Two consequences follow directly. First, wheel *speed* is essentially unbiased on all five bags (scale 1.0002 to 1.0071), so distance travelled is not the error source. Second, the rear-wheel differential is a far worse yaw-rate estimator than the steering-angle bicycle model—residuals of 0.26–0.35 rad s^{-1} against 0.061–0.119—because the track and the wheel base are both 0.42 m and a solid rear axle slips through every turn. The bicycle model is selected automatically and was chosen on all five bags.

The orientation field must not be used

`sensor_msgs/Imu.orientation` in these bags is not an estimate. Measured over `dataset_allsensor_000` it equals ground-truth yaw minus exactly 90°, with residual standard deviation 0.000 00° and 0.000 00° of drift across 87 s: the plugin publishes the true orientation from the simulator's entity-component manager. Consuming it would give a "dead-reckoning" estimator a perfect attitude reference that no real IMU has,

Table 3.23 Sensor noise and scale measured from each bag by `proprio_estimator.py calibrate`. Scales are least-squares gains through the origin against ground truth; σ values are the residual standard deviations. A scale of 1.0 is an unbiased sensor. The bias random walks are assumed, not measured, because the longest stationary window is 1.54 s.

Group	Quantity	`_000` plain	`_001` plain	`_002` plain	`_003` bayline	`_004` bayline	Units
IMU	gyroscope z bias	6.48×10^{-5}	7.75×10^{-5}	9.28×10^{-5}	6.09×10^{-4}	2.58×10^{-5}	rad s^{-1}
	gyroscope σ	4.36×10^{-3}	4.24×10^{-3}	4.81×10^{-3}	5.03×10^{-3}	5.15×10^{-3}	rad s^{-1}
	gyroscope z scale	0.9990	1.0000	1.0002	1.0002	1.0000	–
	accelerometer x bias	0.0191	0.0304	0.0683	0.0309	0.0016	m s^{-2}
	accelerometer x scale	1.0199	1.0249	1.0669	1.0525	1.0465	–
	measured g	9.7809	9.8191	9.7827	9.8037	9.8346	m s^{-2}
Wheel	speed scale	1.0002	0.9985	1.0005	1.0071	1.0001	–
	speed σ	0.0407	0.0484	0.0496	0.0607	0.0505	m s^{-1}
	yaw-rate scale, bicycle	1.0110	0.9527	1.0034	0.9375	0.9250	–
	yaw-rate σ, bicycle	0.0659	0.0804	0.0705	0.1192	0.0613	rad s^{-1}
	yaw-rate scale, differential	1.2162	1.1442	1.2194	1.2748	1.1404	–
	yaw-rate σ, differential	0.2359	0.3074	0.2620	0.3477	0.2828	rad s^{-1}
Window	static duration	0.68	0.58	1.54	1.04	0.52	s
	maximum tilt	0.0016	0.0018	0.0015	0.0018	0.0017	°

and its output would measure ground truth. All three estimators therefore integrate `angular_velocity.z` instead, which drifts honestly by 2.52° over 87 s on `allsensor_000` and 1.70° over 81.8 s on `allsensor_001`. The same check on `allsensor_002` again returns a residual standard deviation of 0.000 000° against ground-truth yaw, so the exclusion is not specific to the first two recordings. Each run records `imu_orientation_used=NO` in its metadata so the exclusion is auditable.

Estimator definitions

Table 3.24 defines the three estimators. The EKF carries the planar state $\mathbf{x} = [p_x, p_y, \theta, v, b_\omega]^\top$, predicted at the 200 Hz IMU rate and corrected at 50 Hz by the wheels.

Table 3.24 Implemented non-visual estimators. "Anchoring" refers to the initial pose: dead reckoning has no global reference, so all three start from the ground-truth pose at their first sample, and this is recorded as an alignment decision rather than treated as neutral.

Estimator	Inputs	State	Correction
Wheel odometry	rear wheel speeds, steering angles (1 kHz)	$[p_x, p_y, \theta]$	none; $\mathbf{P}$ grows without bound
IMU dead reckoning	gyroscope z, accelerometer x (200 Hz)	$[p_x, p_y, \theta, v]$	none
EKF wheel + IMU	both of the above	$[p_x, p_y, \theta, v, b_\omega]$	wheel speed corrects v; $(\omega_{\text{gyro}} - \omega_{\text{wheel}})$ corrects b_ω

EKF formulation

State and inputs. The filter carries five planar states,

$$\mathbf{x} = \begin{bmatrix} p_x & p_y & \theta & v & b_\omega \end{bmatrix}^\top, \tag{3.4}$$

where b_ω is the gyroscope z bias. Roll and pitch stay within 0.002° across every recording, so a full 15-state inertial formulation would spend nine states on quantities this platform does not have, and their covariances would be unobservable noise in the consistency test that motivates the experiment.

The IMU is not a measurement here: it drives the prediction. Bias-corrected yaw rate and longitudinal acceleration act as the input,

$$\omega = \omega_{\text{gyro}} - b_\omega, \qquad a = a_x - b_a, \tag{3.5}$$

with b_a a constant measured during calibration rather than a tracked state. Propagation uses the midpoint heading $\theta_m = \theta + \frac{1}{2}\omega\,\Delta t$, which matters over 117 m of driving:

$$\begin{aligned} p_x &\mathrel{+}= v\cos\theta_m\,\Delta t + \tfrac{1}{2}a\cos\theta_m\,\Delta t^2,\\ p_y &\mathrel{+}= v\sin\theta_m\,\Delta t + \tfrac{1}{2}a\sin\theta_m\,\Delta t^2,\\ v &\mathrel{+}= a\,\Delta t, \qquad \theta \mathrel{+}= \omega\,\Delta t. \end{aligned} \tag{3.6}$$

Covariance and process noise. With $\mathbf{F} = \partial f/\partial \mathbf{x}$ and $\mathbf{G}$ mapping the noise inputs into the state,

$$\mathbf{P} \leftarrow \mathbf{F}\mathbf{P}\mathbf{F}^\top + \mathbf{G}\,\mathrm{diag}(\sigma_a^2,\ \sigma_g^2,\ \sigma_b^2)\,\mathbf{G}^\top. \tag{3.7}$$

The non-trivial entries are $F_{13} = -v\sin\theta_m\Delta t$, $F_{14} = \cos\theta_m\Delta t$, $F_{23} = v\cos\theta_m\Delta t$, $F_{24} = \sin\theta_m\Delta t$ and $F_{35} = -\Delta t$; the last couples bias error directly into heading, which is what makes b_ω worth estimating. In $\mathbf{G}$, accelerometer noise enters position as $\frac{1}{2}\cos\theta_m\Delta t^2$ and speed as Δt, gyroscope noise enters heading as Δt, and the bias row carries $\sqrt{\Delta t}$ rather than Δt because a random walk accumulates as $\sqrt{t}$.

Both IMU noise terms are therefore *process* noise. Nothing from the IMU appears in $\mathbf{R}$, because the IMU propagates the state rather than observing it.

Measurement. The wheels correct at 50 Hz with

$$\mathbf{z} = \begin{bmatrix} v_{\text{wheel}} \\ \omega_{\text{gyro}} - \omega_{\text{wheel}} \end{bmatrix}, \qquad \mathbf{H} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{R} = \mathrm{diag}\big(\sigma_{v,w}^2,\ \sigma_{\omega,w}^2 + \sigma_g^2\big)\,. \tag{3.8}$$

The second variance is a sum because that observation is a *difference* of two noisy sensors and inherits both. The update uses the Joseph form,

$$\mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}(\mathbf{I} - \mathbf{K}\mathbf{H})^\top + \mathbf{K}\mathbf{R}\mathbf{K}^\top, \tag{3.9}$$

symmetrised afterwards. The shorter $(\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}$ does not reliably stay positive definite across the roughly 16 000 updates in a run, and here the covariance is itself the measurement under test, so it cannot be allowed to decay.

Where the numbers come from. σ_a and σ_g are measured from the recording by the calibration pass (table 3.23), not taken from a datasheet, because every covariance field in these bags is identically zero. $\sigma_{v,w}$ and $\sigma_{\omega,w}$ likewise come from the wheel stream against ground truth.

Why the bias, and not the heading, is observed

The EKF's second measurement is deliberately an observation of the *gyroscope bias* and not of heading. Integrating the wheel yaw rate into a heading and correcting θ with it would reproduce the wheel estimator inside the filter and inherit its unbounded drift. Treating the instantaneous rate difference as an observation of b_ω keeps each sensor supplying what it measures well: the wheels know rate, the gyroscope knows change, and neither knows absolute heading, so heading remains unobservable and its variance correctly continues to grow. The update uses the Joseph form, which keeps $\mathbf{P}$ symmetric and positive definite across the roughly 16 000 updates in a run.

Measured scale errors are recorded but never divided out. Correcting them would mean calibrating the estimator with the ground truth it is about to be scored against; the scale error is a real property of encoder odometry and is retained.

3.6.2 Experiment 2: Offline YOLO detector performance

Objective. Measure standalone detector performance independent of SLAM, so that later filtering results can be attributed: how reliably the detector finds and names warehouse objects, and how much of the frame a mask built from its boxes would remove.

Process. The detector is run on every left-camera frame of the dynamic recordings; ground-truth boxes are constructed by projecting the simulator's own object geometry into the camera; predictions are matched by class at an IoU threshold; localization and class assignment are scored separately.

Evaluation metrics. Precision, recall, F1, recall on the moving subset, image-area coverage, and conditional class precision, recall and F1.

Validity rule. Overlays are inspected before any aggregate is accepted, because invalid pose stamps, mesh units or camera transforms produce plausible but wrong boxes.

Detector precision, recall, TP, FP, FN, moving/static recall, IoU, and image-area coverage are computed by `yolo_eval.py` at recorded confidence, matching-IoU, occlusion, minimum-size, and motion thresholds. Mask-use statistics are joined to trajectories by `timestamp_ns`. A semantic class is never treated as ground-truth motion.

Constructing the ground-truth boxes The detector is scored against boxes computed from the simulator's own state rather than against hand annotation. No image is labelled by hand anywhere in this project. The chain from a recorded object pose to a box in pixels is as follows; all of it is in `yolo_eval.py` and `yolo_gt.py`.

1. **Geometry, not just pose.** A pose is a point and a box needs extent, so `yolo_gt.vertices()` reads the prop's collision mesh (`.dae`) from the Gazebo model directory and returns its vertices V in the model frame. Meshes above 4000 vertices are decimated with a fixed seed: a few thousand points bound a silhouette as tightly as 100 000 and keep the per-frame cost flat.
2. **Object to world.** The pose (R_{wo}, p_{wo}) recorded for that object places the vertices: $v_w = R_{wo}V + p_{wo}$.
3. **Camera pose.** The robot pose gives T_{wb}, and the camera is placed on the body by the extrinsic T_{bc} read from `config/wil_sim/stereo_imu.yaml`, so $T_{wc} = T_{wb}T_{bc}$. Reading the extrinsic from the estimator's own configuration file is deliberate: the evaluation and VINS-Fusion then cannot disagree about where the camera is.

4. **World to camera**, by inverting that transform, using the transpose because the rotation is orthonormal: $R_{cw} = R_{wc}^{\top}$, $t_{cw} = -R_{cw}t_{wc}$, $v_c = R_{cw}v_w + t_{cw}$.
5. **Projection.** Vertices with $z \leq 0.05\,\mathrm{m}$ are discarded *before* the division, then the remainder are projected through an ideal pinhole:

$$u = f_x \frac{x_c}{z_c} + c_x, \qquad v = f_y \frac{y_c}{z_c} + c_y. \tag{3.10}$$

6. **Box.** The axis-aligned extent of the projected points, $[\min u, \max u] \times [\min v, \max v]$, clipped to the image.

The z guard in step 5 is not a formality. A vertex behind the camera has negative z, and the division then mirrors it to the wrong side of the image, producing a well-formed box in entirely the wrong place — a failure that looks like a detector error rather than a projection error. Objects that straddle the image plane are projected from their visible vertices only.

Each object also stores $\tilde{z} = \mathrm{median}(z_c)$, which is what the occlusion test below uses to decide which of two overlapping objects is nearer.

Table 3.25 Parameters of the detection evaluation. The intrinsics are not calibrated: Gazebo renders an ideal pinhole, so $f_x = f_y = (W/2)/\tan(\phi/2)$ with every distortion term zero, and the values below are exact rather than estimated. The three exemption rules decide which ground-truth objects are scored at all, and are the reason the recall in section 4.2.1 is computed over a subset of the objects present.

Parameter	Value	Role
f_x, f_y	640, 640	ideal pinhole, 90° horizontal field of view
c_x, c_y	640, 360	principal point at image centre
Image size	1280×720	
Distortion	all zero	the renderer applies none
Mesh vertices per prop	≤ 4000	decimated with a fixed seed
Near-plane guard	0.05 m	vertices nearer than this are dropped before projection
Confidence, `-conf`	0.35	detections below this are never emitted
NMS IoU, `-iou-nms`	0.5	
Matching IoU, `-iou-match`	0.5	prediction counts as a hit above this
Motion threshold, `-move-thresh`	$0.05\,\mathrm{m\,s^{-1}}$	object is "moving" above this, from its pose history
Exemption: minimum size, `-min-box-px`	12 px	smaller ground-truth boxes are discarded and never appear
Exemption: occlusion, `-occlusion`	0.70	ground truth covered more than this by nearer objects is scored as neither hit nor miss (*yellow* in figure 4.4)
Exemption: scenery cover, `-ignore-cover`	0.50	a prediction this covered by unlabelled scenery is neither credited nor penalised

What this convention costs Two consequences follow from scoring against projected geometry, and both bear on how the detector numbers in section 4.2.1 should be read.

The boxes are *tighter than a human would draw*. They are the exact silhouette of the collision mesh, whereas an annotator draws a looser rectangle and rounds generously. A

detector trained on human boxes is therefore scored against a stricter convention than it was fitted to, and loses IoU that represents no error on its part. Of the differences between how this detector is scored here and how it was scored during training, this is the one that cannot be removed without relabelling.

The exemptions are *not symmetric with the training convention*. A human-annotated set has no notion of an exempt object: every labelled box counts. The occlusion and minimum-size rules exist because a crate that is nine tenths behind a shelf is not a fair test of a detector, which is the right decision for reporting detector quality — but it means recall here is computed over the easier subset. The size of that effect is measurable from the stored counts: 34 476 of the 96 758 ground-truth objects across the five recordings are exempt, and recomputing recall with every one of them counted as a miss moves it from 0.593 to 0.382 (`script/yolo_generalization.py`). Any comparison against a training-set recall has to use the second figure, not the first.

3.6.3 Experiment 3: Visual-inertial odometry with dynamic-environment filtering

Objective. Determine whether masking dynamic-object features improves localization, measure the compute cost of doing so, and verify that tracking survives the filtering and that the added detector cost does not push either pipeline below the input frame rate.

Process, simulation. Each visual pipeline is run on each recording in two treatments, baseline and filtered, as separate processes on identical input, three repeats per condition. The treatment pair is valid only when the filter demonstrably acted (matched detections, non-zero mask coverage, removed tracks) and the two runs received the same input to within 5 % in image count and duration. The collection protocol is given in section 3.7.3.

Process, real robot (planned). The online procedure has four steps. *Step 1, initialisation and time synchronisation:* power the NRU-51V, the stereo cameras and the IMU; start the motion-capture system and calibrate the room; synchronise the ROS 2 clock on the robot with the motion-capture server by PTP or Chrony (NTP), since a fraction of a second of offset corrupts ATE and RPE. *Step 2, execution and environment alteration:* drive from the origin in Zone A to the goal in Zone B at a linear speed not above $1.5\,\mathrm{m\,s^{-1}}$ through the Nav2 package; on arrival, trigger the dynamic variable by randomly re-placing the k stacks in Zone A; return, alter Zone B, and alternate for five minutes per test cycle to represent a lifelong-mapping load. *Step 3, data logging:* record with rosbag2 the 6-DoF SLAM pose, the motion-capture reference, the detector boxes with confidences, and the whole-pipeline latency per frame. *Step 4, iteration over the experimental matrix:* repeat steps 1–3 for both pipelines under the three illumination levels. The `.db3` bags are then trajectory-extracted and timestamp-matched before evaluation.

Evaluation metrics. ATE and rotational RMSE after rigid alignment, RPE at $\Delta = 1\,\mathrm{s}$ (all pairs and jump-free), tracking-loss and reset events, retained input fraction, throughput, process CPU and peak memory, and per-module latency against the inter-frame budget.

Validity rule. A trajectory containing steps that imply more than $10\,\mathrm{m\,s^{-1}}$ is not read as ordinary drift; an indoor error above 50 m is divergence. A comparison across treatments is admissible only for input-matched pairs with an active filter.

3.6.4 Experiment 4: Dense map reconstruction and map-quality evaluation

Objective. Verify whether the reconstructed map accurately reflects the physical environment and is usable by a route planner.

Process. RTAB-Map builds a database from a mapping run; the occupancy grid is seated in the world by a fixed transform measured from the map alone; geometry is scored against the world's collision meshes; drivability is scored by stamping the true robot footprint along a ground-truth route that did not build the map.

Evaluation metrics. Occupied-cell precision, recall over observed surfaces and over the whole building, false-obstacle rate under the footprint, unmapped footprint poses, clearance statistics, and the graph's node and link counts.

Validity rule. Placement must be established independently before any metric is read, and a drivability figure computed against the run that built the map is a self-consistency check, not a result.

RTAB-Map grids are placed in the Gazebo world using one fixed rigid world-to-map transform measured by `script/fit_map_pose.py`. It is constant for a database; the live `map` $\rightarrow$ `odom` drift correction is not reused for map placement.

`script/validate_map.py` rasterizes world collision geometry and evaluates occupied cells at a stated tolerance. It reports occupied-cell precision, whole-world and observed-region recall, false obstacles under the true 0.83×0.50 m footprint, unmapped footprint poses, and clearance. Graph evaluation records neighbour, global-closure, local-space, and gravity links. Localization additionally requires time to first fix, accepted and rejected matches, database checksums before and after read-only operation, pose continuity, and known-world bounds. Finite covariance or a localization status message alone does not establish correctness.

At the cutoff this evaluation has been applied to one database, `map/map_tiny.db`, against `small_warehouse_static/small_warehouse_static_tinymap.world`. The equivalent large-warehouse evaluation is **planned** and no database for it exists; results are reported in section 4.4.1.

3.7 Evaluation Metrics and Analysis

3.7.1 Trajectory evaluation

`script/vio_metrics.py` associates estimate and reference over their common time interval and interpolates the reference at estimator timestamps. A rigid Umeyama $SE(3)$ fit maps estimated positions to the reference with scale fixed at one. Fixed scale is appropriate for stereo-inertial estimators and preserves scale error. Outputs include translational ATE RMSE, maximum and final error, rotational geodesic error, path length, overlap duration, pose count, jump count, and longest jump-free segment. Optional startup trimming is relative to each estimator's first overlapping pose, preventing unequal initialization latency from being interpreted as steady-state behaviour.

The older `plot_vio_vs_gt.py` provides a first-pose-aligned diagnostic, but headline comparison uses common $SE(3)$ alignment. Real data without independent ground truth receives no ATE.

Relative pose error. RPE was added to `script/vio_metrics.py` after the reporting cutoff, on 18 September 2026, and is **implemented and exercised on the full campaign**. It follows the conventional definition: for a pair of poses separated by a fixed interval Δ, the error transform is

$$E_i = \left(Q_i^{-1} Q_{i+k}\right)^{-1} \left(P_i^{-1} P_{i+k}\right), \tag{3.11}$$

with P the estimate and Q the reference. Translational RPE is $\|\mathrm{trans}(E_i)\|$ and rotational RPE is the geodesic angle of $\mathrm{rot}(E_i)$, the same angle measure used for orientation error above.

RPE is reported because it needs *no* global alignment, and that is what makes it the appropriate metric for the dynamic runs. A single discontinuity re-seats the rigid $SE(3)$ fit, so every subsequent pose inherits the offset and ATE describes the jump rather than the tracking. RPE instead asks a local question — over the next Δ seconds, did the estimate move as the reference moved — and is insensitive to both the global frame and accumulated drift. It therefore supersedes the longest-jump-free-segment statistic, which answers a similar question through a segment-selection heuristic rather than a standard definition.

Four intervals are swept, $\Delta \in \{0.5, 1, 2, 5\}$ s, giving an error-versus-interval curve; $\Delta = 1$ s is the headline value and corresponds to roughly 1.4 m at the simulated robot's cruising speed. A pair is admitted only when its realised separation lies within 25 % of Δ, so a gap in the log cannot silently widen the interval and inflate the error; the surviving pair count is reported with every statistic. Pairs are overlapping, which is the conventional default.

Each statistic is computed twice: over all admissible pairs, and over only those pairs that do not span a detected tracking jump. The difference isolates the teleport contribution from ordinary local error, mirroring the existing separation between global and clean-segment ATE. Both RMSE and median are reported, because the dynamic-run distributions are strongly right-skewed and a single teleport otherwise sets the RMSE.

The implementation is verified against synthetic trajectories with known answers: an estimate identical to the reference returns zero; a constant relative drift of $0.1\ \mathrm{m\,s^{-1}}$ returns 0.100 m at $\Delta = 1$ s and 0.200 m at $\Delta = 2$ s; a $2\ ^\circ\mathrm{s^{-1}}$ relative rotation returns 2.00° at $\Delta = 1$ s; and an injected teleport separates the all-pairs and jump-free statistics by the expected pair count. Results are written to `rpe_metrics.csv` for the interval sweep, with the $\Delta = 1$ s values carried into `trajectory_metrics.csv`.

3.7.2 Computational performance

Both estimator processes write per-frame `performance.csv` files. These record processing time, delivery and backlog counters, tracking or solver state, initialization and reset evidence, pose and keyframe accounting, cumulative process CPU time, monotonic wall time, and resident memory. VINS additionally records feature-front-end and estimator-back-end stages; ORB-SLAM3 records its tracking front end, local mapping, place recognition, loop closure, map merging, and global bundle-adjustment events. CPU utilization and throughput can therefore be derived over explicit wall-time intervals. The measurements are process-local and exclude Gazebo, rosbag, RViz, RTAB-Map, and the separate YOLO process. Synchronized whole-system monitoring and the repeated Jetson/laptop matrix remain **planned**. Module timings from asynchronous stages are reported as separate distributions and are not added as if they formed one serial critical path.

3.7.3 Collection protocol, validity and stored artifacts

The sensor source, estimator, optional detector, and viewer start as separate processes; launch files do not automatically play bags. Baseline and filtered treatments run separately on the same recording, at a replay rate that permits comparable input consumption. Competing estimators are not run simultaneously in a scientific A/B comparison.

The common trajectory file has no header and uses:

```
t_ns, px, py, pz, qw, qx, qy, qz, vx, vy, vz
```

Timestamps are integer nanoseconds from the source header and quaternion order is w,x,y,z. ORB-SLAM3 writes this representation directly; VINS writes the state after nonlinear optimization. Simulation truth is extracted to the same shape by `script/extract_gt.py`. The tracker-use record `yolo_mask.csv` remains separate from the offline detector score.

Table 3.26 Principal stored artifacts.

Artifact	Purpose
`<run>/vio.csv`	Estimator trajectory and velocity
`<run>/ground_truth.csv`	Simulation reference trajectory, when available
`<run>/yolo_mask.csv`	Actual mask matching, coverage, and rejection
`<run>/run_metadata.csv`	Resolved configuration and supplied provenance
`<run>/performance.csv`	Per-frame timing, state, delivery, CPU, and memory
`<run>/frontend.csv` and `<run>/backend.csv`	VINS feature-tracking, estimator-stage, solver, and state-window diagnostics
`<run>/tracking_frontend.csv`	ORB-SLAM3 tracking-stage timing and feature counts
`<run>/local_mapping.csv`	ORB-SLAM3 keyframe insertion, map-point, and local-BA diagnostics
`<run>/loop_closing.csv`	Place-recognition, loop, merge, and global-correction events
`<run>/run_summary.csv`	Graceful-shutdown validity and robustness totals
`<run>/events.csv`	VINS initialization, reset-cause, and shutdown events
`output/yolo_eval/<experiment>/yolo_eval.csv`	Per-frame detector evaluation
`output/yolo_eval/<experiment>/yolo_detections.csv`	Per-detection matching
`output/yolo_eval/<experiment>/overlay/`	Projection and matching validation
`map/*.db` or `<run>/rtabmap.db`	Persistent RTAB-Map graph and signatures
`output/compare/<domain>/<experiment>/`	Derived figures and summaries

The current analysis code requires at least ten overlapping samples. Scientific acceptance additionally requires the same input interval, successful initialization, comparable pose and frame counts, and no unexplained sensor starvation. Steps implying over $10\,\mathrm{m\,s^{-1}}$ are tracking jumps; an indoor estimate with error beyond 50 m is divergence rather than

ordinary inaccuracy. Without real-data ground truth, path length and mean speed are plausibility diagnostics, not accuracy measurements.

The estimator now records resolved local configuration and optional experiment, dataset, world, replay-rate, command, and note fields in `run_metadata.csv`. It does not automatically determine bag health, repository revisions, complete dependency versions, external-process state, or the analyst's exclusion decision. Empty optional fields therefore remain incomplete provenance. Historical folders created before this instrumentation remain preliminary unless their provenance is recoverable from logs and engineering records.

3.7.4 Reproducibility controls

The following controls govern future experiments and acceptance of historical results:

1. Record absolute bag and exact `.world` file, treatment, configuration, commands, clock and replay rate, hardware, dependencies, and relevant repository commits.
2. Use a unique output directory and retain failed runs with failure labels.
3. Run paired treatments independently on the same sensor data and compare only their common valid interval.
4. Record inputs, outputs, initialization, dropped frames, resets, tracking loss, warnings, and exclusion reasons.
5. Validate camera order, transport, timestamps, metric scale, and every rigid transform before analysis.
6. Inspect representative detector and map overlays before aggregate scores.
7. Repeat nondeterministic RTAB-Map processing and report valid-run count; do not interpret differences within run-to-run variation as algorithmic.
8. Keep estimator, back end, filtering, environment dynamics, floor texture, domain, and hardware separate unless their interaction is the declared factor.
9. Record every switch that changes estimator behaviour in the run manifest, not only those the launch file exposes. Loop-closing and global-bundle-adjustment state are currently absent from `run_metadata.csv`, so two ORB-SLAM3 runs of the same dataset in opposite configurations produce manifests that differ only in identifier and timestamp.
10. Repeat every configuration of a nondeterministic estimator at least three times and report median and spread. ORB-SLAM3 is multi-threaded and its run-to-run variation on identical input is presently unmeasured, which places a floor on the differences that can be attributed to any factor.

3.8 Planned Versus Implemented

Material deviations from the originally proposed method are given in table 3.27. They are differences between the proposal and the system that exists, not failures: each is recorded so that no proposal capability is read into the implemented pipeline.

These deviations define the actual system at the reporting cutoff. Chapter 4 must report only measurements satisfying the validity rules above, while failed, excluded, and retracted runs remain in a clearly labelled diagnostic record.

The principal evidence comprises `README.md`; the four `MEMORY_CLAUDE/README.md` engineering records; the ORB-SLAM3 integration record; `docs/report/project_`

Table 3.27 Planned-versus-implemented deviations at the reporting cutoff. "Added" marks work not present in the original proposal.

Component	Proposed	Implemented	Status
Semantic segmentation	pixel masks, as in the reference RY-SLAM approach	three-class bounding boxes, no motion gate	deviation
Dynamic rejection point	not specified	ORB-SLAM3 rejects before octree distribution; VINS-Fusion also removes persistent KLT tracks	implemented
Estimator coupling	shared output representation	two independent estimators, not one fused estimator	deviation
RTAB-Map correction	integrated back end	separate process consuming external TF odometry; correction never injected into estimator state	deviation
Real-robot evaluation	metric accuracy against ground truth	calibration-limited, no independent trajectory truth	blocked
Metric infrastructure	resource, throughput, uncertainty, manifests	partially implemented; RPE now complete	incomplete
Relocalization and merging	robust arbitrary-start localization, aliasing rejection, extend-mode validation, front-end global optimization, multi-session merging	partially verified relocalization only	planned
Non-visual baseline	not proposed	wheel, IMU and EKF estimators with a data-derived noise model (section 3.6.1)	added
Covariance consistency	not proposed	NEES and σ-coverage testing for the non-visual estimators	added

Table 3.28 Original objectives (section 1.2) against the work reported here.

Objective	Status at the cutoff
1. Visual-inertial odometry with lifelong mapping	**partial**: two VIO front ends implemented; RTAB-Map mapping and read-only localization implemented; multi-session merging unverified (table 3.27)
2. Low-light image enhancement with EnlightenGAN	**not pursued**: no implementation in the repository
3. Dynamic-object perception by deep-learning detection	**implemented**: three-class YOLO26 detector, evaluated offline (section 4.2)
4. Lane detection with PolyNET and PID lane tracking	**not pursued**: no implementation in the repository
5. Integrated system with a 2-D cost map and localization error below 10 cm under differing illumination	**planned**: the integrated live test of section 3.6.3 has not been run; illumination has not been varied in any recording

limitations_and_map_benefits.md; the source, launch, and configuration files cited in this chapter; and script/vio_metrics.py, script/extract_gt.py, script/yolo_eval.py, script/fit_map_pose.py, and script/validate_map.py. These project artifacts establish local implementation. They do not replace primary literature citations for the scientific origin of the algorithms.

CHAPTER 4 EXPERIMENTS AND RESULTS

Reporting cutoff: 25 September 2026. Table 4.1 gives the status of the eight result slots. Four simulation slots hold data; the four real-robot slots are planned, for the reason stated in each. Each filled slot reports what was measured and its validity limits. Passages whose evidence is not yet trustworthy, and the status note of each slot, are collected in the separate Open Concerns document (`docs/report/concerns/concerns.pdf`), generated from this chapter's source.

Table 4.1 Status of the eight result slots at the reporting cutoff.

Experiment	Simulation	Real robot
1. Non-visual baseline and estimator comparison	**completed**: five `allsensor` recordings across four worlds	**planned**
2. Offline YOLO detector performance	**completed**: four dynamic recordings, 9 860 frames	**planned**
3. VIO with dynamic-environment filtering	**untested**: 66 runs exist; the hypothesis has no valid test	**blocked**
4. Dense map reconstruction and map quality	**partial**: tiny map scored; large map **failed**	**planned**

4.1 Experiment 1: Non-Visual Baseline and Estimator Comparison

4.1.1 Simulation

Objective. Establish whether a non-visual estimator provides usable localization on the same recordings the visual pipelines consumed, and whether each estimator's reported uncertainty describes its error (section 3.6.1).

Process. `script/proprio_estimator.py calibrate` measured the sensor noise of each bag against ground truth; `run` produced the wheel, IMU and EKF trajectories offline; `script/analyse_proprio.py` scored them together with the live VINS-Fusion and ORB-SLAM3 runs over the common interval.

Dataset. The five `allsensor` recordings of table 3.19, whose scenes, routes and floor treatments are drawn in figure 3.10 and tabulated in table 3.15. They are the only recordings that carry wheel encoders.

Question, hypotheses, and conditions

Until this experiment, every trajectory figure in this chapter reported visual error against ground truth with nothing to compare it to, so a reader could see how large the error was but not whether it was good. This section establishes the dead-reckoning floor. The question is whether a non-visual estimator—wheel odometry, IMU dead reckoning, or an EKF fusing them—provides usable localization on the same recorded data, and whether the uncertainty each one reports describes the error it actually makes.

Four hypotheses were registered before the metrics were computed:

H1 Wheel-odometry drift is dominated by the measured yaw-rate scale error, so the two bags, whose scales err $+1.1\%$ and -4.7%, should bow in opposite rotational directions.
H2 IMU-only dead reckoning diverges by orders of magnitude more than wheel odometry over the same interval.
H3 The EKF improves on both single-sensor estimators.
H4 All three report overconfident covariances, because a dead-reckoning noise model omits systematic scale error.

The independent variable is the estimator; everything else is held fixed within a dataset. Table 4.2 records the controlled conditions. All five estimators consume the same recordings and all visual runs replayed them under identical settings.

The five datasets are not repeats of one condition. Only `allsensor_000` and `allsensor_001` share a world. The remaining three vary two factors: object placement, where `objonwallonly` restricts obstacles to the wall region and clears the central floor, and floor texture, where `bayline` adds painted bay lines to a surface that is otherwise featureless. No quantity is pooled across worlds; each dataset is read as its own condition.

Table 4.2 Conditions for the estimator comparison. All five estimators consume the same five recordings and are scored against the same ground truth over the same interval. One factor listed here is *not* controlled and is marked as such: the IMU noise model differs systematically between the proprioceptive and visual families, and section 4.1.1 measures what that is worth.

Factor	Level
Worlds	four: `..._static_01_nofloortexture` (000, 001), `..._static_nofloortexture_objonwallonly` (002), `..._static_bayline_objonwallonly` (003), `..._static_01_bayline` (004)
Floor	plain (000–002) versus bayline, i.e. painted lines (003, 004)
Scene dynamics	static
Floor	featureless
Datasets	five, 81.8–102.4 s of simulated time and 113.1–142.3 m driven each
Dynamic filter	disabled in every run (`filter=false`)
Replay rate	$0.5\times$ for both visual estimators
Ground truth	`/ground_truth/odometry`, 50 Hz, header stamps
Alignment	rigid $SE(3)$, scale fixed, for cross-system comparison
IMU noise model	not controlled in this experiment. The EKF's is measured per bag by `proprio_estimator.py calibrate`; both visual estimators read four constants from `config/wil_sim/stereo_imu.yaml`. The factor is confounded with the estimator here and is separated in section 4.1.1
Experimental unit, n	one bag; $n = 5$ across *four* worlds. Only `static_01` plain is repeated, and only twice
Visual configurations	VINS-Fusion; ORB-SLAM3 with loop closing enabled and disabled, one run each

Validity

Table 4.4 rests on thirty-five runs: three proprioceptive baselines and the transferred-noise EKF of section 4.1.1 on each of the five bags, VINS-Fusion once per bag, and ORB-

SLAM3 twice per bag for the two loop-closure configurations. All thirty-five passed their predefined checks.

The proprioceptive baselines completed on all five bags with the planar assumption holding by a wide margin—maximum tilt 0.0015–0.0018° against a 1° threshold—with the bicycle yaw-rate model selected automatically on every bag, `imu_orientation_used=NO` recorded in every run, and the EKF applying 4 355, 4 091, 4 338, 5 120 and 5 026 wheel corrections, matching the wheel-odometry message counts exactly. VINS-Fusion processed every image it received on all five.

The initialization and reset figures below were extracted for `allsensor_000` and `allsensor_001` only and have not been recovered for the other three bags: VINS-Fusion initialized once in 0.33 s with no resets, and both ORB-SLAM3 runs shut down gracefully with no tracking-loss events and inertial initialization at 2.48 s and 2.51 s. Neither `run_metadata.csv` schema records initialization time, reset count or tracking-loss events, so these cannot be recovered from the run record and would have to be re-read from the console logs.

Table 4.3 Loop-closing activity in the enabled ORB-SLAM3 configuration (`logging_20260916_01`). Accepted closures are a small and dataset-dependent fraction of candidates, and two datasets accept none at all despite the closer running throughout. In the disabled configuration (`logging_20260916_02`) no keyframe ever reaches the loop closer on any dataset.

Dataset	Place-recognition events	Candidates	Accepted	Global BA
`allsensor_000`	537	90	0	0
`allsensor_001`	379	2	2	2
`allsensor_002`	274	11	1	1
`allsensor_003`	591	64	1	0
`allsensor_004`	713	114	0	0

Accuracy results

Table 4.4 reports all five estimators. Aligned ATE is the only column comparable across systems, because the visual estimators build their own world frames while the baselines start from the ground-truth pose.

H2 and H3 are supported on every dataset. IMU-only dead reckoning is 7.7 to 14.3 times worse than wheel odometry, and the EKF improves on both single sensors by an order of magnitude everywhere.

H3's stronger form—that the EKF also beats the visual estimators—holds on four of the five datasets *as configured*, but it does not survive a control on tuning, and that control is now the governing result of this section (section 4.1.1). Taken at face value the margin is also much narrower than the three-dataset version of this section reported. Measured against the best visual estimator of each dataset, and using the cross-validated EKF (table 4.8), the margins are 1.38, 1.41, 2.53, 1.31 and 1.13. On `allsensor_004` the in-sample EKF at 0.494 m is beaten outright by VINS-Fusion at 0.425 m; only the cross-validated figure of 0.377 m is ahead. Every one of those margins is smaller than the effect of the noise model itself, so none of them should be read as an estimator ranking. The claim this section can defend is narrower than either version: a five-state proprioceptive filter *whose noise model is calibrated per recording* localises more accurately than stereo visual-inertial SLAM

Table 4.4 Estimator comparison on the five `allsensor` bags, which span four worlds and both floor conditions; they are not repeats of one another except for `allsensor_000` and `allsensor_001`. Bold marks the best aligned ATE of each dataset *as each system was actually configured*—the EKF on four of five, VINS-Fusion on `allsensor_004`. That ranking is not tuning-controlled: the EKF's noise model is measured from each bag, while both visual systems read the hand-set `config/wil_sim/stereo_imu.yaml`. The *EKF @ VSLAM noise* rows are the same filter on the same bags with only its four IMU noise terms replaced by that file's values; it then loses to VINS-Fusion on all five bags (section 4.1.1). "Aligned ATE" is RMSE after a rigid $SE(3)$ fit with scale fixed, the convention used throughout this chapter. "Anchored" columns are the direct difference from ground truth with no alignment and are defined only for the estimators that start from the ground-truth pose; the visual estimators' own world frames make that subtraction meaningless, so those cells are set to –. Drift is the final position error as a fraction of distance driven.

Dataset	World	Estimator	Aligned ATE [m]	Anchored RMS [m]	Anchored final [m]	Drift [%]	Final yaw [°]
`allsensor_000`	`static_01` plain	IMU dead reckoning	21.697	39.353	47.597	40.66	+2.19
		Wheel odometry	2.801	4.732	4.767	4.07	−13.76
		VINS-Fusion	0.616	–	–	–	–
		ORB-SLAM3	0.579	–	–	–	–
		ORB-SLAM3 (no LC)	0.285	–	–	–	–
		EKF wheel+IMU	**0.190**	0.281	0.122	**0.10**	+0.19
		EKF @ VSLAM noise	1.313	1.980	1.443	1.23	−11.27
`allsensor_001`	`static_01` plain	IMU dead reckoning	12.617	14.728	23.741	20.99	+1.33
		Wheel odometry	0.979	1.826	2.426	2.15	−12.95
		VINS-Fusion	0.383	–	–	–	–
		ORB-SLAM3	0.372	–	–	–	–
		ORB-SLAM3 (no LC)	0.516	–	–	–	–
		EKF wheel+IMU	**0.308**	0.351	0.303	**0.27**	+0.11
		EKF @ VSLAM noise	0.681	0.939	1.027	0.91	−9.49
`allsensor_002`	`objonwallonly` plain	IMU dead reckoning	13.237	16.359	17.728	15.39	+1.36
		Wheel odometry	1.485	2.888	4.332	3.76	−30.11
		VINS-Fusion	0.558	–	–	–	–
		ORB-SLAM3	0.602	–	–	–	–
		ORB-SLAM3 (no LC)	1.796	–	–	–	–
		EKF wheel+IMU	**0.256**	0.345	0.346	**0.30**	−0.68
		EKF @ VSLAM noise	0.868	1.320	1.802	1.56	−20.71
`allsensor_003`	`bayline_objonwallonly` bayline	IMU dead reckoning	45.562	62.903	74.745	53.16	−5.84
		Wheel odometry	3.192	3.979	9.231	6.57	−37.17
		VINS-Fusion	0.784	–	–	–	–
		ORB-SLAM3	6.481	–	–	–	–
		ORB-SLAM3 (no LC)	0.515	–	–	–	–
		EKF wheel+IMU	**0.423**	0.668	1.240	**0.88**	−7.20
		EKF @ VSLAM noise	1.230	1.574	3.037	2.16	−24.76
`allsensor_004`	`static_01_bayline` bayline	IMU dead reckoning	26.709	41.476	48.099	33.81	−1.35
		Wheel odometry	3.241	5.124	5.791	4.07	−17.68
		VINS-Fusion	**0.425**	–	–	–	–
		ORB-SLAM3	1.237	–	–	–	–
		ORB-SLAM3 (no LC)	0.905	–	–	–	–
		EKF wheel+IMU	0.494	0.776	0.831	0.58	−5.30
		EKF @ VSLAM noise	1.583	2.650	1.836	1.29	−15.16

running on a hand-set configuration. The calibration is doing at least as much work as the architecture.

Two expectations about scene content are not supported. Clearing the central floor did not penalise vision: the visual errors on `allsensor_002` (0.558 and 0.602 m) sit close to `allsensor_000` (0.616 and 0.579 m). Nor did adding the bay markings help it—the two `bayline` datasets give VINS-Fusion 0.784 and 0.425 m against 0.616, 0.383 and 0.558 m on the plain floors, which is no improvement. That null result is weaker evidence than the factor's name implies: both floor conditions rest on the same flat, untextured base and differ only by fourteen painted bay outlines covering a small fraction of the floor (figure 3.9), so what was varied is bay markings, not surface texture. What the textured datasets did change is the *proprioceptive* side: both carry the largest wheel yaw-rate scale errors of the five, and both degrade the EKF accordingly (section 4.1.1).

The mechanism is visible in the last two columns and is the substantive finding of this section: *the two proprioceptive sensors fail in complementary ways*. The IMU holds heading to 2.19° and 1.33° while losing position catastrophically; the wheels hold position to a few percent while losing 13° of heading. The EKF is better than either in *both* quantities simultaneously, reaching 0.19° of final heading error—an order of magnitude better than the gyroscope alone—because the wheel yaw rate is used to observe the gyroscope bias rather than heading, so the filter never inherits the wheels' own heading drift (section 3.6.1).

Falsification of H1, and the drift mechanism

H1 is falsified across all five datasets. If drift followed the fitted yaw-rate scale, the bag with the most nearly correct scale would drift least. The opposite holds (table 4.5): `allsensor_002` has the best scale of the five, +0.34%, and −30.11° of heading error, while `allsensor_004` has the worst scale, −7.50%, and only −17.68°. All five heading errors share a sign; the scale errors do not.

Table 4.5 Wheel-odometry heading drift against the fitted yaw-rate scale. "Rotation" is the total unsigned ground-truth rotation of the run, and "error/rotation" expresses the final heading error as a fraction of it. Scale error and heading error correspond in neither sign nor magnitude. The last column is the constant-offset prediction described below: the mean yaw-rate residual of the run multiplied by its duration, which matches the measured error to within 1.4° on every dataset, and to within 0.5° on three of five.

Dataset	Yaw-rate scale [%]	Final yaw error [°]	Rotation [°]	Error/rotation [%]	Offset prediction [°]
`allsensor_000`	+1.10	−13.76	536	−2.566	−13.81
`allsensor_001`	−4.73	−12.95	852	−1.519	−13.37
`allsensor_002`	+0.34	−30.11	865	−3.481	−31.03
`allsensor_003`	−6.25	−37.17	1123	−3.309	−38.51
`allsensor_004`	−7.50	−17.68	895	−1.976	−18.03

The proximate mechanism is nevertheless now identified, and it is simpler than a scale error. The wheel model's yaw rate carries a small persistent *offset* against ground truth, averaged over all moving samples: −0.00277, −0.00285 and −0.00624 rad s^{-1} on the three bags. Because heading is the time integral of that rate, an offset produces an error that grows linearly with elapsed time rather than with rotation. Multiplying each offset by its run duration reproduces the measured error to within 1.4° on all five datasets (table 4.5,

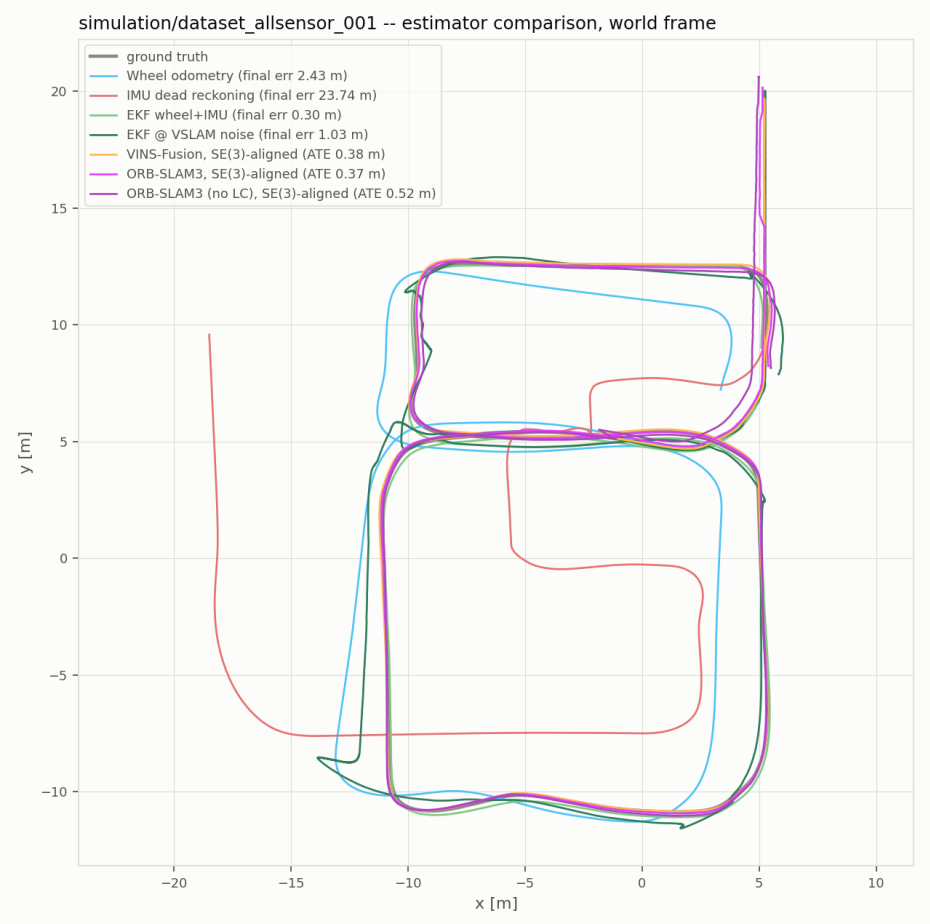


Figure 4.1 Estimator comparison on `dataset_allsensor_001`, world frame. The baselines are drawn as estimated; the visual estimators are drawn after the same rigid $SE(3)$ alignment used for their metrics, because their own world frames are arbitrary. Ground truth is hidden beneath the EKF, VINS and ORB traces for most of the run. Wheel odometry (blue) bows consistently outward; IMU dead reckoning (red) leaves the building.

last column), including the two textured-floor runs that were recorded after the mechanism was proposed. The drift is accounted for.

What remains open is the origin of the offset itself. It is not a pure magnitude error: splitting the residual by turn direction gives $+0.0109$ against -0.0142 on `allsensor_000` and $+0.0075$ against -0.0292 rad s^{-1} on `allsensor_002`, so the model under-rotates in both directions *and* carries a directional bias on top. On `allsensor_001`, whose route returns to its starting heading, both directions are negative and only the bias is visible. A steering-angle zero offset, an unmodelled slip angle, and a kingpin-geometry error are all consistent with what has been measured so far and none has been tested.

This matters beyond wheel odometry, and the two textured-floor datasets turn what was a caveat into a measurement. The EKF observes its gyroscope bias through exactly this wheel yaw rate (section 3.6.1), so a persistent offset in the wheel rate is absorbed into the estimated bias and reappears as heading error. Table 4.6 shows the two tracking each other across the five datasets: where the wheel yaw-rate scale is within 1.1 % of unity the EKF holds heading to 0.42° or better, and where it is 6 to 7.5 % low the EKF's heading error rises roughly tenfold.

Table 4.6 The EKF's heading error follows the quality of the wheel yaw rate it uses to observe its gyroscope bias. The filter is no better than its rate reference, which is the cost of making the bias observable at all.

Dataset	Floor	Wheel yaw scale	Wheel yaw σ [rad s^{-1}]	EKF yaw RMS [°]	EKF ATE [m]
`allsensor_000`	plain	1.0110	0.0659	0.35	0.190
`allsensor_002`	plain	1.0034	0.0705	0.30	0.256
`allsensor_001`	plain	0.9527	0.0804	0.42	0.308
`allsensor_003`	bayline	0.9375	0.1192	3.54	0.423
`allsensor_004`	bayline	0.9250	0.0613	3.41	0.494

An earlier version of this section, written when only the three plain-floor datasets existed, recorded that the EKF held heading below 0.7° everywhere and inferred that the absorbed error was small. That inference does not survive the textured-floor datasets and is withdrawn: the coupling is the dominant term in the EKF's heading error whenever the wheel rate is poor, and it sets a floor on the filter's accuracy that no amount of gyroscope quality can lift.

One prediction of the original reasoning did survive, and is visible in figure 4.2: wheel-odometry error stays flat at roughly 6 mm for the first 9.5 s, the initial straight run, and only grows once the first turn begins. Turning, not distance, starts the error; the offset then sustains it.

Covariance consistency

H4 is supported emphatically. Table 4.7 reports the average normalised estimation error squared, NEES $= \mathbf{e}^{\top}\mathbf{P}^{-1}\mathbf{e}$ with $\mathbf{e} = [\Delta x, \Delta y, \Delta\theta]$, which a correctly scaled covariance makes average 3.0, together with the fraction of the run for which the truth lies inside the reported $k\sigma$ bounds.

The IMU case is the starkest: its 3σ envelope contains the truth for between 2.1 % and 14.7 % of each run, and its worst ANEES, 1504 on `allsensor_003`, is the largest figure

Table 4.7 Covariance consistency of the non-visual estimators. A consistent filter gives ANEES = 3.0 and σ-coverages of 0.683, 0.954 and 0.997. Every estimator is overconfident by roughly two orders of magnitude. Neither VINS-Fusion nor ORB-SLAM3 writes a pose covariance, so neither can be tested here.

Dataset	Estimator	ANEES	1σ	2σ	3σ	Verdict
`allsensor_000`	Wheel odometry	569.4	0.309	0.390	0.408	overconfident
	IMU dead reckoning	311.0	0.005	0.022	0.043	overconfident
	EKF wheel+IMU	64.7	0.426	0.767	0.879	overconfident
`allsensor_001`	Wheel odometry	263.9	0.015	0.131	0.187	overconfident
	IMU dead reckoning	339.9	0.001	0.009	0.021	overconfident
	EKF wheel+IMU	123.2	0.194	0.439	0.537	overconfident
`allsensor_002`	Wheel odometry	729.4	0.140	0.143	0.145	overconfident
	IMU dead reckoning	244.7	0.017	0.045	0.147	overconfident
	EKF wheel+IMU	114.5	0.193	0.350	0.501	overconfident
`allsensor_003`	Wheel odometry	406.7	0.260	0.343	0.459	overconfident
	IMU dead reckoning	1504.3	0.010	0.018	0.035	overconfident
	EKF wheel+IMU	79.1	0.034	0.291	0.476	overconfident
`allsensor_004`	Wheel odometry	785.0	0.154	0.157	0.162	overconfident
	IMU dead reckoning	289.5	0.004	0.017	0.044	overconfident
	EKF wheel+IMU	78.6	0.254	0.330	0.570	overconfident
consistent reference		3.0	0.683	0.954	0.997	–

in the experiment. A consumer weighting this estimate by its stated covariance—a costmap, a pose-graph edge, a relocaliser gate—would trust it tens of times more than it deserves. Wheel odometry on `allsensor_002` is the single worst case in the experiment, ANEES 785 on `allsensor_004` with 3σ coverage of 0.162, which is consistent with the heading offset of section 4.1.1: the covariance model contains no term for a persistent rate bias, so the larger that bias, the further the claimed uncertainty falls behind the real error. The EKF is the least bad on every dataset, at 3σ coverage between 0.476 and 0.879, and figure 4.2 shows why: its NEES is genuinely near 3 during the opening straight and only departs once turning begins, which is the same moment the unmodelled heading error appears.

Cross-validation of the EKF noise model

The EKF's noise model is derived from the same recording it runs on, whereas the visual estimators receive no comparable tuning. The wheel and IMU baselines are unaffected—their measured σ values enter only the covariance, never the trajectory—but the EKF's values enter the Kalman gain and so do change its output. The comparison was therefore repeated with the noise models exchanged between bags (table 4.8).

The noise model transfers between runs, and on four of the five bags the borrowed model is the better one, which no in-sample advantage could produce. The EKF's accuracy is therefore a property of the estimator rather than of having seen the answer. Cross-validated it reaches 0.206, 0.263, 0.220, 0.392 and 0.377 m, which is 1.38, 1.41, 2.53, 1.31 and 1.13 times better than the best visual estimator of each dataset. Notably the cross-validated figure on `allsensor_004` recovers a lead that the in-sample filter did not have.

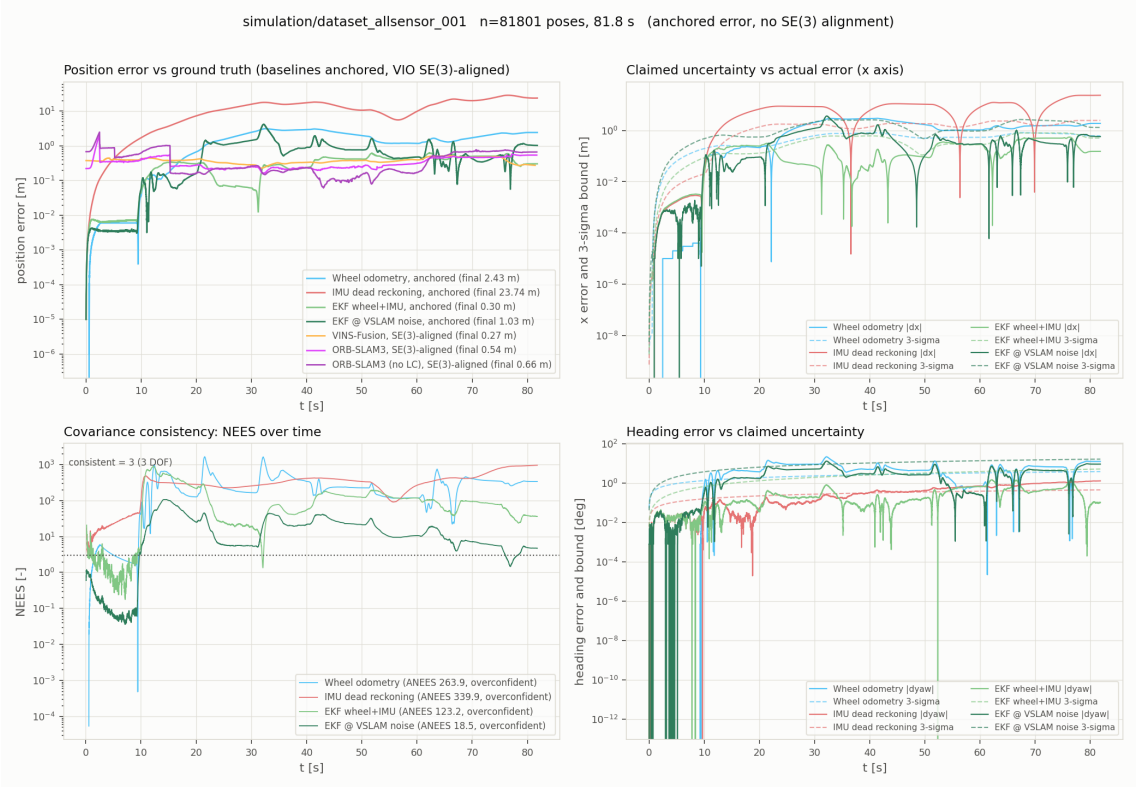


Figure 4.2 Error against claimed uncertainty for `dataset_allsensor_001`. Top left: position error over time, baselines anchored and visual estimators $SE(3)$-aligned. Top right and bottom right: absolute error against the estimator's own 3σ bound, in x and in heading; where the solid line sits above the dashed line of the same colour the filter is overconfident. Bottom left: NEES against the consistent value of 3. The step near 9.5 s in every panel is the first turn of the run.

Table 4.8 EKF accuracy with an in-sample and an out-of-sample noise model. The change is negative on four of five bags—the borrowed model is *better* there—and the mean change is about -10%, which is the signature of no systematic in-sample advantage.

Dataset	Noise model from	Aligned ATE [m]	Change
`allsensor_000`	same bag	0.190	–
	`allsensor_001`	0.206	+8.8%
`allsensor_001`	same bag	0.308	–
	`allsensor_000`	0.263	−14.5%
`allsensor_002`	same bag	0.256	–
	`allsensor_000`	0.220	−13.9%
`allsensor_003`	same bag	0.423	–
	`allsensor_000`	0.392	−7.1%
`allsensor_004`	same bag	0.494	–
	`allsensor_000`	0.377	−23.8%

Transferring the visual systems' noise model to the EKF

Question and hypothesis. Is the EKF's advantage in table 4.4 an estimator effect or a noise-model effect? Stated as a falsifiable hypothesis, **H4**: the EKF's advantage survives being given the visual systems' own IMU noise configuration. H4 is falsified if, on that configuration, the EKF's mean aligned ATE over the five bags is no better than VINS-Fusion's.

Conditions and controls. The asymmetry is measured against declared, not bag against bag. VINS-Fusion and ORB-SLAM3 both read `config/wil_sim/stereo_imu.yaml`, and every one of the fifteen visual runs in table 4.4 records that file in its metadata—the five VINS runs `logging_20260916_01`, and the ten ORB runs `_01` and `_02`—so a single hand-set configuration stands behind the whole visual column. The cheap half of the control is therefore to re-run the filter on that same file. It needs no new recording: the EKF reads the bags offline.

The independent variable is the four IMU noise terms, at three levels: the measured per-bag values, the values `stereo_imu.yaml` declares, and a set of one-term-at-a-time ablations between them. Held fixed across every condition are the five bags, the ground truth and its interval, the filter code and commit, the bicycle yaw-rate model, the 50 Hz wheel-update stride, and the alignment convention. The accelerometer and gyroscope biases and the whole wheel block stay at their measured values, so the IMU noise model is the only treatment; the single exception is the R-only ablation described below, which moves the wheel yaw-rate σ by construction. Converting the file's densities to the per-sample σ the filter stores ($\sigma = n\sqrt{f}$, $f = 200\,\text{Hz}$) gives a gyroscope term 5.5 to 6.7 times the measured value, an accelerometer term between 0.68 and 1.44 times it, and a bias random walk ten times it.

Validity and sample count. $n = 1$ run per bag per condition, thirty runs in all. The filter is deterministic given a bag and a noise model, so repeats would be bit-identical and no run-to-run uncertainty can be quoted; the spread reported below is across *bags*, not runs, and is given as a range rather than a standard deviation. All thirty completed, produced the same pose counts as their measured-noise counterparts (17 416, 16 364, 17 350, 20 477 and 20 104), applied the same number of wheel corrections, and were scored over the same matched interval against the same ground truth. The measured-noise condition is the published baseline and was not re-run. Re-scoring it through this experiment's own code reproduces table 4.4 to three decimals, which is the check that the two paths agree.

Results. Those are the *EKF @ VSLAM noise* rows of table 4.4.

The result reverses the section. Mean aligned ATE over the five bags rises from 0.334 m to 1.135 m, a degradation on every bag without exception. Against that figure VINS-Fusion's 0.553 m wins on *five of five*, ORB-SLAM3 without loop closing (0.804 m) on four of five, and ORB-SLAM3 with loop closing on four of five—its mean of 1.854 m is set by the `allsensor_003` outlier, and its median of 0.602 m is the fairer summary. On the metric this chapter uses throughout, run on the tuning the visual systems actually had, both visual systems are ahead of the filter.

Table 4.9 locates the cause. Moving the four terms one at a time shows that the gyroscope noise density accounts for essentially all of the damage; the accelerometer term is inert and the bias random walk contributes little. The density enters the filter twice—as heading

process noise in Q, and in the bias observation's R as $\sigma_w^2 + \sigma_g^2$. Raising the wheel yaw-rate σ so that R matches the transferred case exactly while Q stays measured reproduces the baseline, so the damage is entirely in Q. A gyroscope density 6.5× too loose inflates the heading process variance forty-fold, and because the measurement matrix touches only v and b_ω, heading can then be corrected only through the $P_{2,3}$ and $P_{2,4}$ cross terms—so the wheel yaw-rate residual begins steering heading instead of staying confined to the bias. Anchored yaw RMS rises from 1.60° to 8.67°.

Relative pose error confirms the attribution independently, and on a local metric that no global alignment can flatter. At $\Delta = 1$ s the transferred model degrades translation from 0.100–0.493 m and rotation from 0.285–2.262°; the gyroscope-only ablation reaches 0.485 m and 2.179°, almost all of it, while the accelerometer ablation is indistinguishable from the measured model to three decimals. The R-only case is marginally *better* than the measured model on all three metrics and on every bag, which says the measured wheel yaw-rate σ is itself slightly too tight and confirms that the R path is benign rather than merely null.

Table 4.9 Which of the four IMU noise terms costs the EKF its accuracy. Each row moves one term from its measured value to the one `config/wil_sim/stereo_imu.yaml` declares, averaged over the five bags. The last row isolates where the gyroscope term acts: it reproduces that term's effect on the measurement noise R exactly, by raising the wheel yaw-rate σ, while leaving the process noise Q at its measured value.

EKF, one term moved	Anchored RMS [m]	Aligned ATE [m]	Anchored yaw RMS [°]
measured throughout	0.484	0.334	1.60
gyroscope density, ×6.5	1.654	1.105	8.67
accelerometer density	0.479	0.329	1.60
gyroscope bias random walk, ×10	0.654	0.452	2.97
all four (*EKF @ VSLAM noise*)	1.693	1.135	8.88
gyroscope density through R only	0.475	0.327	1.50

Interpretation. Two findings survive the control and one does not. ORB-SLAM3's rotational error stays at 13.6 to 16.5° in every condition, still 2.4 times worse than the handicapped filter's 6.21°, so that result is a property of ORB-SLAM3 and not of tuning. The complementary-failure mechanism of the two proprioceptive sensors is likewise untouched, since it does not depend on which σ the filter was given. What does not survive is the ranking of the EKF against the visual estimators.

The consistency picture moves the other way, and is worth stating because it is the one place the transferred model does better. The EKF's ANEES falls from 64.7 to 123.2 down to 12.6 to 38.3—still overconfident by more than an order of magnitude, but far less so. A loose noise model buys a covariance closer to honest at the cost of the estimate itself, which is the textbook trade and is the first time this chapter has been able to show it directly.

Interpretation and limitations

Across these five recordings, a five-state planar filter over wheel encoders and a gyroscope localises more accurately than either stereo visual-inertial system *when its noise model is calibrated per bag and theirs is not*. Under the tuning control of section 4.1.1 that ordering

reverses on every bag, so the uncomfortable result for the thesis premise is weaker than it first appeared: what these bags demonstrate is that the noise model is worth more than the architecture over this range, not that dead reckoning outperforms vision. The practical reading is unchanged in one respect and sharpened in another. It still does not show that vision is unnecessary, and the justification for it still has to come from capabilities dead reckoning does not have at all—loop closure, relocalization after tracking loss, and the map itself—or from conditions these bags do not contain. It now also shows that the visual systems have never been tuned to this platform, which is the cheapest remaining source of improvement and is the first item of section 4.5.

Two further limitations are structural rather than queried. All three baselines begin from the ground-truth pose, because dead reckoning has no global reference; a deployed system would have to obtain that pose some other way, which is exactly the relocalization problem listed as future work. And the covariance result cannot yet be extended to the visual estimators, because neither writes a pose covariance—adding that to VINS-Fusion's marginalisation output is an instrumentation change, not an analysis one, and is the prerequisite for applying this consistency test to the pipelines the thesis is actually about.

Summary. IMU-only dead reckoning drifts by 12.6–45.6 m and wheel odometry by 1.0–3.2 m of aligned ATE; the EKF stays below 0.5 m on every recording and the visual estimators between 0.29 and 0.78 m in their best configuration. The EKF's lead over vision is a tuning effect, not an architectural one: with the visual systems' own inertial noise model it loses to VINS-Fusion on all five bags. The two proprioceptive sensors fail in complementary ways, and every dead-reckoning covariance is overconfident by roughly two orders of magnitude. Against the project's target of 0.05 m ATE, no estimator in this slot qualifies.

Future plan. Feed the VIO pose into the EKF as a third measurement, so that the filter takes distance from the wheels, heading from the gyroscope and drift correction from vision; and re-run in a realistic environment with wheel slip and encoder quantisation, since both idealisations of section 3.4.3 favour dead reckoning.

4.1.2 Real robot

Reason. Of the three real recordings (table 3.19) only `dataset_real_000` has inertial data, and none has the `/model/.../joint_state` and `odometry` topics the wheel and EKF estimators consume. The motion-capture reference of section 3.5.4 has not yet been recorded with the robot.

Next action. Bridge the wheel encoder and steering topics on the real platform, record the motion-capture trajectory alongside the sensors, and repeat the process of section 4.1.1 unchanged; the calibration pass will then also give the first measured noise model of the real IMU.

4.2 Experiment 2: Offline YOLO Detector Performance

4.2.1 Simulation

Objective. Measure detector localization and class assignment independently of either SLAM integration (section 3.6.2).

Process. `script/yolo_eval.py` projected the collision meshes of every prop into `cam0` to form truth boxes, ran the project checkpoint `weight/best.pt`, matched predictions greedily by class at IoU 0.5, and wrote per-frame and per-detection records; overlays were inspected before the aggregates were accepted.

Dataset. The four `dataset_dynamic_nofloortexture` recordings of table 3.19.

YOLO was evaluated independently of either SLAM integration on all four dynamic no-floor-texture rosbags. Ground-truth object poses came from `/world/default/pose/info`; because that topic's message headers are zero, its rosbag receive times were mapped to simulation time by interpolation against the recorded `/clock` stream. Known model meshes were transformed into the camera frame and projected into `cam0` to form two-dimensional boxes. Sampled overlays from every bag were inspected before calculating metrics and showed spatially plausible agreement between the projected boxes and visible objects.

Bounding-Box Detection and Localization

The evaluation used 9,860 camera frames, confidence threshold 0.35, NMS IoU threshold 0.50, and matching IoU threshold 0.50. Ground-truth boxes smaller than 12 pixels on either side were omitted; heavily occluded ground-truth boxes and predictions explained by explicitly unlabelled scenery were ignored rather than counted as errors. Table 4.10 reports per-dataset results and a micro-average formed by pooling TP, FP, and FN counts across all frames.

Table 4.10 Offline YOLO performance against projected Gazebo object ground truth. This is a detector-capability measurement, not an evaluation of delivery to the online SLAM filters.

Dataset	Frames	TP	FP	FN	Precision	Recall	F1	Moving recall
Dynamic 00/000	2,347	9,987	4,826	5,900	0.674	0.629	0.651	0.617
Dynamic 00/001	2,236	10,158	4,265	6,868	0.704	0.597	0.646	0.599
Dynamic 01/000	2,564	8,142	4,226	6,064	0.658	0.573	0.613	0.568
Dynamic 02/000	2,713	8,661	3,743	6,502	0.698	0.571	0.628	0.581
Pooled	9,860	36,948	17,060	25,334	0.684	0.593	0.635	0.591

The pooled evaluation contains 62,282 scored ground-truth boxes, of which 46,491 are moving, plus 34,476 projected boxes excluded as heavily occluded. The model produced 58,776 raw predictions. Mean image coverage was 10.42% for all scored ground truth, 7.88% for moving ground truth, and 9.07% for predictions. At the stated thresholds, the detector misses approximately 41% of scored moving objects; moving-object recall ranges from 0.568 to 0.617 between datasets. Conversely, the results demonstrate that the trained model can produce detections on every recorded bag. They do not contradict the zero-box VINS+YOLO logging anomaly: offline detector capability and online message delivery/timestamp matching are different mechanisms. Average precision is not reported because this evaluation retained predictions only above a single confidence threshold.

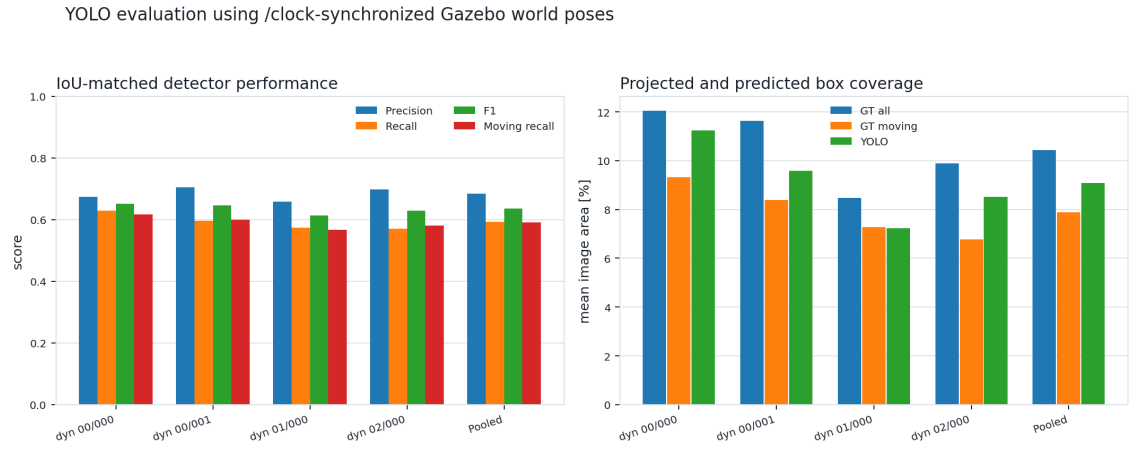


Figure 4.3 Per-dataset and pooled detector scores and box-area coverage. Pooled precision, recall, F1, and moving-object recall are calculated from pooled counts, rather than by averaging the four displayed dataset scores.



Figure 4.4 Representative validation overlay used before accepting the offline metrics. Green boxes are projected ground truth and red boxes are YOLO predictions. A **yellow** box is a ground-truth object that a nearer object covers by more than 70 %; it is excluded from scoring, so the detector is neither credited for finding it nor penalised for missing it. Only the green boxes are scored. An asterisk marks a ground-truth object classified as moving from its world-pose history.

Object-Class Assignment Conditional on Localization

This experiment is not whole-image classification: an image can contain many objects, and the model outputs a class for each predicted bounding box. To keep class assignment separate from localization, predictions and ground-truth boxes were greedily associated by IoU ≥ 0.50 without requiring their classes to agree. Classification was then evaluated only on the 37,061 localized object pairs. Missed ground-truth objects and unmatched predictions remain bounding-box detection errors in Section 4.2.1; they are not treated as an additional semantic class here.

Table 4.11 Object-class assignment conditional on successful IoU association, pooled across the four dynamic datasets.

Class	Associated objects	Precision	Recall	F1
Bin	2,639	1.000	1.000	1.000
Box	28,076	0.996	0.999	0.998
Bucket	6,346	0.996	0.983	0.989
Macro average	37,061	0.997	0.994	0.995

The conditional class accuracy is 0.996: 36,924 of 37,061 associated objects are on the confusion-matrix diagonal, leaving 137 class substitutions. The largest substitution is 110 buckets labelled as boxes. This high conditional accuracy does not cancel the weaker bounding-box recall: an object that was never localized cannot enter the conditional class-assignment evaluation.

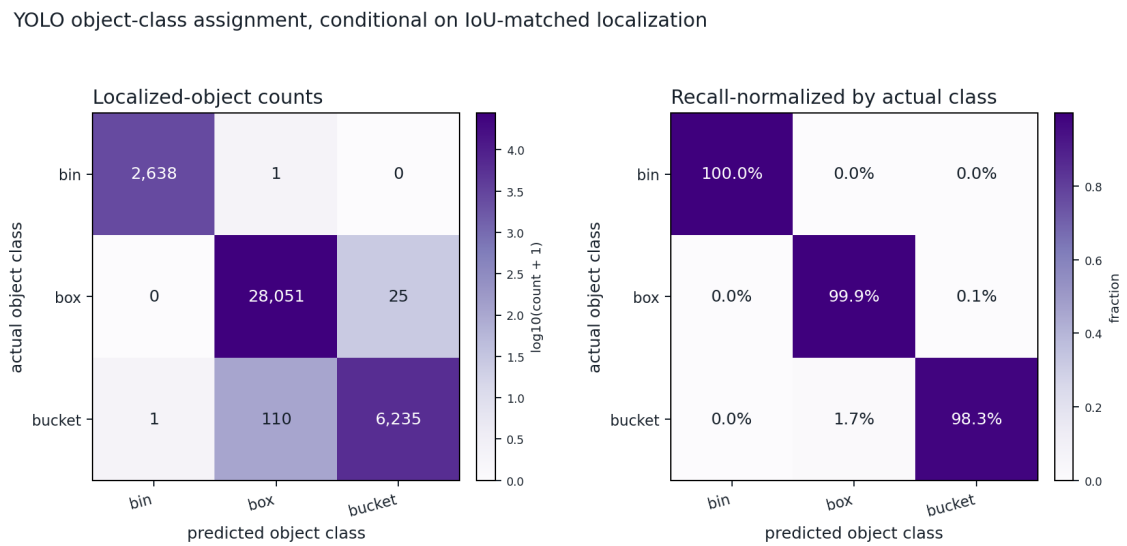


Figure 4.5 Object-class confusion conditional on class-agnostic IoU association. Rows are actual object classes and columns are predicted object classes. The left panel gives counts on a logarithmic colour scale; the right panel normalizes within each actual class. Background is intentionally excluded.

Summary. The detector is reliable at naming and unreliable at finding: pooled precision 0.684, moving-object recall 0.591, conditional class F1 0.995. The 41 % of moving objects it misses is the limit any downstream filter inherits, and it is computed over the easier, exemption-filtered subset; counting every exempt object as a miss gives a recall of 0.382.

Future plan. Validate the projected ground truth against manual annotation on a frame subset; add object velocity from consecutive detections so that the mask targets moving objects rather than object classes; and re-run on the realistic warehouse worlds of section 3.4.6, built after the cutoff, where the moving objects are pallet jacks on logistics jobs.

4.2.2 Real robot

Reason. The projected-geometry ground truth of section 3.6.2 needs object poses and meshes, which the real site does not provide. The three real recordings contain 2 562 to 3 405 usable stereo pairs, so the detector can be run, but nothing can score it until frames are annotated by hand.

Next action. Annotate a stratified frame subset from `dataset_real_000` to `_002` under the three illumination levels of section 3.5.4, run the detector at the same thresholds, and report the same metrics as section 4.2.1; the comparison between projected and manual truth planned for the simulation slot then also calibrates the two conventions against each other.

4.3 Experiment 3: Visual-Inertial Odometry With Dynamic-Environment Filtering

4.3.1 Simulation

Objective. Determine whether masking dynamic-object features improves localization and at what compute cost, and whether tracking survives it (section 3.6.3).

Process. The relogged campaign of 16 September 2026: both pipelines on seven featureless-floor recordings, baseline and filtered treatments as separate processes on identical input at 0.5× replay, three repeats per condition, scored by `script/vio_metrics.py` and rendered by `script/analyse_relogged_runs.py`.

Dataset. The three static and four dynamic `nofloortexture` recordings of table 3.19.

Campaign validity audit

The relogged campaign contains 66 estimator runs across seven featureless-floor simulation datasets. Each condition was requested three times. The four dynamic datasets contain baseline and nominal YOLO-filtered folders for both estimators ($4 \times 3 \times 4 = 48$ runs). The three static datasets contain baseline ORB-SLAM3 and VINS-Fusion runs ($3 \times 3 \times 2 = 18$ runs). All directories contain a trajectory, metadata, process-performance records, module records, and a graceful-shutdown summary.

The validity audit distinguishes an operational estimator run from a valid treatment comparison. All 66 runs initialized and produced usable log rows. Two ORB-SLAM3 runs—`dataset_static_nofloortexture_000/logging_20260916_02` and `logging_20260916_03`—have incomplete `run_summary.csv` schemas: their final inertial and map counters are absent. Their frame-level `performance.csv` records nevertheless reach `imu_initialized=1`, `inertial_ba1=1`, and `inertial_ba2=1`, while `local_mapping.csv` records mapping activity. They are therefore retained as operational runs; the missing summary fields are reported as a logging anomaly rather than

interpreted as estimator failure.

Table 4.12 Relogged experiment progress and validity. "Operational" requires graceful shutdown, estimator initialization, at least ten processed frames, and a trajectory. Treatment validity additionally requires the intended filter to be active and paired input counts and durations to differ by no more than 5%.

Run family	Attempted	Complete	Operational	Valid pairs	Interpretation
Static baseline	18	18	18	–	two incomplete ORB summaries
Dynamic baseline	24	24	24	–	descriptive estimator logs
Dynamic ORB+YOLO	12	12	12	1 of 12	11 input-mismatched pairs
Dynamic VINS+YOLO	12	12	12	0 of 12	filter inactive in every run
Total	66	66	66	1 treatment pair	preliminary campaign

Ground truth was extracted directly from each exact-name dataset rosbag's `/ground_truth/odometry` topic and associated with every repeat using source header timestamps. Dataset association follows the containing folder name, not the path recorded in `run_metadata.csv`. The PC copies were used for `dataset_dynamic_nofloortexture_01_000` and `dataset_static_nofloortexture_001`; the other five bags were read from the removable drive. That split was not a preference: both drive copies are unreadable despite carrying complete metadata, as recorded in table 3.19. Each estimate is aligned to truth by a rigid $SE(3)$ fit with scale fixed. Translational ATE RMSE and rotational geodesic RMSE can therefore be reported. Relative pose error was added to the analysis module after the cutoff and is reported in section 4.3.1; it required no estimator re-runs.

The nominal VINS+YOLO treatment did not pass the mechanism check. Although its metadata says `filter=true`, all 12 runs report zero matched detection frames, zero nonzero mask coverage, and zero removed dynamic tracks. These runs remain useful for operational and compute inspection but cannot test semantic filtering. ORB+YOLO recorded 9,706 matched and 10,764 missed detector frames over 20,470 mask-use rows, with no rejected masks. Its median mask coverage was zero (IQR 0.0798), reflecting many frames without a used mask. Eleven of its 12 baseline/filter pairs differed in received image count by more than 5%; only one pair satisfies the predefined pairing rule. Consequently, no cross-treatment accuracy conclusion is made.

Observed anomaly catalogue

Table 4.13 records anomalous behaviour visible in the logs and derived metrics. It deliberately separates observation from causal explanation: no item below identifies a root cause, and the possible mechanisms listed in the analysis catalogue remain untested.

Table 4.13 Anomaly catalogue for the relogged campaign. Severity prioritizes impact on interpretation, not certainty about cause.

ID	Severity	Observation	Quantitative evidence and reporting consequence
A1	Critical	Requested VINS+YOLO filtering was inactive	All 12 folders contain 28,038 mask rows in total, but zero matched detector messages, positive-box rows, or nonzero-coverage rows. They cannot measure an active semantic-filter treatment.
A2	High	ORB+YOLO exposure varied between repeats	Positive-box fractions range from 1.4% to 99.2%, including wide variation among repeats of the same bag. A matched detector message can contain zero boxes, so match count alone is not treatment dose.
A3	Critical	Every dynamic trajectory contains discontinuities	All 48 dynamic runs contain impossible-speed steps. Median affected-step fractions are 59.6% (VINS), 30.3% (ORB), 61.1% (requested VINS+YOLO), and 26.9% (ORB+YOLO). Global ATE cannot be read as ordinary drift.
A4	High	Global and clean-segment errors are strongly separated	Median global-to-clean ATE ratios are 100, 146, 101, and 239 respectively. Locally coherent intervals coexist with catastrophic full-run discontinuities.
A5	Medium	Static ORB has an isolated initialization-adjacent jump	All nine static ORB runs have one or two jumps, with the first near 2.6 s. Where an inertial transition timestamp exists, the jump precedes it by 0.10–0.17 s; this temporal association does not establish causation.
A6	Medium	Two ORB run summaries are incomplete	The two static repeat summaries omit final inertial/map fields, although frame and local-mapping logs record initialized operation. Summary-only validation would falsely exclude these runs.
A7	High	ORB baseline/filter inputs are not paired in 11 of 12 cases	Invalid pairs differ in received-frame count by 9.95–30.06%, while their durations differ by less than 0.13%. Most between-mode differences are descriptive, not controlled treatment effects.
A8	Medium	VINS reset counts are non-discriminating	All 33 VINS baseline and requested-filter runs have failure detection disabled and record zero resets. Zero resets therefore cannot establish healthy estimation.

Trajectory accuracy, plausibility and robustness

Table 4.14 reports medians and IQRs because the dynamic-run distributions are strongly skewed. In static scenes, VINS attained a median translational ATE RMSE of 0.370 m and rotational RMSE of 1.56° over nine valid runs. The nine valid static ORB runs attained 0.454 m and 17.92°. The orientation difference is much larger than the positional difference and must be retained as a separate result. Section 4.3.1 shows that it is produced almost entirely by one or two discrete events per run rather than by continuously worse orientation tracking.

Every dynamic run contains at least one impossible-speed step. Nine of 12 baseline

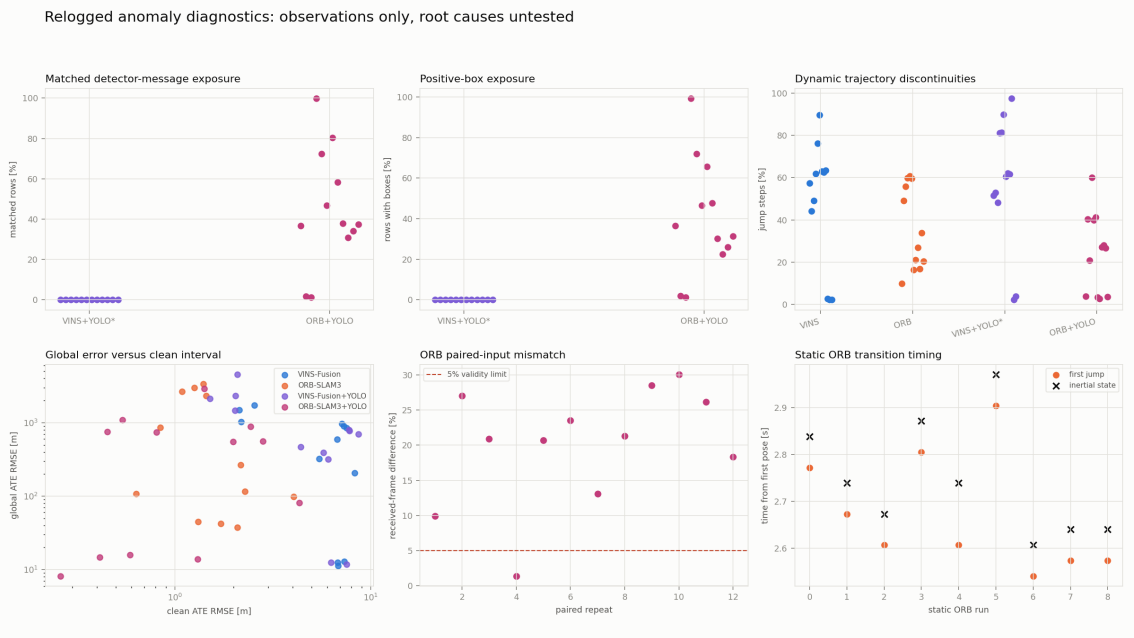


Figure 4.6 Run-level anomaly diagnostics. The panels show detector-message and positive-box exposure, dynamic jump fractions, global versus clean-segment ATE, ORB paired-input mismatch, and static ORB jump timing. These are observations only; the plot does not assign a root cause.

Table 4.14 Trajectory error against rosbag ground truth, median (IQR). "Clean" is ATE RMSE after independently aligning the longest segment without an implied speed above $10\,\mathrm{m\,s^{-1}}$. Requested VINS+YOLO values are not active-filter measurements.

Scene	Requested mode	ATE RMSE (m)	Rotation RMSE (deg)	Clean ATE (m)	Jumps
Static	VINS-Fusion	0.370 (0.063)	1.56 (0.98)	0.370 (0.063)	0 (0)
Static	ORB-SLAM3	0.454 (0.157)	17.92 (1.72)	0.341 (0.151)	1 (1)
Dynamic	VINS-Fusion	724.13 (823.52)	53.97 (39.38)	6.80 (2.57)	1116 (1157.75)
Dynamic	ORB-SLAM3	190.48 (2315.44)	108.29 (116.13)	1.42 (0.89)	484 (418.5)
Dynamic	VINS-Fusion+YOLO*	738.63 (1251.84)	56.50 (37.88)	5.91 (5.53)	1559.5 (717)
Dynamic	ORB-SLAM3+YOLO	556.05 (769.39)	138.42 (60.34)	1.06 (1.58)	208 (225.5)

*Filter requested but no detection frame was matched.

VINS runs, all 12 baseline ORB runs, 10 of 12 nominal VINS+YOLO runs, and all 12 ORB+YOLO runs also exceeded the reference-free mean-speed limit of $3\,\mathrm{m\,s^{-1}}$. Peak translational error exceeded 50 m in 9, 12, 10, and 10 runs respectively. Global ATE is therefore dominated by trajectory breakdown. The clean-segment ATE is substantially smaller, which an earlier draft read as evidence that the runs contain locally coherent intervals. Section 4.3.1 withdraws that reading: measured directly, the jump-free relative error is 1.8–3.8 m per second, so the estimate is locally wrong by more than the distance the robot travels. Aligning a short segment absorbs much of that error, which is why clean-segment ATE looked reassuring.

The per-repeat values behind those aggregates are given in table 4.17.

Relative pose error

RPE was implemented after the reporting cutoff and applied to the campaign without re-running any estimator; section 3.7.1 defines it. It is reported here because it needs no global alignment, so unlike ATE it cannot be distorted by a single discontinuity re-seating the rigid fit. Table 4.15 gives the headline interval.

Table 4.15 Relative pose error at $\Delta = 1\,\mathrm{s}$, median (IQR) over runs. "Jump-free" excludes pairs spanning a detected tracking jump; the difference between the two columns is the teleport contribution. Requested VINS+YOLO values are not active-filter measurements.

			Translational (m)		Rotational (deg)	
Scene	Requested mode	Jumps	all pairs	jump-free	all pairs	jump-free
Static	VINS-Fusion	0 (0)	0.063 (0.027)	0.063 (0.027)	0.57 (0.05)	0.57 (0.05)
Static	ORB-SLAM3	1 (1)	0.310 (0.308)	0.096 (0.064)	10.56 (0.69)	0.25 (0.33)
Dynamic	VINS-Fusion	1116 (1158)	44.652 (49.397)	2.825 (1.595)	2.05 (1.63)	2.76 (2.01)
Dynamic	ORB-SLAM3	484 (418)	19.477 (158.567)	2.079 (0.267)	15.15 (4.82)	2.49 (2.51)
Dynamic	VINS-Fusion+YOLO*	1560 (717)	44.106 (95.564)	3.784 (1.611)	2.33 (4.76)	2.72 (1.02)
Dynamic	ORB-SLAM3+YOLO	208 (226)	38.174 (78.432)	1.805 (0.634)	21.60 (9.72)	6.33 (10.39)

Two results follow, and the first corrects this chapter.

The dynamic runs are not locally coherent. Excluding every pair that spans a jump, dynamic translational RPE at $\Delta = 1\,\mathrm{s}$ is still 1.8–3.8 m. The robot covers about 1.4 m in that second, so between teleports the estimate misstates its own motion by more than the distance travelled. Against the static runs, whose jump-free figures are 0.063–0.096 m, dynamic local error is 20 to 60 times worse. The corruption is therefore continuous, not confined to isolated steps, and the earlier reading of clean-segment ATE as evidence of local coherence does not survive. This was a registered prediction before the metric was run, and it failed in the direction that matters.

Table 4.16 shows why this is one mechanism rather than two. Error grows with interval at 4.5–6.7× over a tenfold increase in Δ, and that growth exponent is the same for static and dynamic scenes; only the magnitude differs, by roughly 40×. The dynamic runs are not failing in a qualitatively different way from the static ones — they are running the same error process far hotter.

ORB-SLAM3's static rotational penalty is entirely jump-localised. Static ORB-SLAM3 carries a rotational ATE of 17.92° against VINS-Fusion's 1.56°, an elevenfold

Table 4.16 Jump-free translational RPE against interval, median over runs, in metres. "Growth" is the ratio from 0.5 s to 5 s; linear accumulation would give 10× and a pure random walk 3.2×.

Scene	Requested mode	0.5 s	1.0 s	2.0 s	5.0 s	Growth
Static	VINS-Fusion	0.038	0.063	0.112	0.252	6.7×
Static	ORB-SLAM3	0.057	0.096	0.162	0.307	5.4×
Dynamic	VINS-Fusion	1.524	2.825	5.067	10.082	6.6×
Dynamic	ORB-SLAM3	1.158	2.079	3.776	6.787	5.9×
Dynamic	VINS-Fusion+YOLO*	2.189	3.784	5.455	10.985	5.0×
Dynamic	ORB-SLAM3+YOLO	0.976	1.805	3.041	4.384	4.5×

gap that invites the reading that ORB-SLAM3 tracks orientation poorly. RPE refutes it. Across the nine static ORB runs, all-pairs rotational RPE is 9.28–11.63°, but jump-free rotational RPE is 0.10–0.66° — at or below VINS-Fusion's 0.49–0.69° on the same recordings. Every one of those runs contains one or two jumps and no more.

The whole rotational gap is thus produced by one or two discrete re-seating events per run, not by continuously worse tracking. This gives anomaly A5 its significance: that jump occurs near 2.6 s, immediately before inertial initialization completes, and it is predominantly rotational. Between those events ORB-SLAM3's orientation tracking is competitive with VINS-Fusion's. No causal mechanism is established here — the association is temporal — but the error is now localized to an event rather than attributed to the estimator's steady-state behaviour.

Baseline ORB-SLAM3 logged 7.75 ± 6.03 tracking-loss events and 9.52 ± 6.49 s of lost tracking per dynamic run (mean $\pm$ sample SD, $n = 12$). ORB+YOLO logged 9.17 ± 3.01 events and 11.46 ± 5.32 s. VINS logged no estimator resets in either requested mode, but the absence of resets does not make its trajectories plausible. The event types are estimator-specific and are not treated as numerically equivalent.

Real-time and computational measurements

Table 4.18 summarizes process-local measurements over the 12 dynamic runs in each requested mode. CPU is the change in cumulative process CPU time divided by monotonic wall time; 100% denotes one fully used logical core, so values above 100% represent multithreaded execution. Memory is peak resident set size. The separate detector process, rosbag, Gazebo, RViz, and RTAB-Map are not included.

The ORB+YOLO folders show lower achieved throughput and retention than baseline ORB, while process CPU is of similar order. This is an observation, not an isolated detector-cost effect: the external detector is excluded from CPU and 11 paired runs received materially different image counts. Baseline VINS processed all enqueued images in its logs, whereas baseline ORB discarded frames whenever the one-frame queue was overwritten.

Per-module results use each run's event distribution. Figure 4.10 shows pooled event boxplots for shape, overlaid with one median point per run so frames are not mistaken for independent experimental repeats. Across run medians, ORB tracking was 63.19 ± 7.06 ms and ORB local mapping was 189.58 ± 16.95 ms. The requested ORB+YOLO mode measured 112.53 ± 24.61 ms and 356.30 ± 68.36 ms respectively. VINS feature tracking was 48.00 ± 10.83 ms and its back end was 60.41 ± 17.12 ms; nominal VINS+YOLO values were 43.61 ± 11.49 ms and 57.90 ± 16.29 ms, but cannot be interpreted as filter

Table 4.17 Truth-based trajectory error for every repeat of the relogged campaign. Every dataset is a no-floor-texture world, so that shared part of each dataset name is omitted. "ATE" and "clean" are global and longest-clean-segment ATE RMSE in metres, "rot" is rotational geodesic RMSE in degrees, and "jmp" counts steps implying more than $10\,\mathrm{m\,s^{-1}}$. All 66 runs are operational; two static ORB run summaries have incomplete schemas, and only one of 12 ORB baseline/filter pairs passed input matching. Static datasets were not run in either filtered mode; those cells are set to –.

		VINS-Fusion				ORB-SLAM3				VINS-Fusion+YOLO*				ORB-SLAM3+YOLO			
Dataset	Rep	ATE (m)	rot (deg)	clean (m)	jmp (n)	ATE (m)	rot (deg)	clean (m)	jmp (n)	ATE (m)	rot (deg)	clean (m)	jmp (n)	ATE (m)	rot (deg)	clean (m)	jmp (n)
dynamic 00_000	01	596.37	38.5	6.77	858	45.21	176.5	1.31	109	392.55	107.0	5.74	1122	13.84	76.7	1.31	37
dynamic 00_000	02	206.97	92.2	8.32	609	857.95	162.0	0.84	512	466.72	108.0	4.39	1183	892.31	129.9	2.45	213
dynamic 00_000	03	324.01	88.4	5.47	580	3364.34	70.8	1.40	610	318.66	108.7	6.07	991	81.20	173.1	4.33	119
dynamic 00_001	01	1030.40	63.6	2.18	1374	3002.57	29.7	1.26	1150	2309.93	74.6	2.04	1805	2885.77	82.2	1.42	1159
dynamic 00_001	02	1481.00	62.8	2.14	1695	2669.73	26.1	1.09	1141	2129.97	75.2	1.51	1810	553.35	93.5	1.98	454
dynamic 00_001	03	1724.94	175.1	2.55	1995	2311.30	23.1	1.45	1098	1457.89	59.6	2.03	1997	558.76	112.8	2.82	378
dynamic 01_000	01	904.94	53.3	7.29	1608	98.88	169.1	4.05	239	698.15	52.0	8.67	1545	8.20	102.3	0.26	44
dynamic 01_000	02	851.90	53.8	7.55	1596	115.45	172.4	2.28	327	796.03	52.6	7.77	1582	14.78	173.8	0.41	26
dynamic 01_000	03	965.82	54.1	7.14	1619	107.60	167.7	0.64	456	779.10	53.3	7.79	1574	749.16	162.6	0.45	203
dynamic 02_000	01	12.49	6.1	6.80	72	37.83	59.3	2.09	388	4518.77	25.1	2.09	2610	1097.95	159.7	0.54	231
dynamic 02_000	02	12.88	5.5	7.36	61	265.50	62.9	2.17	689	11.85	6.4	7.55	62	737.27	148.9	0.81	219
dynamic 02_000	03	11.30	4.3	6.81	58	42.37	145.8	1.72	408	12.55	6.9	6.29	66	15.81	146.9	0.59	31
static 000	01	0.37	1.6	0.37	0	0.58	17.9	0.55	1	–	–	–	–	–	–	–	–
static 000	02	0.41	1.5	0.41	0	0.79	20.4	0.77	1	–	–	–	–	–	–	–	–
static 000	03	0.43	1.6	0.43	0	1.20	19.1	0.30	2	–	–	–	–	–	–	–	–
static 001	01	0.39	2.3	0.39	0	0.45	18.9	0.27	2	–	–	–	–	–	–	–	–
static 001	02	0.29	1.8	0.29	0	0.45	18.1	0.21	2	–	–	–	–	–	–	–	–
static 001	03	0.48	2.2	0.48	0	0.38	17.2	0.26	1	–	–	–	–	–	–	–	–
static 002	01	0.34	0.8	0.34	0	0.42	16.7	0.39	1	–	–	–	–	–	–	–	–
static 002	02	0.36	0.8	0.36	0	0.46	17.6	0.43	1	–	–	–	–	–	–	–	–
static 002	03	0.34	0.7	0.34	0	0.36	16.9	0.34	1	–	–	–	–	–	–	–	–

*Filter requested but no detection frame was matched in any repeat; these columns are not active-filter measurements.

Requested mode	Poses (n)	Overlap (s)	Path (m)	ATE RMSE (m)	Rot RMSE (deg)	Clean ATE (m)	Jumps (n)
VINS-Fusion	1494	77.0	5486.93	596.367	38.51	6.771	858
ORB-SLAM3	1105	77.3	408.22	45.206	176.53	1.314	109
VINS-Fusion+YOLO*	2175	77.1	1958.14	392.546	107.04	5.743	1122
ORB-SLAM3+YOLO	943	77.2	239.53	13.839	76.72	1.310	37

*Filter requested but no detection frame was matched.

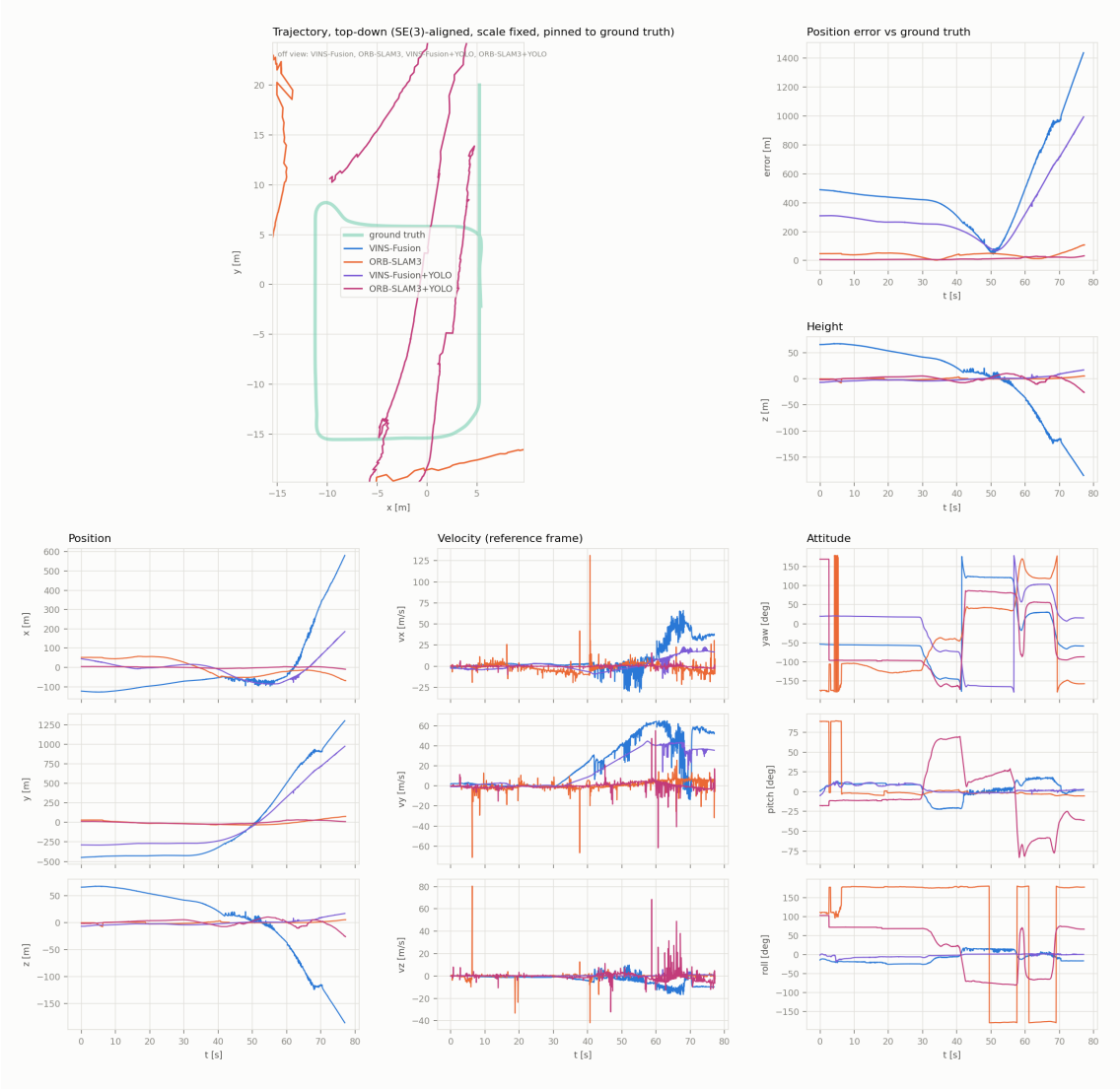


Figure 4.7 Representative repeated-run trajectory diagnostic for `dataset_dynamic_noflo ortexture_00_000/logging_20260916_01`, in the same visual format as the existing comparison figures. The table gives the per-system summary for this run; "clean" is ATE RMSE over the longest jump-free segment. The very large paths and frequent jumps are retained rather than clipped into a favourable view. ATE and rotational errors use the rosbag `/ground_truth/ odometry` topic after rigid $SE(3)$ alignment with scale fixed. The nominal VINS+YOLO trace is invalid as a filtering treatment because no masks were matched.

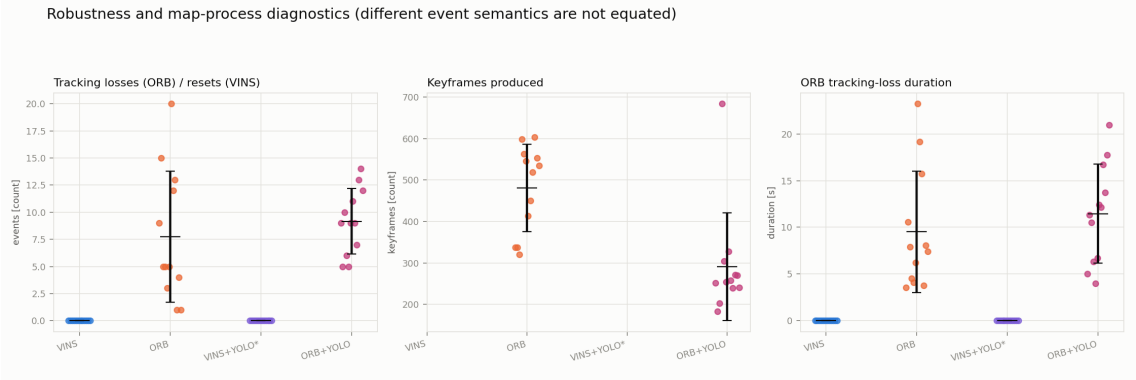


Figure 4.8 Per-run robustness and keyframe observations for the dynamic datasets. Points are experimental runs and black marks are mean ± sample SD. VINS reset count and ORB tracking-loss count have different semantics. The asterisk identifies the requested VINS filter mode in which no masks were matched.

Table 4.18 Dynamic-run capacity and resource measurements, mean ± sample SD, $n = 12$ per row. Nominal VINS+YOLO values are not active-filter measurements.

Requested mode	Throughput (Hz)	Retained input (%)	CPU (% core)	Peak RSS (MiB)
VINS-Fusion	27.14 ± 5.81	100.00 ± 0.00	443.66 ± 58.96	629.51 ± 282.62
ORB-SLAM3	23.47 ± 4.77	88.55 ± 13.42	385.22 ± 27.71	1226.06 ± 81.83
VINS-Fusion+YOLO*	28.83 ± 3.16	100.00 ± 0.00	406.43 ± 89.95	646.14 ± 306.15
ORB-SLAM3+YOLO	14.47 ± 5.42	66.96 ± 15.26	410.86 ± 42.83	1054.64 ± 91.11

*Filter requested but no detection frame was matched.

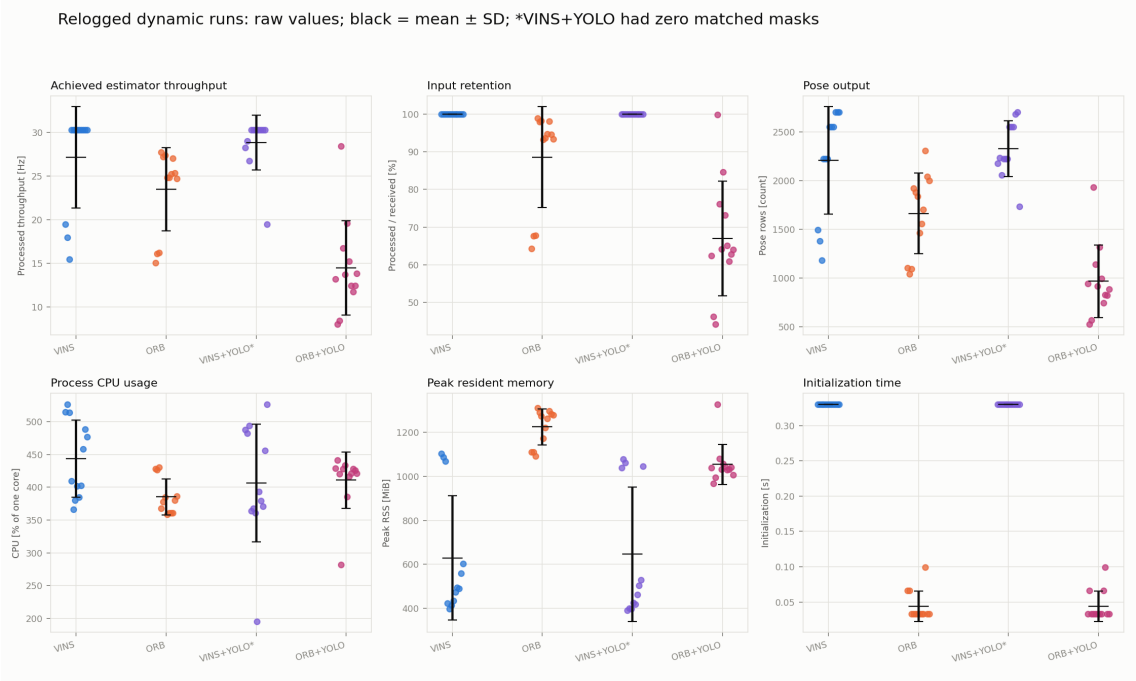


Figure 4.9 Raw dynamic-run capacity and process-resource values. Each point is a run; black marks show mean ± sample SD. This plot exposes run-to-run spread rather than presenting only aggregate bars.

effects because that filter was inactive. These timings refer to different estimator stages and are not added or treated as directly equivalent algorithms.

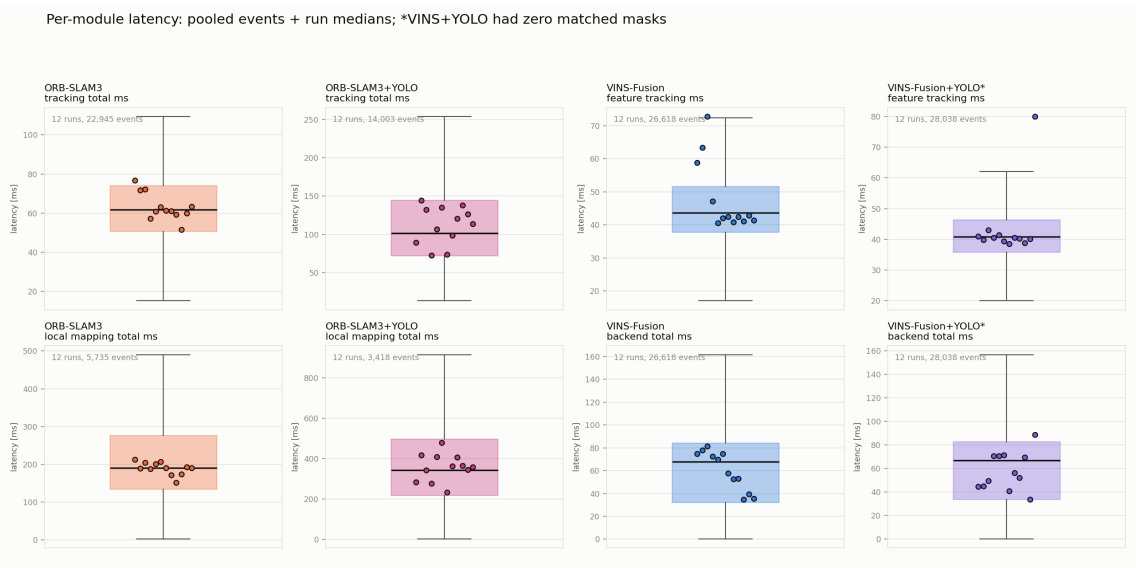


Figure 4.10 Module-latency distributions for dynamic datasets. Boxes summarize all recorded module events; coloured points are per-run medians ($n = 12$ in each requested mode). ORB local mapping is asynchronous per keyframe, while VINS back-end rows follow estimator updates.

Loop closure, map merging and global-correction evidence

The expanded ORB logs now expose back-end events. Baseline ORB-SLAM3 recorded six loop closures: two in each repeat of `dataset_static_nofloortexture_002`. Each of those runs also recorded two global bundle-adjustment executions. One baseline dynamic run (`dataset_dynamic_nofloortexture_02_000/logging_20260916_03`) and one ORB+YOLO run (`dataset_dynamic_nofloortexture_01_000/logging_20260916_02`) recorded a map merge. These counters demonstrate that the native ORB back-end paths executed; they do not demonstrate that the resulting corrections were correct. The failed trajectory-plausibility checks and the absence of an event-aligned before/after counterfactual prevent a correction-benefit claim. RTAB-Map reconstruction remains a separate pipeline and is not exercised by this relogged campaign; its dense-map evidence is reported independently in section 4.4.1, on a different world and database.

Summary. In static scenes the two front ends reach a median aligned ATE of 0.370 m (VINS-Fusion) and 0.454 m (ORB-SLAM3), and ORB-SLAM3's large rotational error is produced by one or two discrete initialisation-adjacent jumps rather than by its steady-state tracking. The dynamic-scene numbers are withheld as findings: every dynamic trajectory is locally wrong by more than the distance the robot travels, and the environment that produced them is not physical. The core hypothesis therefore has no valid test yet, and neither pipeline processes frames as fast as they arrive at the replay rate used.

Future plan. Fix the detection-to-VINS integration so that masked tracks are actually culled, verified by the removed-track counter before any re-run; reduce ORB-SLAM3+YOLO26 latency so that filtered and unfiltered runs see the same frames; and rebuild the dynamic worlds with physically simulated props.

4.3.2 Real robot

Reason. The three calibration gaps of section 3.5.3 bound what any real run can show to plausibility and tracking continuity; metric ATE and RPE need the motion-capture reference of section 3.5.4, which has not been recorded. Runs on `dataset_real_000` are consequently not reported here as results.

Next action. Complete the camera–IMU calibration (extrinsic translation, per-camera intrinsics), record the motion-capture trajectory, then execute the four-step online procedure of section 3.6.3 for both pipelines at the three illumination levels, at $1.5\,\mathrm{m\,s^{-1}}$, with the dynamic variable triggered at each zone transfer.

4.4 Experiment 4: Dense Map Reconstruction and Map-Quality Evaluation

4.4.1 Simulation

Objective. Verify that the reconstructed map reflects the physical environment and is usable by a planner (section 3.6.4).

Process. RTAB-Map 0.23.7 built `map/map_tiny.db`; its placement was measured by `script/fit_map_pose.py`; geometry and drivability were scored by `script/validate_map.py` against the world file and a ground-truth route.

Dataset. `dataset_map_tiny` (the build run) and the independent route `dataset_static_tiny_repeat`, which no longer exists; and, for the large warehouse, `dataset_map_01` (table 3.19).

Question, scope, and conditions

Trajectory error answers whether the robot knew where it was. It does not answer whether the map it produced describes the building, nor whether a planner could have used that map. This section evaluates the one dense map reconstructed so far against two references of very different strength: the simulator's own collision geometry, which is independent of the map, and the ground-truth route of the run that built the map, which is not. The two questions are kept separate because they fail in different ways — a map can be geometrically faithful and still unusable, and it can be navigable over an easy route while misrepresenting the parts of the building that route never entered — and because only the first reference is independent evidence.

The evaluation is confined to the **tiny** warehouse. A large-warehouse equivalent is **planned**; no database for it exists at the reporting cutoff. Tiny-map and large-map results must not be pooled or read as repeated trials of one condition, because the two worlds differ in extent, aisle width, and route length. Table 4.19 records the conditions.

Map placement as a validity precondition

Both metrics depend on seating the occupancy grid in world coordinates, so placement is a precondition rather than a display choice: a mis-seated grid scores real structure against the wrong cells and makes a good map look bad. The transform is a constant for a given database and is measured once, by fitting occupied cells to the world file's wall and obstacle edges without using odometry or ground truth. It is deliberately not taken from the live

Table 4.19 Conditions for the dense-map evaluation. The database and the ground-truth route come from the *same* run, which bounds what the drivability figures can establish.

Factor	Level
Map under test	`map/map_tiny.db`, RTAB-Map 0.23.7, created 2026-09-08
World	`aws-robomaker-small-warehouse-world/worlds/small_warehouse_static/small_warehouse_static_tinymap.world` (tiny geometry, despite residing in the `small_warehouse_static` family directory)
Scene dynamics	static
Grid resolution	0.05 m
Geometry tolerance	0.25 m
Robot footprint	0.83×0.50 m rectangle from `nav2_ackermann.yaml`, not a circumscribed radius
World-to-map placement	$x = 1.850$ m, $y = 8.600$ m, $\theta = -89.90°$, measured once by `script/fit_map_pose.py`
Drivability reference, as plotted	`dataset_static_tiny_repeat`, 2973 poses; an independent run, no longer present in the workspace or on the removable drive
Substitute available	`dataset/dataset_map_tiny`, `/ground_truth/odometry`: 3032 messages over 60.6 s of simulation time. This is the run that *built* the map, so it is not an equivalent replacement
Evaluator	`script/validate_map.py`
Experimental unit, n	one database; $n = 1$

`map` → `odom` correction, which moves with front-end drift.

The measured fit is $(1.850, 8.600)$ m at $-89.90°$, with a margin of 60.9 % over the best rival orientation more than 5° away. An independent prediction — the spawn pose of the IMU body frame, which sits 0.338 m ahead of `base_footprint` — gives $(1.800, 8.662)$ m. The two agree to 0.080 m, inside the grid's own 0.05 m resolution. Two independent routes to the same number is what makes this a measurement rather than a tuned parameter.

Graph and database summary

Table 4.20 summarizes the stored graph, read directly from the database in read-only mode. The counts are small: 30 nodes over a 59.3 s span, reflecting RTAB-Map's retention policy rather than the rate at which images were offered to it.

Table 4.20 Stored contents of `map/map_tiny.db`. Link rows are stored per direction; the undirected count is the number of distinct node pairs.

Element	Rows	Undirected	Note
Nodes	30	–	ids 1–30, stamp span 59.3 s
Neighbour links	56	28	sequential odometry constraints
Global loop closures	5	3	pairs $(9,22)$, $(10,23)$, $(29,30)$
Gravity links	30	30	one per node
Local-space links	0	0	none recorded
Visual words	10 357	–	vocabulary size
Features	28 374	–	stored keypoints

Of the three distinct closure pairs, $(9, 22)$ and $(10, 23)$ join nodes roughly half a lap apart and are genuine revisits of an earlier place. The third, $(29, 30)$, joins consecutive nodes and is therefore not a revisit; it should not be counted as evidence of place recognition.

Geometry against the world file

Geometry is scored against the rasterized collision geometry of the world file at a 0.25 m tolerance. Recall is measured against *surfaces* rather than filled interiors, because a stereo camera can only observe the outside of a shelf; counting its interior as missed geometry would cap even a perfect map well below 100 %.

Table 4.21 Occupied-cell geometry of `map/map_tiny.db` against `small_warehouse_static_tinymap.world` at 0.25 m tolerance.

Metric	Value	Population
Occupied-cell precision	96.6 %	11 634 occupied cells
Recall, region the map observed	89.5 %	observed surfaces
Recall, whole building	67.2 %	all surfaces in the world file

The gap between 89.5 % and 67.2 % is route coverage, not mapping failure: the run never entered the side aisles, so roughly a third of the building was never observed by any sensor. The two numbers answer different questions and neither replaces the other — the first is a statement about mapping quality, the second about where the robot went.

Drivability against the true route

The robot demonstrably drove the reference route, so every pose on it was collision-free in reality. Stamping the real footprint onto the map at each pose therefore converts any collision into a *proven* false obstacle, and any unknown cell into a hole a planner would have to cross blind. This metric needs no assumption about what a good map looks like.

It does, however, depend on where the route came from, and that is the weak point of this result. The route used to produce table 4.22 came from a separate run, `dataset_static_tiny_repeat`, which is no longer present in the workspace or on the removable drive. The figures below therefore stand as a dated independent measurement that cannot currently be re-derived.

The only surviving recording of this warehouse, `dataset/dataset_map_tiny`, is not an equivalent substitute, because it is the run that built the map. Three measurements establish this. Applying the placement of section 4.4.1 to the stored node poses puts them a median of 0.069 m from that recording's ground-truth route; comparing each node against the ground-truth pose carrying the *same* timestamp gives a median of 0.340 m and a maximum of 0.52 m; and all 30 nodes fall inside the recording's sim-time interval. The second of these is the discriminating one, since a different run over the same loop would agree in space but not in time. The two routes are also visibly different: the lost run is a single oval circuit, while this one is a figure-eight through the centre of the building.

The evaluation was nevertheless repeated against this recording, and the outcome confirms the objection rather than resolving it. Both results are reported in table 4.22.

The difference between the two columns is the independence effect, measured rather than assumed. Against the run that built it, the map records *no* false obstacle at all: it cannot

Table 4.22 Drivability of `map/map_tiny.db` under the true 0.83×0.50 m nav2 footprint, evaluated against two different routes. "Independent" is `dataset_static_tiny_repeat`, the run that did not build the map and is no longer available. "Build run" is `dataset/dataset_map_tiny`, the run the map was constructed from. Geometry is identical in both cases because it does not depend on the route.

Metric	Independent	Build run	Meaning
Poses scored	2973	3032	
Clear and mapped	99.3 %	100.0 %	footprint free and observed
False obstacle	0.7 %	0.0 %	proven error
Unmapped under footprint	0.0 %	0.0 %	blind cells on the route
Clearance, median	1.07 m	1.01 m	to nearest obstacle
Clearance, 5th percentile	0.45 m	0.46 m	inflation radius is 0.80 m

contradict the trajectory it was constructed from. The 0.7 % that the independent run exposes is precisely the part of the map that only a route the mapper had not seen could reveal. The right-hand column is therefore not a better result than the left; it is a weaker question, and the 0.0 % should not be quoted as a map-quality figure.

Clearance is nearly unchanged between the two, which is expected: it is a property of where each route runs relative to mapped geometry, not of which run built the map. Both routes spend part of their length inside nav2's inflation radius.

Interpretation and limitations

Within its stated scope the tiny map is geometrically faithful and navigable: 96.6 % of what it marks as occupied is real, it recovers 89.5 % of the surfaces it actually observed, and it contains no blind cells and only 0.7 % proven false obstacles along a route the robot really drove. The geometry half of that is independent evidence and is the strongest map result the project currently holds; the drivability half is a self-consistency check and is weaker than it looks. Both are a direct improvement on earlier assessments that judged the same database unnavigable — those were measured with the grid mis-seated, so the comparison was against the wrong cells. That correction is a methodological finding in its own right and is why section 4.4.1 reports placement before any metric.

Four limitations bound what these numbers support.

- **The drivability reference cannot be re-read.** The run that provided it is lost, and the only surviving recording of this warehouse is the one that built the map, which would answer a weaker question. Re-establishing this result needs a fresh independent recording — a second run, not a second map.
- **The route is easy.** It is one large loop down the wide central aisle and never squeezes into a side aisle. The 0.7 % false-obstacle figure is therefore a floor, not a worst case. A route through the narrow aisles is the test that would actually stress this map, and it has not been run.
- **Clearance couples to planner cost.** The 5th-percentile clearance of 0.45 m sits below nav2's 0.80 m inflation radius, and a visible share of the true route lies inside it. Nothing is blocked, but a planner would charge a high cost for driving where the robot demonstrably drove. That coupling should be examined before any planned path is trusted.
- **$n = 1$, and the scope is one world.** A single database on the tiny warehouse supports

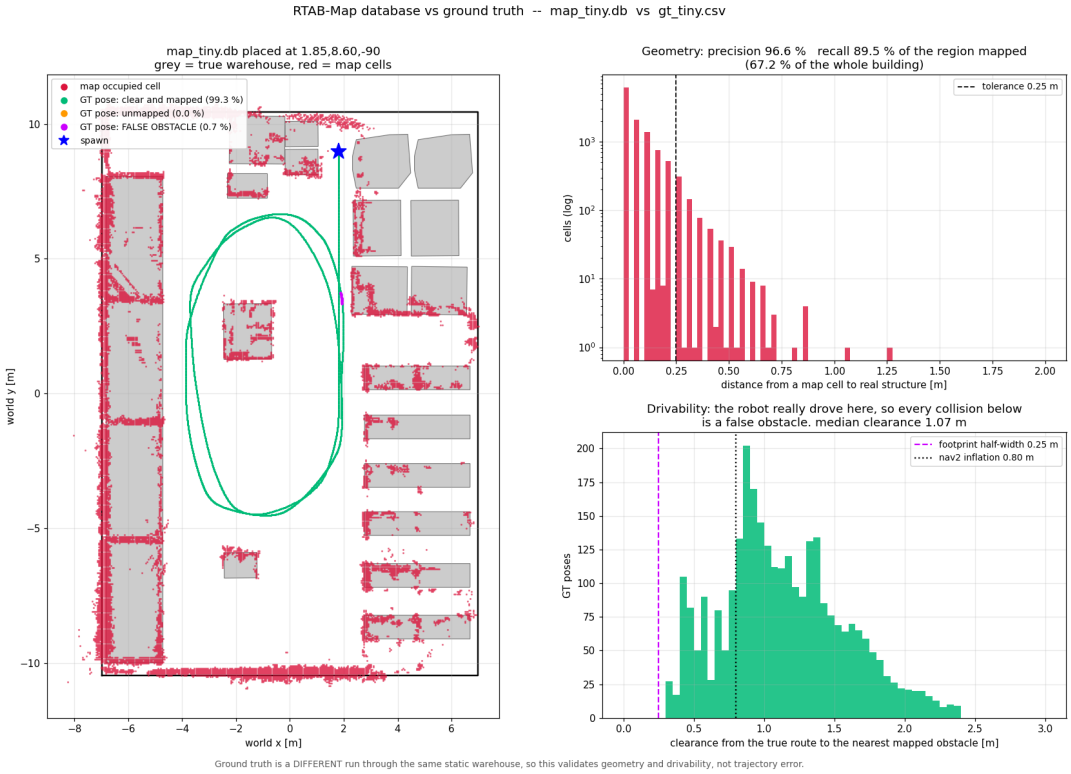


Figure 4.11 Dense-map evaluation of `map/map_tiny.db`. Left: occupied cells (red) over the world file's true collision geometry (grey), with the ground-truth route coloured by drivability and the spawn marked. Top right: distance from each occupied cell to real structure, with the 0.25 m tolerance marked; the bulk falls well inside tolerance with a thin tail to about 1.25 m. Bottom right: clearance from the true route to the nearest mapped obstacle, against the footprint half-width and the nav2 inflation radius. The ground truth was a different run through the same static warehouse, so this validates geometry and drivability, not trajectory error; that run is no longer available (section 4.4.1).

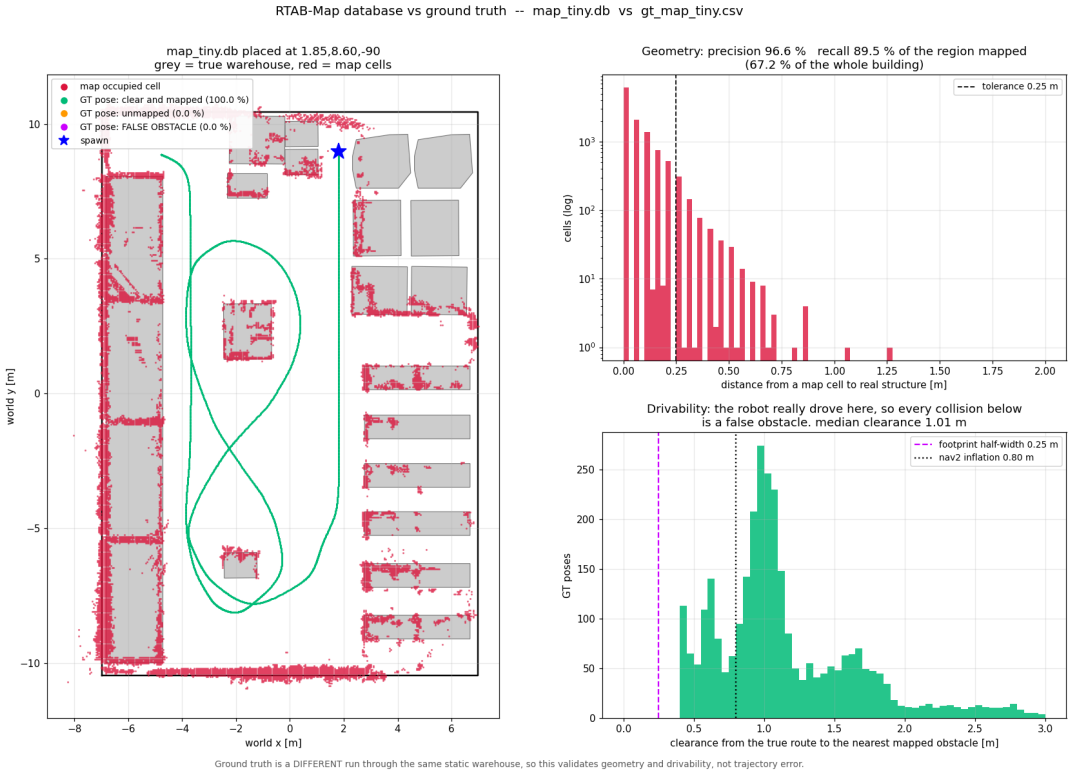


Figure 4.12 The same database scored against the run that built it, `dataset/dataset_map_tiny`. The route is a figure-eight rather than the oval of figure 4.11, and no pose on it collides with the map: the false-obstacle rate falls from 0.7 % to 0.0 %. The geometry panels are identical to figure 4.11 because geometry does not depend on the route. The footer claiming a different run is emitted unconditionally by the evaluator and is incorrect for this figure.

no uncertainty estimate and no claim about the large warehouse, where longer routes and narrower bays change both the drift exposure and the clearance margin.

- **Residual distortion is not a placement problem.** Walls appear doubled and thickened and shelf rows bowed in the overlay. No rigid transform removes that; it is internal map distortion and is the property that would limit navigation, independently of how well the grid is seated.

Planned extension to the large warehouse

The same evaluation is **planned** for the large warehouse and is not started: no large-map database exists at the cutoff. Three things must be settled before those numbers would mean anything. The world-to-map placement must be measured for the new database rather than assumed, since the tiny-map constant $(1.850, 8.600, -89.90^{\circ})$ is specific to that database and world. A ground-truth route must be recorded from a run *other* than the one that builds the map, which the tiny-map case did not do and which is what would turn drivability into independent evidence. Finally, the route should deliberately include narrow bays, so that the false-obstacle rate is exercised rather than merely reported. Until then, no map-quality claim extends beyond the tiny warehouse.

Large-warehouse attempt

The progress update of 23 September 2026 reports that the large-warehouse map attempt **failed**: the front end lost tracking in the repetitive bay structure of the large world, the estimated pose diverged from the ground-truth route within a few time steps of the loss, and the resulting map was not scorable. The failure was attributed there to the front end under repeated structure, not to the mapping back end. No database, run directory or figure for this attempt is registered in the repository at the cutoff, so the result is reported on the strength of the progress record and cannot yet be audited. The attempt must be repeated with its artifacts retained before it is carried into any conclusion.

Summary. The mapping pipeline is geometrically sound where tracking is sound: on the tiny map, occupied-cell precision is 96.6 % and observed-surface recall 89.5 %, and the whole-building recall of 67.2 % is route coverage, not mapping failure. The drivability half is a dated measurement whose independent route is lost. On the large map the pipeline fails through repetitive structure, before the back end is exercised.

Future plan. Study methods robust to repeated features (place-recognition gating, geometric verification of loop candidates); enable ORB-SLAM3's Atlas so that a tracking loss opens a new sub-map instead of corrupting the existing one, merging sub-maps afterwards; and record an independent route on both maps and score drivability against it.

4.4.2 Real robot

Reason. The geometry metric of section 3.6.4 scores occupied cells against known collision geometry; the real site has no such model yet, and the placement step needs it. The drivability metric needs the motion-capture route. Neither input exists at the cutoff.

Next action. Survey the mock warehouse (walls, stacks, walkway) into a reference occupancy grid at the same 0.05 m resolution, build a database from a real mapping run, and then score it with the unchanged evaluator, using a motion-capture route from a second

run for drivability.

4.5 Cross-Experiment Discussion, Simulation-to-Real Comparison and Limitations

4.5.1 Simulation against the real robot

No simulation-to-real comparison can be drawn at the cutoff, because no real-robot slot holds a valid result. What the simulation slots establish for the real campaign is the order in which its risks should be retired: the noise model matters more than the estimator architecture (section 4.1.1), so the real IMU must be characterised before any pipeline is tuned; the detector's miss rate bounds any filtering benefit (section 4.2.1), so real-image recall must be measured before the filter is judged; and the front end, not the back end, is where the large-map failure sits (section 4.4.1), so tracking robustness under repeated bay structure is the first mapping risk on the real site.

4.5.2 Where each pipeline is weak, block by block

Figures 4.13 and 4.14 put every instrumented stage of the three pipelines on one page and colour it by a stated criterion. Table 4.23 through table 4.27 give the evidence behind every colour: the metric, its value, the criterion it was tested against, and which of three rules produced the verdict. Both the figures' fills and the tables are generated by `script/block_performance.py` from the run artifacts; no colour in either figure was chosen by hand.

The three rules are deliberately distinguished, because they answer different questions and only one of them is about accuracy:

ablation
: The block was removed, disabled or swapped and the run repeated. The value is the ratio of aligned ATE without it to aligned ATE with it. This is the only rule that speaks to trajectory error.

latency
: The block's median time as a share of its stage, or a stage against the 66.0 ms inter-frame budget implied by 30.3 Hz imagery at $0.5\times$ replay. Describes cost, not accuracy.

outcome
: A counter the block emits, against a stated threshold—accepted loop candidates, usable solver solutions, filter consistency.

Where the criteria come from. Every verdict is a comparison against a threshold, and those thresholds do not all have the same standing. The tables label each one, and the distinction matters more than the values:

derived
: Follows from something outside this report. There are two. The 66.0 ms frame budget is arithmetic on the recorded conditions, $1000/(30.3\,\text{Hz} \times 0.5\ \text{replay})$: a per-frame stage slower than this cannot keep up, and no judgement is involved. The consistency target ANEES $= 3.0$ is the standard result that a correctly scaled covariance makes the normalised error squared average the degrees of freedom,

which for a pose error is three.

convention

A round number adopted here so the rule is stated rather than left implicit. Nothing external fixes 40 % of a stage rather than 35 %, 2.0× for a load-bearing block rather than 1.5×, 10 % of loop candidates accepted, or 99 % of solver calls returning a usable solution. These are contestable, which is why the measured value sits beside the threshold in every row: a reader who prefers a different line can redraw it without re-running anything.

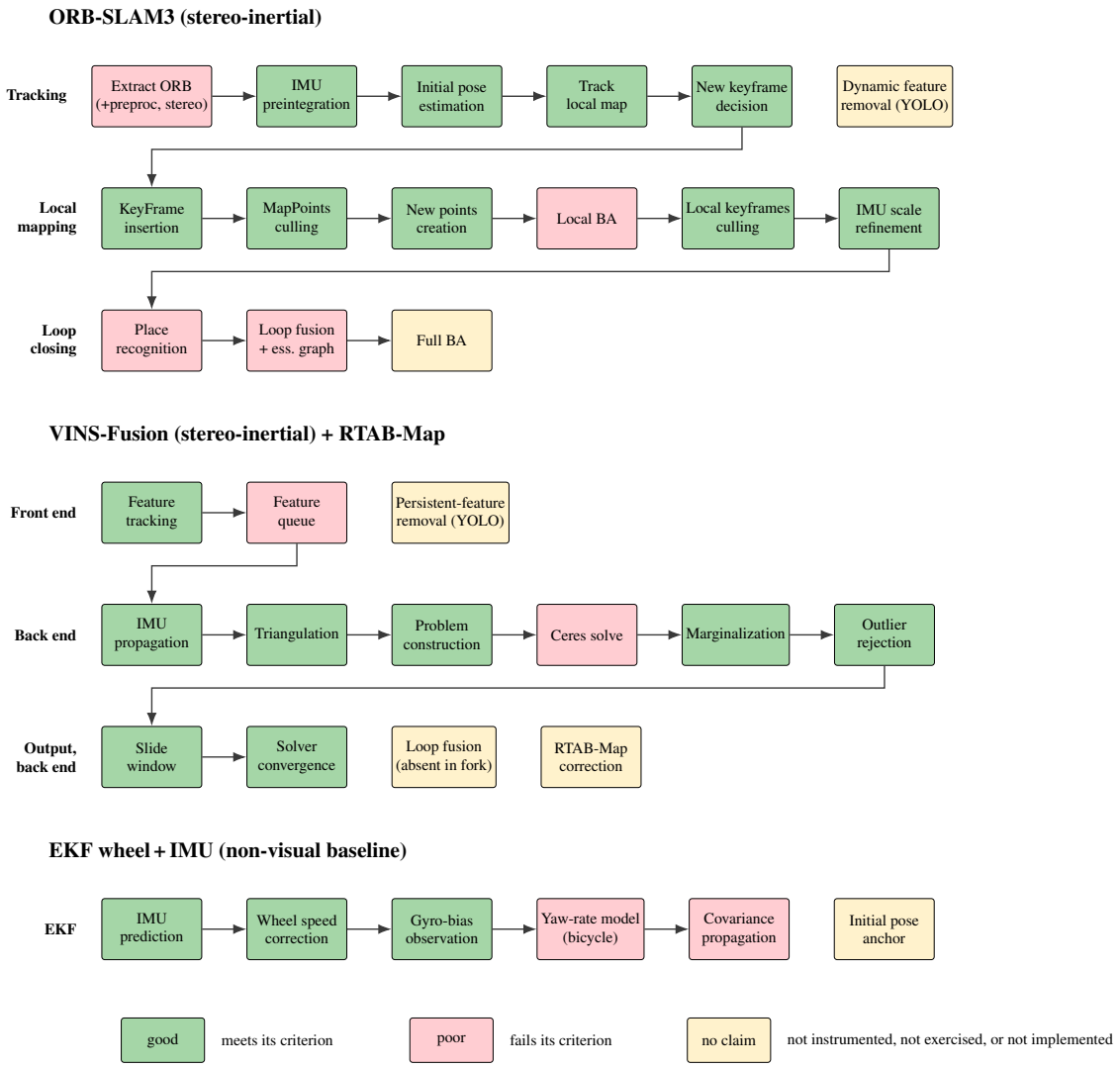


Figure 4.13 Per-block performance in the **static** condition, over the five `allsensor` datasets. The EKF baseline appears here and not in figure 4.14 because no dynamic dataset carries wheel encoders. Evidence for every colour is in tables 4.23 to 4.25, one table per algorithm.

Table 4.23 Per-block evidence for **ORB-SLAM3** in the static condition, figure 4.13. Five `allsensor` datasets, median across runs, loop closing enabled (`logging_20260916_01`).

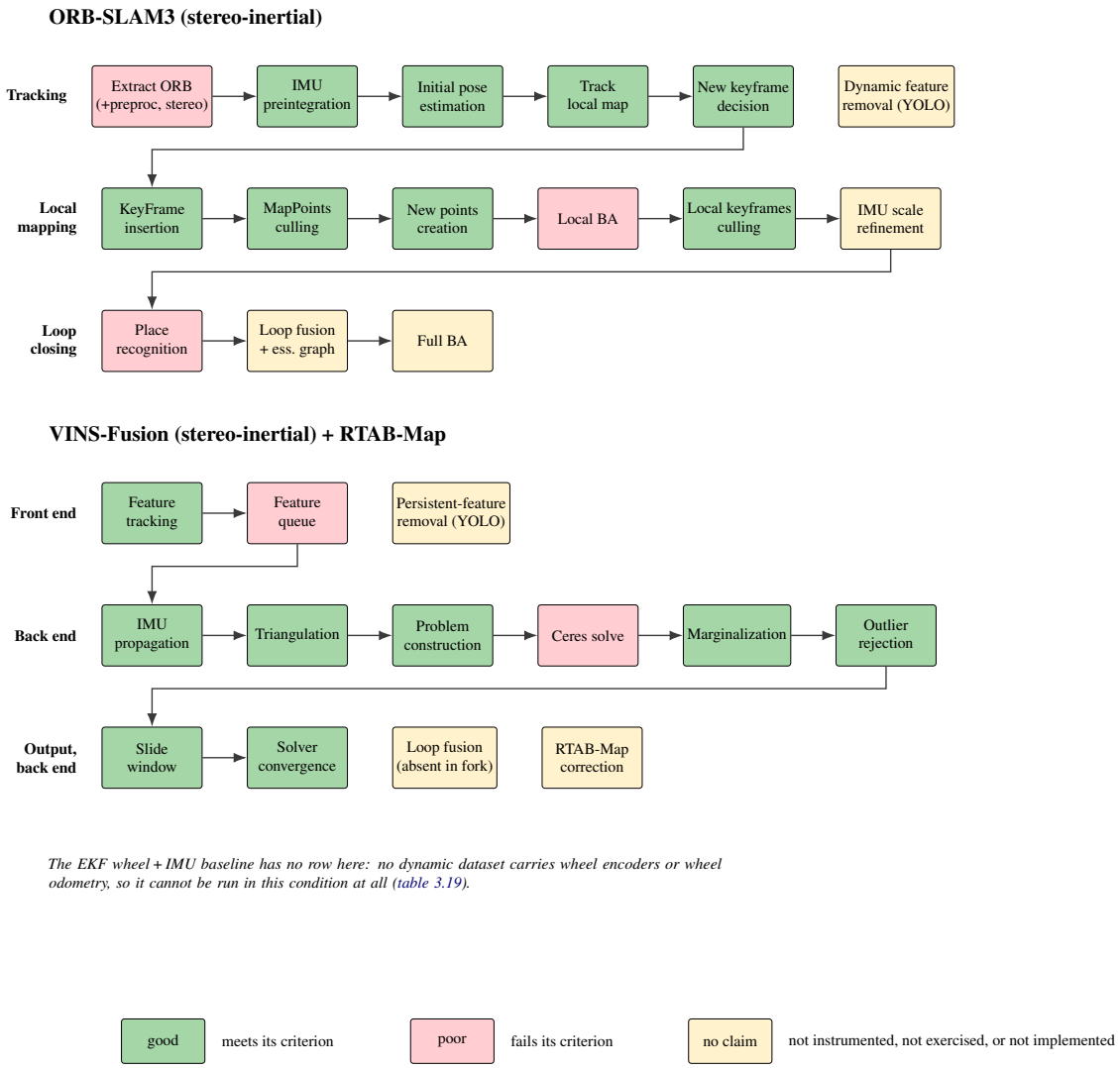
Block	Rule	Metric	Value	Criterion / note	Verdict
Tracking					
Extract ORB (+preproc, stereo)	latency	median image_preprocessing_feature_stereo_ms	33.1	*convention:* <40% of stage (61.4% of stage)	poor
IMU preintegration	latency	median imu_preintegration_ms	0.120	*convention:* <40% of stage (0.2% of stage)	good
Initial pose estimation	latency	median pose_prediction_ms	0.087	*convention:* <40% of stage (0.2% of stage)	good
Track local map	latency	median local_map_tracking_ms	16.5	*convention:* <40% of stage (30.7% of stage)	good
New keyframe decision	latency	median keyframe_decision_ms	0.169	*convention:* <40% of stage (0.3% of stage)	good
TRACKING total	latency	median tracking_total_ms	53.8	*derived:* <66.0 ms frame budget	good
Dynamic feature removal (YOLO)	none	–	–	– (filter disabled in every run reported here)	no claim
Local mapping					
KeyFrame insertion	latency	median keyframe_insertion_ms	10.3	*convention:* <40% of stage (3.3% of stage)	good
Recent MapPoints culling	latency	median map_point_culling_ms	0.383	*convention:* <40% of stage (0.1% of stage)	good
New points creation	latency	median map_point_creation_fusion_ms	41.5	*convention:* <40% of stage (13.3% of stage)	good
Local BA	latency	median local_ba_ms	244.4	*convention:* <40% of stage (78.1% of stage)	poor
Local keyframes culling	latency	median keyframe_culling_ms	10.9	*convention:* <40% of stage (3.5% of stage)	good
IMU initialization	latency	median imu_initialization_ms	0.002	*convention:* <40% of stage (negligible)	good
IMU scale refinement	latency	median inertial_refinement_ms	0.001	*convention:* <40% of stage (negligible)	good
Loop closing					
Place recognition	outcome	candidates accepted	4 of 281	*convention:* >10 % of candidates accepted (2494 events)	poor
Loop fusion + essential graph	ablation	ATE(on)/ATE(off)	1.37	*convention:* <1.0 (enabling it should help) (one run per configuration)	poor
Full BA	outcome	global BA executions	–	– (see table 4.3)	no claim

Table 4.24 Per-block evidence for **VINS-Fusion** in the static condition, figure 4.13. Five `allsensor` datasets, median across runs.

Block	Rule	Metric	Value	Criterion / note	Verdict
Front end					
Feature tracking	latency	median feature_tracking_ms	55.2	<66.0 ms frame budget	good
Feature queue	latency	median feature_queue_wait_ms	403.7	<66.0 ms frame budget (backlog 4.0 frames)	poor
Persistent-feature removal (YOLO)	none	–	–	– (filter disabled in every run reported here)	no claim
Back end					
IMU propagation	latency	median imu_propagation_ms	0.272	*convention:* <40% of stage (0.4% of stage)	good
Triangulation	latency	median triangulation_ms	0.063	*convention:* <40% of stage (0.1% of stage)	good
Problem construction	latency	median problem_construction_ms	3.2	*convention:* <40% of stage (4.7% of stage)	good
Ceres solve	latency	median solver_ms	48.3	*convention:* <40% of stage (71.9% of stage)	poor
Marginalization	latency	median marginalization_ms	4.1	*convention:* <40% of stage (6.1% of stage)	good
Outlier rejection	latency	median outlier_rejection_ms	0.155	*convention:* <40% of stage (0.2% of stage)	good
Slide window	latency	median slide_window_ms	0.212	*convention:* <40% of stage (0.3% of stage)	good
BACK END total	latency	median backend_total_ms	67.2	<66.0 ms frame budget	poor
Solver convergence	outcome	solves returning a usable solution	99.64%	*convention:* ≥99 % usable (22.8 % hit the iteration or 80 ms cap without converging; reported, not graded)	good
Loop closing					
Loop fusion	none	–	–	– (not implemented in this fork)	no claim
RTAB-Map correction	none	–	–	– (separate process; correction never injected)	no claim

Table 4.25 Per-block evidence for the **EKF wheel + IMU** baseline, figure 4.13. Every row but the last two is an ablation: the block was removed or swapped and all five datasets re-run, so these are the only verdicts in this section that speak to trajectory error.

Block	Rule	Metric	Value	Criterion / note	Verdict
EKF					
Wheel speed correction	ablation	ATE without the wheel update / with	54.08	*convention:* ≥2.0× (block is load-bearing) (median over 5 bags; joint with the other row – one flag removes both)	good
Gyro-bias observation	ablation	ATE without the wheel update / with	54.08	*convention:* ≥2.0× (block is load-bearing) (median over 5 bags; joint with the other row – one flag removes both)	good
IMU prediction	ablation	ATE of wheel-only / EKF	6.56	*convention:* ≥2.0× (block is load-bearing) (median over 5 bags)	good
Yaw-rate model (bicycle)	ablation	ATE with differential / with bicycle	1.02	*convention:* ≥2.0× (block is load-bearing) (median over 5 bags)	poor
Initial pose anchor	none	–	–	– (ground-truth pose; not a performance block)	no claim
Covariance propagation	outcome	ANEES (3 DOF)	–	*derived:* 3.0 for a consistent filter (64.7–123.2; see table 4.7)	poor



The EKF wheel + IMU baseline has no row here: no dynamic dataset carries wheel encoders or wheel odometry, so it cannot be run in this condition at all (table 3.19).

Figure 4.14 Per-block performance in the **dynamic** condition, over the twelve relogged dynamic runs of each estimator. Loop closing carries no ablation here because the relogged campaign has no loop-closure-disabled variant. Evidence in tables 4.26 and 4.27, one table per algorithm.

Table 4.26 Per-block evidence for **ORB-SLAM3** in the dynamic condition, figure 4.14. Twelve relogged dynamic runs, median across runs.

Block	Rule	Metric	Value	Criterion / note	Verdict
Tracking					
Extract ORB (+preproc, stereo)	latency	median image_preprocessing_feature_stereo_ms	45.1	*convention:* <40% of stage (73.8% of stage)	poor
IMU preintegration	latency	median imu_preintegration_ms	0.116	*convention:* <40% of stage (0.2% of stage)	good
Initial pose estimation	latency	median pose_prediction_ms	0.055	*convention:* <40% of stage (0.1% of stage)	good
Track local map	latency	median local_map_tracking_ms	12.2	*convention:* <40% of stage (19.9% of stage)	good
New keyframe decision	latency	median keyframe_decision_ms	0.179	*convention:* <40% of stage (0.3% of stage)	good
TRACKING total	latency	median tracking_total_ms	61.2	*derived:* <66.0 ms frame budget	good
Dynamic feature removal (YOLO)	none	–	–	– (filter disabled in every run reported here)	no claim
Local mapping					
KeyFrame insertion	latency	median keyframe_insertion_ms	10.9	*convention:* <40% of stage (5.7% of stage)	good
Recent MapPoints culling	latency	median map_point_culling_ms	0.714	*convention:* <40% of stage (0.4% of stage)	good
New points creation	latency	median map_point_creation_fusion_ms	50.0	*convention:* <40% of stage (26.3% of stage)	good
Local BA	latency	median local_ba_ms	122.9	*convention:* <40% of stage (64.6% of stage)	poor
Local keyframes culling	latency	median keyframe_culling_ms	4.0	*convention:* <40% of stage (2.1% of stage)	good
IMU initialization	latency	median imu_initialization_ms	–	*convention:* <40% of stage (not logged)	no claim
IMU scale refinement	latency	median inertial_refinement_ms	–	*convention:* <40% of stage (not logged)	no claim
Loop closing					
Place recognition	outcome	candidates accepted	0 of 2	*convention:* >10 % of candidates accepted (5726 events)	poor
Loop fusion + essential graph	ablation	ATE(on)/ATE(off)	–	*convention:* <1.0 (enabling it should help) (one run per configuration)	no claim
Full BA	outcome	global BA executions	–	– (see table 4.3)	no claim

Table 4.27 Per-block evidence for **VINS-Fusion** in the dynamic condition, figure 4.14. Twelve relogged dynamic runs, median across runs.

Block	Rule	Metric	Value	Criterion / note	Verdict
Front end					
Feature tracking	latency	median feature_tracking_ms	42.5	<66.0 ms frame budget	good
Feature queue	latency	median feature_queue_wait_ms	502.1	<66.0 ms frame budget (backlog 7.5 frames)	poor
Persistent-feature removal (YOLO)	none	–	–	– (filter disabled in every run reported here)	no claim
Back end					
IMU propagation	latency	median imu_propagation_ms	0.204	*convention:* <40% of stage (0.3% of stage)	good
Triangulation	latency	median triangulation_ms	0.062	*convention:* <40% of stage (0.1% of stage)	good
Problem construction	latency	median problem_construction_ms	1.1	*convention:* <40% of stage (1.7% of stage)	good
Ceres solve	latency	median solver_ms	57.2	*convention:* <40% of stage (89.8% of stage)	poor
Marginalization	latency	median marginalization_ms	2.4	*convention:* <40% of stage (3.7% of stage)	good
Outlier rejection	latency	median outlier_rejection_ms	0.068	*convention:* <40% of stage (0.1% of stage)	good
Slide window	latency	median slide_window_ms	0.167	*convention:* <40% of stage (0.3% of stage)	good
BACK END total	latency	median backend_total_ms	63.7	<66.0 ms frame budget	good
Solver convergence	outcome	solves returning a usable solution	99.55%	*convention:* ≥99 % usable (45.1 % hit the iteration or 80 ms cap without converging; reported, not graded)	good
Loop closing					
Loop fusion	none	–	–	– (not implemented in this fork)	no claim
RTAB-Map correction	none	–	–	– (separate process; correction never injected)	no claim

Four readings follow that the trajectory tables alone do not give.

Both visual front ends are the dominant cost, and neither keeps up cleanly. ORB-SLAM3 spends 33.1 ms of a 53.8 ms tracking budget on preprocessing, extraction and stereo matching in static scenes, rising to 45.1 ms of 61.2 ms in dynamic ones. VINS-Fusion's back end waits 403.7 ms per frame on its feature queue in static scenes and 502.1 ms in dynamic ones, against a 66.0 ms frame budget, with a backlog of four to seven frames. These are the same deficit wearing different clothes: neither estimator processes frames as fast as they arrive, and the two differ in what they do about it. ORB-SLAM3 subscribes best-effort at depth five and discards the overflow, which is the frame shortfall of section 4.3.1; VINS-Fusion subscribes reliably at depth 100 and absorbs it as latency. The frame counters see one and not the other.

Local BA dominates ORB-SLAM3's local mapping at 244.4 ms of 312.9 ms in static scenes. It runs per keyframe rather than per frame, so it does not compete directly with the frame budget, but it is where that pipeline's compute goes.

Loop closing is not earning its place in these scenes. Across the five `allsensor` datasets, enabling it leaves the median aligned ATE 1.37 times *worse* than disabling it, and only 4 of 281 place-recognition candidates were ever accepted (table 4.3). This is the project's one ORB-SLAM3 ablation, and it is still one run per configuration, so it sits inside an unmeasured run-to-run spread—the verdict is reported, not relied upon.

The EKF's accuracy rests on one intervention. Removing the wheel update costs a factor of 54 in aligned ATE; removing the IMU prediction costs 6.6. The 54 covers the wheel speed correction and the gyroscope-bias observation *together*: both are measurement rows of the same update, one flag removes both, and no flag removes only one, so table 4.25 lists the figure against each row while marking it as joint rather than presenting two independent results. Swapping the yaw-rate model from the bicycle formulation to the rear-wheel differential, by contrast, changes nothing measurable—median 1.0, and on `allsensor_004` the noisier estimator is the better one. That is worth stating plainly, because the differential model is four times worse as a *rate* estimator (table 3.23) and dominates standalone wheel odometry's error. Inside the filter it barely matters, because the wheel rate is used only to observe the gyroscope bias and is weighted by its own measured σ. The design choice of section 3.6.1 is vindicated by its own ablation.

The campaign succeeds as an instrumentation milestone: repeated run-level data, front-end and back-end timing, input retention, process resources, initialization, tracking loss, keyframes, loop closures, map merges, and raw trajectories are now available in auditable form. Ground-truth accuracy can now be measured, but the campaign does not support the intended semantic-filter hypothesis or a clean dynamic-scene estimator ranking. The limitations are:

Table 4.28 lists them with the consequence each one carries.

No root cause is assigned in this report. In particular, differences are not attributed to calibration, timestamp handling, masks, replay scheduling, or a specific estimator component without a controlled diagnostic experiment. The immediate advisor decision is whether the next work period should first restore active, input-matched filtering and plausible full trajectories, or instead prioritize the planned integration of relocalization, global optimization, and map merging. The latter remains future work and would provide a unified localization/map input for navigation; path planning itself remains outside scope.

Table 4.28 Limitations of the results reported in this chapter, and what each one prevents. "Scope" identifies the experiment affected: R is the relogged campaign, E the estimator comparison of section 4.1.1, and A both.

Scope	Limitation	Consequence
A	RPE was missing	now implemented and reported (section 4.3.1); it withdrew this chapter's local-coherence reading
R	every dynamic trajectory fails the jump-plausibility criterion	no dynamic estimator ranking
R	the VINS semantic treatment was inactive in all repeats	no VINS filtering conclusion
R	11 of 12 ORB treatment pairs fail the paired-input rule	no cross-treatment accuracy conclusion
R	YOLO truth is projected simulator geometry, not annotated visible extent, and the retained confidence threshold is fixed	no AP claim
R	class metrics are conditional on successful IoU localization	must not be read as end-to-end accuracy
R	two static ORB run summaries have incomplete schemas	retained as operational; counters unavailable
A	CPU and memory are per-estimator-process figures	not whole-system cost
A	one laptop and one replay rate throughout	no hardware or input-rate stress surface
E	replay rate $0.5\times$ for both visual estimators	accuracy figures are not evidence of real-time capability
E	baselines are offline, visual estimators ran live	timing-induced error is present on one side of the comparison only
E	only `allsensor` bags carry wheel encoders	the baseline exists for static featureless-floor scenes only; no dynamic condition
E	$n = 5$ across four worlds, no condition repeated more than twice	no mean, deviation or interval; between-bag differences are not effects
E	ORB-SLAM3 drops frames at a depth-5 best-effort subscriber; VINS buffers 100 deep	input delivery is not matched between the two; no timing comparison is admissible
E	ORB-SLAM3 run-to-run spread unmeasured, one run per configuration	loop-closure differences cannot be attributed; repeats required
E	loop-closure state absent from `run_metadata.csv`	configurations distinguishable only by inspecting `loop_closing.csv`
E	origin of the wheel yaw-rate offset untested	the drift is predicted but not yet attributable or correctable (section 4.1.1)
E	wheel-rate offset is absorbed by the EKF gyroscope-bias state	now quantified: EKF heading error rises tenfold where the wheel rate is poor (table 4.6)
E	accelerometer characterisation unstable across bags	bias 0.019 to 0.068, scale 1.020 to 1.067 for one simulated sensor
E	neither visual estimator writes a pose covariance	the consistency test cannot be applied to them

CHAPTER 5 CONCLUSION AND RECOMMENDATIONS

This project set out to replace infrastructure-based navigation with a vision-based pipeline that stays accurate when the warehouse changes, and to decide between two ways of building it. At the reporting cutoff the pipeline exists in both forms, the four experiments that judge it are defined for both domains, and the four simulation slots hold results. The four real-robot slots do not. The conclusions below are therefore conclusions about the simulated warehouse, and each is bounded by the validity limits recorded beside it in chapter 4.

5.1 Conclusions by Experiment

5.1.1 Experiment 1: is vision earning its place?

On five static recordings, a five-state wheel-plus-gyroscope EKF whose noise model is calibrated per recording localises to 0.19–0.49 m aligned ATE, ahead of stereo visual-inertial SLAM running on a hand-set configuration on four of five recordings (table 4.4). That ordering is not a property of the architectures: giving the filter the visual systems' own inertial noise model degrades it on every recording and puts VINS-Fusion ahead on all five (section 4.1.1). The defensible conclusion is narrower than either reading: over this range the noise model is worth more than the estimator, and the visual systems have never been tuned to this platform. Two results survive the control. The proprioceptive sensors fail in complementary ways, the IMU holding heading and losing position, the wheels the reverse, and the EKF is better than either in both quantities because it observes the gyroscope bias rather than the heading (section 4.1.1). And every dead-reckoning covariance is overconfident by about two orders of magnitude (table 4.7). Against the project's target of 0.05 m ATE, no estimator qualifies; against the 1.5 m RPE target at 1 s, all of them do in static scenes. The comparison is confined to static, encoder-equipped recordings and to a floor on which the wheels effectively cannot slip.

5.1.2 Experiment 2: how good is the detector?

The three-class detector is reliable at naming and unreliable at finding: pooled precision 0.684, moving-object recall 0.591, conditional class F1 0.995 over 9 860 frames (tables 4.10 and 4.11). It misses about 41 % of the moving objects that are scored, and 62 % if every occluded or tiny object is counted; that miss rate is the ceiling on any filtering benefit downstream. The ground truth is projected simulator geometry rather than annotation, which makes the boxes tighter and the scored subset easier than a human-labelled set, and no average precision is reported because a single confidence threshold was retained.

5.1.3 Experiment 3: does filtering help?

The question the project is named for has no valid answer yet. Sixty-six runs exist, fully instrumented, but the VINS-Fusion filter never activated, eleven of twelve ORB-SLAM3 treatment pairs received different input, and the scripted dynamic worlds were found to be an invalid testbed, so every dynamic-scene trajectory is withheld as a finding (section 4.3.1).

This is untested, not tested and failed. The static runs do establish two things: the two front ends reach medians of 0.370 m and 0.454 m aligned ATE, and ORB-SLAM3's eleven-fold rotational penalty is produced by one or two discrete events near inertial initialisation, not by its steady-state tracking (section 4.3.1). Neither front end keeps up with the frame rate at the replay rate used, and they hide that deficit differently, one by dropping frames and one by queueing them (section 4.5.2).

5.1.4 Experiment 4: is the map usable?

Where tracking is sound, the mapping pipeline is: the tiny-warehouse map is 96.6 % precise in what it marks occupied, recovers 89.5 % of the surfaces it observed, and contains no blind cells and 0.7 % proven false obstacles along an independently driven route (tables 4.21 and 4.22). The whole-building recall of 67.2 % is route coverage, not mapping failure. The drivability half of that result is dated, because its independent route is lost. On the large warehouse the attempt failed through tracking loss in repetitive structure, before the back end was exercised, and its artifacts were not retained.

5.2 Limitations and Threats to Validity

The limitations of table 4.28 carry over without softening. The ones that bound the conclusions above most tightly are these. Every result is from one laptop at one replay rate, so nothing here is evidence of real-time capability on the NRU-51V. The proprioceptive baseline exists only for static, encoder-equipped, near-slip-free recordings, with $n = 5$ across four worlds and no condition repeated more than twice. ORB-SLAM3's run-to-run spread is unmeasured and its loop-closure configuration is absent from the run manifest, so its loop-closure results cannot be attributed. The detector is scored against projected geometry with exemptions the training convention lacks. The dynamic worlds move props kinematically and so do not pose the problem the project poses. And no real-robot slot is filled: the rig's calibration has a placeholder translation and a reused intrinsic set, and no recording carries motion-capture truth, wheel odometry or varied illumination.

5.3 Recommendations and Next Steps

Fill the four real-robot slots, in the order the simulation results suggest. First calibrate the real camera–IMU pair and record the motion-capture reference with the wheel and steering topics bridged, which unlocks Experiments 1 and 3 at once and yields the first measured noise model of the real IMU. Then annotate a frame subset under the three illumination levels for Experiment 2, and survey the site into a reference grid for Experiment 4. The integrated live test of section 3.6.3, both pipelines at $1.5\,\mathrm{m\,s^{-1}}$ under scheduled lighting change with the dynamic variable triggered at each zone transfer, is the run that turns the eight-slot table into a comparison.

Give Experiment 3 a valid test before repeating it. Repair the detection-to-VINS integration and prove it by the removed-track counter; input-match the ORB-SLAM3 treatment pairs by reducing filtered-mode latency; rebuild the dynamic worlds with physically simulated props; and record the `allsensor` topics in a dynamic world so that the proprioceptive baseline extends to the condition that matters.

Tune the visual systems to the platform. The cheapest remaining improvement is a

measured inertial noise model for the visual estimators; the one attempt so far was bimodal, better on four of five recordings and divergent on three of ten runs, and needs a proper stationary recording behind it.

Record every behaviour-changing switch in the run manifest, repeat every nondeterministic configuration at least three times, and retain the artifacts of failed attempts; the large-map failure and the lost drivability route are both results the project cannot presently audit.

Future direction. Integrated relocalization against a saved map, global pose-graph optimisation with loop-closure constraints, and alignment and merging of maps across sessions remain future work, to be taken up only once the front end tracks reliably in repeated structure; path planning stays outside the project.

REFERENCES

[1] Saputra, M.R.U., Markham, A. and Trigoni, N., 2018, "Visual SLAM and Structure from Motion in Dynamic Environments: A Survey," **ACM Computing Surveys**, Vol. 51, No. 2, pp. 1–36.

[2] Cadena, C., Carlone, L., Carrillo, H., Latif, Y., Scaramuzza, D., Neira, J., Reid, I. and Leonard, J.J., 2016, "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age," **IEEE Transactions on Robotics**, Vol. 32, No. 6, pp. 1309–1332.

[3] BMW Group, 2024, **Autonomous Driverless Transport Vehicle Navigates Precisely through BMW Group Plant Regensburg Press Plant** [Online], Available: https://www.press.bmwgroup.com/global/video/detail/PF0009726/ [2026, September 25].

[4] Campos, C., Elvira, R., Gómez Rodríguez, J.J., Montiel, J.M.M. and Tardós, J.D., 2021, "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multi-Map SLAM," **IEEE Transactions on Robotics**, Vol. 37, No. 6, pp. 1874–1890.

[5] Qin, T., Li, P. and Shen, S., 2018, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," **IEEE Transactions on Robotics**, Vol. 34, No. 4, pp. 1004–1020.

[6] Qin, T. and Shen, S., 2018, "Online Temporal Calibration for Monocular Visual-Inertial Systems," **2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)**, 1–5 October 2018, Madrid, pp. 3662–3669.

[7] Labbé, M. and Michaud, F., 2019, "RTAB-Map as an Open-Source Lidar and Visual Simultaneous Localization and Mapping Library for Large-Scale and Long-Term Online Operation," **Journal of Field Robotics**, Vol. 36, No. 2, pp. 416–446.

[8] Chen, X., Li, S., Ren, S. and Wang, J., 2025, "RY-SLAM: A Robust YOLO-Based Real-Time Visual SLAM Solution for Dynamic Environments," **2025 International Conference on Virtual Reality and Visualization (ICVRV)**, IEEE.

[9] ASEAN Secretariat and AFFA Logistics Institute, 2019, "Chapter 3: Warehouse Layout, Storage, Handling & MHE," **Development of the Common Standard Curricula on International Transport and Logistics Basic Training for ASEAN Member States**, training material, 57 slides.

[10] Mecalux, n.d., **Warehouse Layout: Warehouse Design** [Online], Available: https://www.mecalux.com/warehouse-manual/warehouse-design/warehouse-layout [2026, October 6].

[11] Mobile Industrial Robots, n.d., **Eaton Case Study with MiR AMRs** [Online], Available: https://mobile-industrial-robots.com/cases/eaton [2026, October 8].

[12] Kanso, H., Singh, A., El Zarif, E., Almohammed, N., Mounsef, J., Maalouf, N. and Arain, B., 2025, "Semantic SLAM: A Comprehensive Survey of Methods and Applications," **Intelligent Systems with Applications**, Vol. 28, 200591.

[13] Keith, R. and La, H.M., 2024, "Review of Autonomous Mobile Robots for the Warehouse Environment."

CURRICULUM VITAE

NAME	Mr. Bolton Athit Davies
Date of Birth	[to be completed]
EDUCATIONAL RECORD	
HIGH SCHOOL	[school, year — to be completed]
BACHELOR'S DEGREE	Bachelor of Engineering (Robotics and Automation Engineering), Institute of Field Robotics, King Mongkut's University of Technology Thonburi, 2026
SCHOLARSHIP/ RESEARCH GRANT	[to be completed, or "—"]
EMPLOYMENT RECORD	Work-integrated learning placement, Gensurv Robotics, 2026
PUBLICATION	—